

第一章 信息论与编码理论概述

李丽香

北京邮电大学网络空间安全学院

编码机制

- **编码**：信息从一种形式或格式转换为另一种形式的过程。
- **编码机制**：用预先规定的方法将文字、数字或其它对象编成数码，或将信息、数据转换成规定的电脉冲信号。

信源编码

- **目的**：提高通信的有效性，即以更少的符号来表示原始消息
- **设计机制**：根据消息符号序列的统计特性来设计各种方法，把消息序列变换为较短的码字序列，同时又能保证无失真地恢复原来的消息序列
- 简而言之，就是通过减少消息符号序列中的冗余度，提高平均信息量
- 信源编码就是通常所说的数据压缩

随机错误

- **随机错误**：接收序列中的传输错误是随机出现的。
- **随机错误信道**：这样的信道称为随机错误信道。
- 常见于无记忆信道中，因为噪声随机独立地影响每个传输信号。
- 深空通信和卫星通信信道都是典型的随机错误信道。

突发错误

- 由于噪声和干扰或读写头接触不良所引起的错误，往往不是随机错误，而是成群成串出现的错误。
- **突发错误**：不同状态特性下，错误出现的概率不一样，这种错误为突发错误。
- 突发错误的例子：无线通信(由多径传输引起的信号衰落造成的错误)，有线和电缆传输(开关脉冲和串话的影响)，磁带记录(涂层缺损和灰尘引起的带脱落)。
- **突发错误信道**：由于信道状态特性变化较大，进而有突发错误信道的概念。这种产生比较密集错误的信道称为突发错误信道或者有记忆信道。
- 突发错误常出现在有记忆信道中，此时噪声对各次传输的影响是彼此相关的。

纠错码

- **目的**：检测和纠正由于非人为(自然)因素引起的传输错误。
- 即提高通信的可靠性
- 纠错编码主要用于实现信道纠错，又叫做信道编码或者差错控制编码。
- **设计机制**：通过对消息符号序列加入冗余信息，使得接收方收到信号后，可以通过这些冗余信息进行检错和纠错
- 本课程主要讲述各种纠错码

纠错码的分类

- **纠随机错误码**：为纠正随机错误而设计的码称为纠随机错误码。
- **纠突发错误码**：为纠正突发错误而设计的码称为纠突发错误码。

数字通信系统模型

所有的数字通信系统，如雷达、遥控遥测、数字存储系统、计算机等，都可以归结为下图所示的模型：

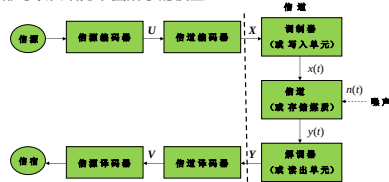


图1 数字通信(传输)系统模型

数字通信系统的简化模型

编码理论关注的是信道编码器和信道译码器。为集中考虑信道编码与译码，将图1的模型简化为：

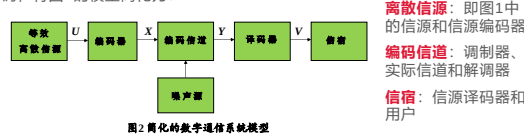


图2 简化的数字通信系统模型

$$\begin{aligned} U &= [u_1, u_2, \dots, u_K] & u_k \in U = \{a_1, a_2, \dots, a_Q\} & & X = [x_1, x_2, \dots, x_N] & & k = 1, 2, \dots, K \\ V &= [v_1, v_2, \dots, v_K] & v_k \in V = \{a_1, a_2, \dots, a_Q\} & & Y = [y_1, y_2, \dots, y_N] & & n = 1, 2, \dots, N \end{aligned}$$

信道模型

设计通信系统的关键问题之一：如何解决可靠性和有效性(快速性)这对矛盾？

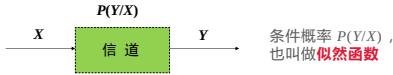


图3 信道模型

$$\begin{aligned} Y &= [y_1, y_2, \dots, y_N] & k &= 1, 2, \dots, K \\ X &= [x_1, x_2, \dots, x_N] & n &= 1, 2, \dots, N \end{aligned}$$

无记忆信道和有记忆信道

无记忆信道：如果在给定时间间隔上，检测器的输出只与在该时间间隔上传送的信号有关，而与任何前面时间的传送的信号无关，称此信道为无记忆信道。该信道可以用如下公式表示：

$$P(Y/X) = \prod_n p(y_n/x_n)$$

有记忆信道：一种M元输入、Q元输出的信道模型。该信道可以用如下公式表示：

$$P(y_n/x_{n-K+1}, \dots, x_{n-1}, x_n)$$

离散无记忆(DMC)信道

是一种M元输入、Q元输出的信道模型。

条件概率P(Y/X)是描述该信道的最好方式，也叫转移概率，其中X表示调制器输入符号，Y表示解调器输出符号，P(Y/X)表示发送为X接收为Y的概率。

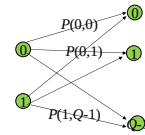


图4 二元输入Q元输出的离散无记忆信道模型

二元对称(BSC)信道

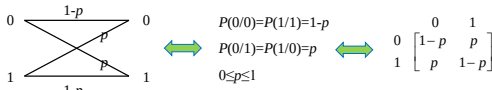


图5 二元对称信道模型

信道矩阵：一个信道也可由它的转移概率组成的矩阵来表示。

信道矩阵

一般离散单符号信道的信道矩阵为：

$$\begin{matrix} & \begin{matrix} b_1 & b_2 & \cdots & b_s \end{matrix} \\ \begin{matrix} a_1 \\ a_2 \\ \vdots \\ a_r \end{matrix} & \begin{bmatrix} P(b_1|a_1) & P(b_2|a_1) & \cdots & P(b_s|a_1) \\ P(b_1|a_2) & P(b_2|a_2) & \cdots & P(b_s|a_2) \\ \vdots & \vdots & \ddots & \vdots \\ P(b_1|a_r) & P(b_2|a_r) & \cdots & P(b_s|a_r) \end{bmatrix} \end{matrix}$$

加性高斯白噪声(AWGN)信道

- **加性噪声：**叠加在信号上的一种噪声。
- **白噪声：**噪声的功率谱密度在所有的频率上均为一常数。
- **高斯白噪声：**白噪声取值的概率分布服从高斯分布，其自相关系数为无延时的冲击函数。

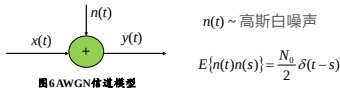


图6 AWGN信道模型

《编码理论》课程

第二章 信息论基础与信源编码理论

李丽香

北京邮电大学网络空间安全学院

信息的概念

- 1948年，仙农发表了一篇重要的论文《通信的数学理论》，对信息作了科学的定义。
- 先请读几个句子：
 - 明天，太阳从东方升起。
 - 电话铃声将会在一个小时之内响。
 - 这个冬天，哈尔滨将下雨。
- 一个句子中所含信息的多少，同句子中所表达的事件出现的概率有关，呈现相反的关系。
- 仙农对于**信息的定义**：**信息是事物运动状态或存在方式不确定性的描述。**

自信息

- **定义：**考虑离散随机变量X，其样本空间为{x_i, i=1, 2,...,n}，则事件X=x_i的自信息的定义为
$$I(x_i) = \log\left(\frac{1}{P(x_i)}\right) = -\log P(x_i)$$
- 自信息的单位由对数的底来决定，以2为底，单位就是比特(bits)，以e为底就是奈特(nats)。自信息非负。

例子：考虑一个可以同时输出2个比特的离散无记忆的信源C。该信源有4种可能的输出{00, 01, 10, 11}，其中每一种可能的输出都是独立不相关的，其出现的概率为P(C)=0.25。因此，信源C的每个可能的输出的自信息为I(C)=-log₂P(x_i)=-log₂0.25=2bits

条件自信息

• **定义：** 在给定 $Y=y_i$ 的情况下，考虑离散随机变量 X ，其样本空间为 $\{x_i, i=1, 2, \dots, n\}$ ，则事件 $X=x_i$ 的条件自信息量的定义为

$$I(x_i|y_i) = \log\left(\frac{1}{P(x_i|y_i)}\right) = -\log P(x_i|y_i)$$

其中， $P(x_i|y_i)$ 是 x_i 的条件概率，也称为后验概率。 $P(x_i)$ 称为先验概率。

• 条件自信息量非负。

互信息

• **定义：** x_i 和 y_j 之间的互信息定义为 $I(x_i; y_j) = \log\left(\frac{P(x_i|y_j)}{P(x_i)}\right)$

• 互信息的性质：

- 性质1：互易性 $I(x_i; y_j) = I(y_j; x_i)$ 。
- 性质2：当 x_i 和 y_j 是统计上独立的，即 $P(x_i|y_j) = P(x_i)$ ，则 $I(x_i; y_j) = 0$ 。
- 性质3：互信息量可以是正的，也可以是负的。

思考题：计算二元对称(BSC)信道的互信息量？

平均自信息量(信息熵)

• **定义：** 离散随机变量 X 的平均自信息定义为，其样本空间为 $\{x_i, i=1, 2, \dots, n\}$ ，则事件 $X=x_i$ 的平均自信息量的定义为

$$H(X) = E\left[\log\left(\frac{1}{P(x_i)}\right)\right] = \sum_{i=1}^n P(x_i) I(x_i) = -\sum_{i=1}^n P(x_i) \log P(x_i)$$

其中 $H(X)$ 表示每个信源符号的平均信息量。

例子：变量 X ， $P(X=x_1)=0.99$ ， $P(X=x_2)=0.01$ ，变量 Y ， $P(Y=y_1)=0.5$ ， $P(Y=y_2)=0.5$ ，则信息熵为 $H(X)=-0.99\log 0.99-0.01\log 0.01=0.08$ (比特/符号)， $H(Y)=-0.5\log 0.5-0.5\log 0.5=1$ (比特/符号)。

平均条件自信息量(条件熵)

• **定义：** 平均条件自信息 $H(X|Y)$ 定义为

$$H(X|Y) = \sum_{i=1}^n \sum_{j=1}^m P(x_i, y_j) \log \frac{1}{P(x_i|y_j)}$$

思考题：计算二元对称(BSC)信道的平均条件自信息量(条件熵)？

等长信源编码定理

• **定理：** 一个熵为 $H(X)$ 的离散无记忆信源，若对信源长为 N 的符号序列进行等长编码，设码字是从 r 个字母的码符号集合中，选取 l 个码元组成。对于任意的 $\epsilon > 0$ ，只要满足

$$\frac{l}{N} \geq \frac{H(X) + \epsilon}{\log r}$$

则当 N 足够大时，可实现几乎无失真编码，即译码错误概率可为任意小。

变长码

表1 译码不唯一的变长码例子

字母	码字	字母	码字
A	0	E	10
B	1	F	11
C	00	G	000
D	01	H	111

- 若对字母序列ABADCAB进行编码，则得到0 1 0 01 00 0 1
- **译码不唯一：** 可能被译为 DADAAAB(01 0 01 0 0 0 1) 或者AEDGB(0 10 01 000 1)

注：变长码必须是惟一可译码，才能实现无失真编码。

纠错编码的理论基础

●**香农编码定理**：如果系统的传输率小于信道容量，那么适当选择编码技术就能实现可靠通信，即将差错率减小到任意小的程度。更确切地，每个信道都具有固定的信道容量 C ，对任何小于 C 的信息传输率 R ，存在一个码长为 n 码率为 R 的分组码，若用最大似然译码，则其译码错误概率为 $P_e \leq Ae^{-nK_0(R)}$ 。对于码率为 R 约束长度为 n 的卷积码，其译码错误概率也有类似的关系，即 $P_e \leq Be^{-nK_0(R)}$ ，其中 A 和 B 都为大于0的数， $E_b(R)$ 和 $E_c(R)$ 为正实函数，叫做误差指数。

31

科学哲学：过程比结果更重要(课堂思政)

- 香农编码定理解决了纠错码的存在性问题，虽然仙农当时未能构造出一个纠错码，但是从此之后纠错编码的研究与应用开始了前所未有的发展，人们一直在不停的努力解决构造性问题。为了接近信息传输率的上限，已经提出了很多的纠错编码技术。
- 我们从香农(1916-2001)生平故事中得到一个启示：富有创造性的富有成果的生活也可以是快乐的生活。

科学哲学家 史蒂文·沃德·艾利斯：科学是一个过程



2016年11月11日，荷兰人史蒂文·沃德·艾利斯在阿姆斯特丹去世，享年91岁。他是一个开明、一个有创造力的人。他对人类知识完全开放，他认为知识是开放的，知识是开放的，知识是开放的。他相信这个世纪，知识是开放的，知识是开放的，知识是开放的。

仙农的主要贡献：香农定理



2016年11月11日，美国，仙农的主要贡献是提出了信息论理论。他在1948年发表了《通信的数学原理》一书，提出了“熵”、“信道容量”等概念，奠定了现代通信理论的基础。他的主要贡献是提出了信息论理论。他在1948年发表了《通信的数学原理》一书，提出了“熵”、“信道容量”等概念，奠定了现代通信理论的基础。

32

信道编码定理

●**定理**：假设DMS有信源字符集 X ，熵为每信源符号 $H(X)$ 比特，而且信源每 T_s 秒产生一个符号，那么信源的平均信息率为每秒 $H(X)/T_s$ 比特，假设信道可以每 T_c 秒使用一次，而信道容量为每次信道使用 C 比特，那么每单位时间的信道容量为每秒钟 C/T_c 比特。如果 $H(X)/T_s \leq C/T_c$ ，那么就存在编码方案使得在有噪声的信道上传输的信源消息，能够以任意小的错误概率进行恢复。

例子：考虑BSC信道，信道容量 $C=1+p\log 2p+(1-p)\log 2(1-p)$ ，如果转移概率 $p=10^{-2}$ ，则可计算出 $C=0.919$ ，那么就可以知道肯定存在信息传输率为 $r<0.919$ 的编码方案，使得信息可以在有噪声的信道上传输的信源消息，能够以任意小的错误概率进行恢复。

33



北京邮电大学

BEDING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第三章 线性分组码

李丽香, 彭海朋

北京邮电大学网络空间安全学院

网络与交换技术国家重点实验室





北京邮电大学

课堂思政



1986年3月7日, 汉明在贝尔通信研究中心给200多名科学家们, 做了一次演讲, 题目:《你和你的研究》。他试图回答下面这样一个问题:“**为什么有的科学家做出了影响深远的重大成果, 而大多数其他人的成果都被历史遗忘了?**”这篇演讲非常有名, 每一个从事创造性工作的人, 都应该读一下。

要有做大事的想法
年轻的时候就要有勇气追求答案
忍受不确定的状态
比别人多努力10%
全力投入重要的问题

敞开办公室大门
做可以成为他人基石的工作
重视推销和表达自己
阅读方法比数量更重要
重视书写
远见超出你的能力时, 才去做管理

数学是一种有趣的智力运动, 但不允许它阻碍获取关于物理过程的有效信息。——Richard W. Hamming

Page 35



北京邮电大学

分组码

分组码: 将消息序列分组进行编码

每组消息 u 有 k 个信息位, 共有 2^k 个不同的消息

码字: 编码器按照一定规则将每个输入消息 u 转换成二元 n 重 v , $n > k$, 这个二元 n 重 v 称作消息 u 的码字或码矢。

所有 2^k 个码字组成的集合称作是分组码

思考题: 为什么在分组码中码字的个数是 2^k 个?

Page 36



北京邮电大学

线性分组码

线性分组码即具有线性性质的分组码

定义: 长为 n , 有 2^k 个码字的分组码, 当且仅当其 2^k 个码字构成 $GF(2)$ 上所有 n 重向量空间的一个 k 维子空间时, 称作**线性 (n, k) 分组码**。

表1 一个 $(7,4)$ 线性分组码的例子

message	Code words
(0000)	(0000000)
(1000)	(1101000)
(0100)	(0110100)
(1100)	(1011100)
(0010)	(1110010)
(1010)	(0011010)
(0110)	(1000110)
(1110)	(0100110)
(0001)	(1010001)
(1001)	(0110001)
(0101)	(1100101)
(1101)	(0000101)
(0011)	(0100011)
(1011)	(0010111)
(0111)	(1101111)
(1111)	(0001111)

Page 37



北京邮电大学

生成矩阵

思考题: 在线性分组码中, 如何进行编码?

$\therefore$ 线性 (n, k) 分组码 C 是一个 k 维子空间
 $\therefore$ 在码 C 中找到 k 个线性独立的码字 $g_0, g_1, \dots, g_{k-1}$, 使得 C 中的每个码字 v 都是这 k 个码字的一种线性组合, 即
$$v = u_0g_0 + u_1g_1 + \dots + u_{k-1}g_{k-1}$$

生成矩阵: 将 k 个线性独立的码字作为行, 得到 $k \times n$ 阶矩阵

$$G = \begin{pmatrix} g_0 \\ g_1 \\ \vdots \\ g_{k-1} \end{pmatrix} = \begin{bmatrix} g_{00} & g_{01} & \cdots & g_{0n-1} \\ g_{10} & g_{11} & \cdots & g_{1n-1} \\ \vdots & \vdots & \ddots & \vdots \\ g_{k-10} & g_{k-11} & \cdots & g_{k-1n-1} \end{bmatrix}$$

Page 38



北京邮电大学

一致校验(监督)矩阵

$\therefore (n, k)$ 线性码有 $r(r = n - k)$ 个校验码元
 $\therefore$ 必须有 r 个独立的线性方程

对于任一码字 c , 有 $cH^T = 0$
 $\therefore c = iG, \therefore GH^T = 0$

校验矩阵: 该矩阵由 r 行 n 列组成, 即

$$H = \begin{bmatrix} h_{1,0} & h_{1,1} & \cdots & h_{1,n-1} \\ h_{2,0} & h_{2,1} & \cdots & h_{2,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ h_{r,0} & h_{r,1} & \cdots & h_{r,n-1} \end{bmatrix}$$

$$\begin{matrix} cH^T = 0 \\ GH^T = 0 \end{matrix}$$

一致校验矩阵的标准形式: 经过初等变换, 得到 $H = [Q \ I_r]$

Page 39

1

定义：若 (n,k) 线性分组码 C 的生成矩阵形如 $G=[P \ I_k]$ 或 $G=[I_k \ P]$ ，此时称 C 为线性系统分组码。

$$G = \begin{pmatrix} g_0 \\ g_1 \\ \vdots \\ g_{k-1} \end{pmatrix} = [P \ I_k] = \begin{pmatrix} p_{00} & p_{01} & \cdots & p_{0,n-k-1} & \vdots & 1 & 0 & 0 & \cdots & 0 \\ p_{10} & p_{11} & \cdots & p_{1,n-k-1} & \vdots & 0 & 1 & 0 & \cdots & 0 \\ p_{20} & p_{21} & \cdots & p_{2,n-k-1} & \vdots & 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ p_{k-1,0} & p_{k-1,1} & \cdots & p_{k-1,n-k-1} & \vdots & 0 & 0 & 0 & \cdots & 1 \end{pmatrix}$$

此时，如果用任意消息 u 同 G 做乘法，就会发现每个码字都可以被分成两个部分：消息部分和冗余部分。

Page 40

对于线性系统分组码

$$v = u \cdot G = (u_0, u_1, \dots, u_{k-1}) \cdot \begin{pmatrix} p_{00} & p_{01} & \cdots & p_{0,n-k-1} & \vdots & 1 & 0 & 0 & \cdots & 0 \\ p_{10} & p_{11} & \cdots & p_{1,n-k-1} & \vdots & 0 & 1 & 0 & \cdots & 0 \\ p_{12} & p_{21} & \cdots & p_{2,n-k-1} & \vdots & 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ p_{k-1,0} & p_{k-1,1} & \cdots & p_{k-1,n-k-1} & \vdots & 0 & 0 & 0 & \cdots & 1 \end{pmatrix}$$

对应于消息 $u=(u_0, u_1, \dots, u_{k-1})$ 的码字是 $v=(v_0, v_1, \dots, v_{n-1})=u \cdot G$ ，即

$$v_{n-k+i} = u_i, \quad 0 \leq i \leq k-1$$

$$v_j = u_0 p_{0j} + u_1 p_{1j} + \cdots + u_{k-1} p_{k-1,j}, \quad 0 \leq j \leq n-k-1$$

上面两个式子正好反映系统的组成特性，最后这 $n-k$ 个方程称为码 C 的一致校验方程。

Page 41

定义：令 S 是 V_n 的 k 维子空间，并令 S_d 是 V_n 中这样向量的集合，即对 S 中的任意 u 和 S_d 中的任意 v ，有 $u \cdot v = 0$ ，则 S_d 也是 V_n 的一个子空间，它称为 S 的对偶空间或零化空间。

定理：令 S 是 V_n 中的一个 k 维子空间，则 S 的对偶空间的维数是 $n-k$ 。

对偶码：以 H 为生成矩阵得到的 $(n, n-k)$ 码称为码 C 的对偶码，记为 C_d 。

Page 42

若码 C 的生成矩阵具有系统形式 $G=[P \ I_k]$ 或 $[I_k \ P]$ ，则其一致校验矩阵形如 $H=[I_{n-k} \ P^T]$ 或 $H=[P^T \ I_{n-k}]$ 。

举例：某个 $(7,3)$ 系统线性码的生成矩阵和一致校验矩阵为

$$G = \left[\begin{array}{ccc|ccc} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{array} \right] \quad H = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{array} \right]$$

有的书中把 $G=[I_k \ P]$ 称为标准型生成矩阵。

Page 43

设 (n,k) 线性分组码 C 的生成矩阵为 G ，其校验矩阵为 H

令 $v=(v_0, v_1, \dots, v_{n-1})$ 是经有扰信道传送的码字

$r=(r_0, r_1, \dots, r_{n-1})$ 是信道输出端的接收向量

称矢量和 $e=v+r=(e_0, e_1, \dots, e_{n-1})$ 为错误矢量

伴随式：当接收到 r 后，译码器计算下述 $n-k$ 重
 $S=r \cdot H^T=(s_0, s_1, \dots, s_{n-k-1})$
 则称 S 为 r 的伴随式。

当且仅当是一个码字(即无传输错误)时有 $S=0$ ，否则错误矢量本身就是一个码字，此时出现了不可检错误。只要码 C 设计适当，就几乎不会出现不可检错误。

Page 44

根据伴随式定义，我们有

$$S=r \cdot H^T=(v+e) \cdot H^T=v \cdot H^T+e \cdot H^T=e \cdot H^T$$

展开后，可以得到

$$s_0 = e_0 + e_{n-k} p_{0,n-k-1} + e_{n-k+1} p_{0,n-k-2} + \cdots + e_{n-1} p_{0,k-1}$$

$$s_1 = e_1 + e_{n-k} p_{1,n-k-1} + e_{n-k+1} p_{1,n-k-2} + \cdots + e_{n-1} p_{1,k-1}$$

$$\vdots$$

$$s_{n-k-1} = e_{n-k-1} + e_{n-k} p_{n-k-1,n-k-1} + e_{n-k+1} p_{n-k-1,n-k-2} + \cdots + e_{n-1} p_{n-k-1,k-1}$$

可以看出，伴随式是错误图样的组合，即伴随式包含了一定程度的错误图样信息，因而可以用来纠错——伴随式纠错。

Page 45

上述方程有 2^n 个解，即对同一个伴随式，存在 2^n 个可能的错误图样，因此，上述方程虽然包含错误图样的信息，但是实际应用起来纠错困难。

实际中的纠错，都是如何有效地从这些错误图样中选取真正的错误图样，从而得到正确的发送矢量。对于BSC信道，最可能的错误图样是非零数字最少的那个。

Page 48

考虑某(7,3)码，该码的生成矩阵 G 和 H 为

$$G = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}, H = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

令 $u=(101), v=(1001101)$ 是发送码字， $r=(0001101)$ 是接收矢量，收到 r 后首先计算 $S=rH^T=(1000) \neq 0$ ，由伴随式和错误图样的关系方程有：
 $1=e_0+e_2+e_6, 0=e_1+e_4+e_6, 0=e_2+e_4+e_5+e_6, 0=e_3+e_4+e_5$ ，则得到 $2^4=8$ 个错误图样
 $(1000000), (1010111), (1101011), \dots$ ，其中 (1000000) 是非零分量最少的图样，考虑BSC信道，则 (1000000) 是最为可能的错误矢量，因而确定
 $v=r+e=(0001101)+(1000000)=(1001101)$

Page 49

汉明重量：令 $v=(v_0, v_1, \dots, v_{n-1})$ 是二元 n 重， v 的汉明重量 $w(v)$ 定义为 v 中非零分量的个数。

举例：设有两个码字 $\{0100, 1111\}$ ，则它们的汉明重量为
 $w(0100)=1, w(1111)=4$

重量分布：对于码长为 n 的一个分组码，不同的码字可能具有相同的汉明重量，若记 A_i 为该码中汉明重量为 i 的码字个数，称 $\{A_0, A_1, \dots, A_n\}$ 是该码的重量分布。

Page 48

定义：令 $u=(u_0, u_1, \dots, u_{n-1})$ 和 $v=(v_0, v_1, \dots, v_{n-1})$ 是两个二元 n 重，则 u 和 v 之间的汉明距离 $d(u, v)$ 定义为 $u+v$ 的汉明重量。

举例：设有两个码字 $\{0100, 1111\}$ ，则它们的汉明距离
 $d(0100, 1111)=3$

汉明距离的性质：

- 1) 非负性： $d(u, v) \geq 0$;
- 2) $d(u, v)=0$ ，当且仅当 $u=v$ 的时候；
- 3) 对称性： $d(u, v) = d(v, u)$ ；
- 4) 三角不等式： $d(u, v) + d(v, w) \geq d(u, w)$

Page 49

定义：在 (n, k) 码中，任意两个码字之间都有一个汉明距离，两组不同码字之间可能有相同的汉明距离。若记 $D_i (0 \leq i \leq n)$ 为距离为 i 的码字组数，那么称 $\{D_0, D_1, \dots, D_n\}$ 为此分组码的距离分布，并且称能够使 $D_i \neq 0$ 的那个最小整数 i 为该码的最小码间距离。

特别的，对于线性分组码，有如下定理：

定理： (n, k) 线性码的最小码间距离等于非零码字的最小汉明重量。

Page 50

分组码的最小码间距离是决定该码纠错和检错能力的重要指标

定理：设 (n, k) 线性码 C 的最小码间距离为 d ，则1)若 $d \geq t+1$ ，则码 C 能检测 t 个随机错误；2)若 $d \geq 2t+1$ ，则码 C 能纠正 t 个随机错误；3)若 $d \geq t+e+1$ ，则码 C 能纠正 t ($t \leq e$)个随机错误，同时还能检测 e 个随机错误。

思考题：分组码的最小汉明距离同该码的纠错和检错能力有什么关系？

Page 51

定理： (n,k,d) 线性码C的最小码间距离 d 满足 $d' \leq n-k+1$ 。

该定理给出了 d 的上界。

定义：若 (n,k) 线性码的最小码间距离 d 满足 $d=n-k+1$ ，那么称该码为最大距离可分码，简称MDS码。

Page 52

令 (n,k) 线性码C的所有码字是接收矢量 r 是一个 n 重，则按如下方式构造码C的标准阵：

$$\begin{bmatrix} V_1 = 0 & V_2 & \cdots & V_i & \cdots & V_{p'} \\ e_2 & e_2 + V_2 & \cdots & e_2 + V_i & \cdots & e_2 + V_{p'} \\ e_3 & e_3 + V_2 & \cdots & e_3 + V_i & \cdots & e_3 + V_{p'} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ e_{p+1} & e_{p+1} + V_2 & \cdots & e_{p+1} + V_i & \cdots & e_{p+1} + V_{p'} \end{bmatrix}$$

举例：考虑码C={0000,1011,0101,1110}，对应的标准阵为

码字 $\rightarrow$ 0000 1011 0101 1110
 1000 0011 1101 0110
 0100 1111 0001 1010
 0010 1001 0111 1100

陪集首

Page 53

性质1：同一行中任意两个 n 重之和为一个码字。

性质2：在标准阵中，同一行没有两个 n 重是相同的，每个 n 重在且仅在一行中出现。

性质2的证明：

- 1) 假设第 i 行有2个 n 重是相同的，如对 i 有 $e_i + V_i = e_j + V_j$ ，即 $V_i = V_j$ ，这与标准阵的构造相矛盾，故性质2的第一句话得证。
- 2) 首先由定义知每个 n 重至少出现一次，假设一个 n 重在第 i 行和第 m 行($i < m$)都出现，则必存在 l ，使得该 n 重等于 $e_l + V_l$ ，且存在 j ，使得该 n 重等于 $e_m + V_j$ ，即有 $e_m = e_l + (V_j + V_l) = e_l + V_l$ ，这意味着 e_m 在第 i 行，这与标准阵的构造定义相矛盾。

Page 54

定义：标准阵中共有 2^{n-k} 行，它们称为码C的陪集。

陪集首：每个陪集中的第一个 n 重 e_i 称为陪集首。陪集中的任何一个元素都可以作为陪集首，需要做置换操作。

对于一个码字 V_i ，如果信道造成的错误图样是陪集首，则接收矢量 r 在陪集中，此时，可以将接收矢量正确的译码为 V_i ；否则，若信道造成的错误图样不是陪集首，则会造成错误译码。

定理：陪集首是可纠正的错误图样，共有 2^{n-k} 个可纠正的错误图样。

Page 55

方法：如果接收矢量 r 落在标准阵中的第 i 行第 j 列，那么就将 r 译码为 V_j ，同时错误图样为 e_i ，即 $r = e_i + V_j$ 。

举例：考虑码C={0000,1011,0101,1110}，对应的标准阵为

码字 $\rightarrow$ 0000 1011 0101 1110
 1000 0011 1101 0110
 0100 1111 0001 1010
 0010 1001 0111 1100

陪集首

如果接收矢量是1101，
那么估计的码字是0101，
错误图样就是1000。

Page 56

方法：对BSC信道，为了使译码错误概率最小，可以选择汉明重量最小的 n 重做为陪集首，即将接收矢量 r 译码为与 r 的汉明距离最小的那个码字。

举例：考虑码C={00000,01010,10101,11111}，

该码的最小距离是2。

如果传输的码字是11111，接收的矢量是11110，那么，

$$d(11110, 00000)=4, d(11110, 01010)=2,$$

$$d(11110, 10101)=3, d(11110, 11111)=1.$$

用最小距离译码方法可以得出传输的码字就是11111这样的结论。

Page 57

缺点：当所考虑的码C的n和k的值都很大的时候，标准阵列的长度就变得非常的巨大，此时，用标准阵来译码就变得不实用了。

思考题：有办法来给标准阵降阶么？

提示：结合前面计算伴随式的例子，可以知道伴随式比标准阵的长度短很多。因此，可以用伴随式译码来代替标准阵译码方法。

定理：一个陪集的所有 2^k 个n重有同样的伴随式。不同陪集的伴随式不同。

证明：如果 x, y 属于同一个陪集，那么

$$\begin{aligned} &\Leftrightarrow x + C = y + C \\ &\Leftrightarrow x - y \in C \\ &\Leftrightarrow (x - y)H^T = 0 \\ &\Leftrightarrow xH^T = yH^T \\ &\Leftrightarrow S(x) = S(y) \end{aligned}$$

举例：考虑码C={0000,1011,0101,1110}，对应的标准阵就是

码字	0000	1011	0101	1110	00	如果接收矢量是 $v=(1101)$ ，先通过公式 $S=vH^T$ ，求出伴随式(11)， 确定对应的陪集首 (1000)，这就是错误向量 e 。然后 根据 $e+v=(1000)+(1101)$ 求出 传输的码字为(0101)
	1000	0011	1101	0110	11	
	0100	1111	0001	1010	01	
	0010	1001	0111	1100	10	
陪集首		伴随式				
简化译码表						
错误图样	0000	1000	0100	0010		
伴随式	00	11	01	10		

思考题：伴随式译码的步骤是什么？

定义：一个能达到汉明界的码称为完备码，满足

$$M \left\{ \binom{n}{0} + \binom{n}{1}(q-1) + \binom{n}{2}(q-1)^2 + \dots + \binom{n}{t}(q-1)^t \right\} = q^n$$

其中，M为码字数目，q是q元，n是码长，t是纠错能力

特殊的，对于二元完备码，满足 $2^{n-k} = \sum_{i=0}^t \binom{n}{i}$

汉明码是可以纠1个错的完备码。

定理：q元汉明码Ham(r,q)是参数为(n,k,d)的线性码，其中，
 $r=n-k$, $n=(q^r-1)/(q-1)$, $k=(q^r-1)/(q-1)-r$, $d_{\min}=3$ 。

特殊的，对于二元汉明码，满足 $(n, k)=(2^r-1, 2^r-1-r)$ 。

提示：为啥汉明码是完备码，要根据完备码的定义考虑。

构造汉明码的两种方法：

- 1) 构造H阵的标准形式 $H=[Q \ I_r]$ ，Q是构造 I_r 后剩下的 2^r-1-r 列任意排列。
- 2) H阵的列是按r重表示的二进制数的顺序排列。

举例：(7,4)码的H阵的列为001, 010, ..., 111, 即

$$H = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$



根据生成矩阵计算码字

假设某线性分组码有如下的生成矩阵：

$$G = \begin{pmatrix} g_0 \\ g_1 \\ g_2 \\ g_3 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

若 $u=(1101)$ ，则 $v=1 \cdot g_0 + 1 \cdot g_1 + 0 \cdot g_2 + 1 \cdot g_3 = (0001101)$

做题方法：根据公式 $v=u_0g_0+u_1g_1+\dots+u_{k-1}g_{k-1}$ 进行计算

Page 64



Macwilliam恒等式

若将 (n,k) 线性分组码 C 与其对偶码 $C^\perp$ 的重量分布分别记为 $\{A_0, A_1, \dots, A_n\}$ 和 $\{B_0, B_1, \dots, B_n\}$ ，又记多项式

$$A(z) = \sum_{i=0}^n A_i z^i, B(z) = \sum_{i=0}^n B_i z^i$$

那么多项式 $A(z)$ 和 $B(z)$ 之间满足如下关系：

$$A(z) = 2^{-\frac{1}{2}(n-k)} (1+z)^n B\left(\frac{1-z}{1+z}\right)$$

Page 65



一致校验矩阵

定义：与 (n,k) 线性分组码 C 的生成矩阵 G 相对应有一个 $(n-k) \times n$ 阶矩阵 H ，它的 $n-k$ 个行是码 C 的对偶空间的一组基底，该矩阵 H 称为码 C 的一致校验矩阵。

定理：如果线性码 C 是 H 矩阵的零化空间，则对每一个重量为 w 的码矢， H 中有相应的 w 列线性相关。反之， H 中若有 w 列线性相关，那么就有相应的重量为 w 的一个码矢。

这是由于 $vH^T = (v_1, v_2, \dots, v_n) \begin{bmatrix} h_1 & h_2 & \dots & h_n \end{bmatrix}^T = \sum_{i=1}^n v_i h_i = 0$

Page 66



一致校验矩阵与汉明重量

定理：令 C 是一致校验矩阵为 H 的 (n,k) 线性码，对于汉明重量为 l 的每个码矢，在 H 中都有 l 列，使得这 l 列的矢量和为0，反之，若 H 中存在 l 列，其矢量和为0，则 C 中必有一个汉明重量为 l 的码矢。

Page 67



举例

特别的，对于线性分组码，有如下定理：

定理： (n,k) 线性码的最小码间距离等于非零码字的最小汉明重量。

例如：线性分组码 $C=(0000, 1010, 0101, 1111)$ ，其中分组长度为 $n=4$ ，计算码字之间的汉明距离
 $d(0000, 1010) = 2, d(0000, 0101) = 2,$
 $d(0000, 1111) = 4, d(1010, 0101) = 4,$
 $d(1010, 1111) = 2, d(0101, 1111) = 2.$
 该码的最小距离是2，并且该码的最小重量也是2
 所以最小距离和最小重量是相等的。

Page 68



纠错和检错能力举例

例1：码 $C_1=\{000,111\}$ ，码的最小距离是3，因此重量为1和2个错误图样可以被检测出来，即错误图样如果是 $\{011,101,110,001,010,100\}$ ，就可以被检测出来。

例2：对于码 $C_2=\{001,110,101\}$ ，码的最小距离是1，由于 $d-1=0$ ，所以我们什么都不能说。重量为1的错误图样010可以被检测出来，但是该码不能检测所有重量为1的错误图样，比如错误图样100就不能被检测出来。

Page 69

令 C 是一致校验矩阵为 H 的 (n,k) 线性码:

- 1) 若 H 中没有 $d-1$ 列或者更少的列矢量和为0, 则 C 的最小码间距离至少为 d ;
- 2) C 的最小码间距离等于 H 中中和为0的最小列数。

- 步骤1:** 在第一行写下所有合法的 2^k 个码字 $v=\{v_1, \dots, v_{2^k}\}$, 第一个码字为零码字;
- 步骤2:** 选择一个第一行没有出现的矢量作为 e_2 , 标准阵第2行写 e_2+v ;
- 步骤3:** 继续选择一个第一行和第二行都没有出现的矢量 e_3 , 标准阵第3行写 e_3+v ;
- 步骤4:** 依次类推, 直到 $GF(2^n)$ 中所有的矢量都被列出来一次

对转移概率为 p 的二元对称信道而言, 陪集首的重量分布 $\{A_0, A_1, \dots, A_n\}$ 与译码错误概率之间满足

$$P(E) = 1 - \sum_{i=0}^t A_i p^i (1-p)^{n-i}$$

定理: 对码间最小距离为 d 的 (n,k) 线性码, 重量 $w \leq (d-1)/2 = t$ 的 n 重都可以被用作标准阵的陪集首, 并且至少有一个重量为 $t+1$ 的 n 重不能作为陪集首。

注: 此定理再次表明了码 C 能纠正 t 个错误, 但是不能纠正所有 $t+1$ 个错误。

定理: 若 C 是 $GF(q)$ 上的一个 (n,k) 线性码, 则

- 1) 长度为 n 的矢量 b 一定在码 C 的某个陪集中;
- 2) 每个陪集都包含 q^k 个矢量;
- 3) 两个陪集要么是不相交的要么就是重合的(即相互部分重叠是不可能的);
- 4) 如果 $a+C$ 是码 C 的一个陪集, 且有 $b \in a+C$, 那么一定有 $b+C=a+C$ 。

定理: 1) 长度为 n 的矢量 b 一定在码 C 的某个陪集中;

证明: $b = b + 0 \in b + C$ 。

定理: 2) 每个陪集都包含 q^k 个矢量;

证明: 对于所有的 $x \in C$, 由 $x \rightarrow a+x$ 定义的映射 $C \rightarrow a+C$ 是个一一映射, 所以陪集 $a+C$ 中矢量的个数和码集中码矢的个数相等, 即 q^k 。

定理: 3) 两个陪集要么是不相交的要么就是重合的(即相互部分重叠是不可能的);

证明: 反证法。假设陪集 $a+C$ 和 $b+C$ 是相交的, 即它们至少有一个向量是相同的, 令 $v = (a+C) \cap (b+C)$, 那么对于任意的 $x, y \in C$, 有 $v = a+x = b+y$, $b = a+x+y = a+z$, 其中 $z \in C$, 这是根据码字集合的封闭性, 两个码字之和仍然是一个码字, 因此, $b+C = a+z+C = a+C+z$ 或者 $(b+C) \subset (a+C)$ 。类似的, 还可以得到 $(a+C) \subset (b+C)$, 所以 $(b+C) = (a+C)$ 。

定理：4) 如果 $a+C$ 是码 C 的一个陪集，且有 $b \in a+C$ ，那么一定有 $b+C=a+C$ ；

证明：由于 $b \in a+C$ 表明对于某个 $x \in C$ ， $b=a+x$ 。接下来，如果 $b+y \in b+C$ ，那么 $b+y=(a+x)+y=a+(x+y) \in a+C$ ，即 $(b+C) \subseteq (a+C)$ 。
另一方面，类似的，如果 $a+z \in a+C$ ，那么 $a+z=(b+x)+z=b+(x+z) \in b+C$ ，即 $(a+C) \subseteq (b+C)$ ，因此， $b+C=a+C$ 。

Page 76

- 步骤1：**计算接收矢量 r 的伴随式 $r \cdot H^T$ ；
步骤2：确定伴随式等于 $r \cdot H^T$ 对应的陪集首 e_i ，然后假定 e_i 是信道引起的错误图样；
步骤3：将接收矢量 r 译码为码字 $v=r+e_i$ 。

Page 77

用码长和最小距离计算(p 为信道转移概率)：

$$P_e = \sum_{i=d_{\min}}^n \binom{n}{i} p^i (1-p)^{n-i} \quad \text{或} \quad P_e = 1 - \sum_{i=0}^{d_{\min}-1} \binom{n}{i} p^i (1-p)^{n-i}$$

用码的重量分布计算(A_i 为重量分布)：

$$P_e = \sum_{i=1}^n A_i p^i (1-p)^{n-i}$$

Page 78

线性扩张：设 S 为一个长度为 n 且分量在 $GF(q)$ 上的向量集合。 S 中所有向量的线性组合称为 S 的线性扩张，记为 $\langle S \rangle$ 。即线性扩张是由 S 生成的 $GF(q^n)$ 的一个子空间。

给定 $GF(q^n)$ 的任意子集 S ，可以得到一个由 S 生成的线性码 $C=\langle S \rangle$ ，它恰好包含下列码字：

- 1) 全零码字；
- 2) S 中所有的字；
- 3) S 中两个或两个以上的字的所有线性组合。

Page 79

定义：设 C 是二元 (n, k, d) 线性分组码，若对每一个码字 $c=(c_0, c_1, \dots, c_{n-1})$ 增加一个校验位 c_n 。满足如下校验关系 $c_0 + c_1 + c_2 + \dots + c_{n-1} = 0$ ，则称新构造的 $(n+1, k, d)$ 线性分组码 C_1 为原码的扩展码，称 c_n 为奇偶校验位。

若原线性分组码 C 的校验矩阵为 H ，则可以得到扩展码 C_1 的校验矩阵为 H_1 ，其最小距离 $d'=d$ 或 $d+1$

$$H_1 = \begin{bmatrix} 0 \\ H \\ \vdots \\ 0 \\ 1 \ 1 \ \dots \ 1 \ 1 \end{bmatrix}$$

Page 80

删余码：将二元 (n, k, d) 线性分组码 C 删去一个校验元得到 $(n-1, k, d')$ 线性分组码 C_1 为原码 C 的删余码，其最小距离 $d'=d$ 或 $d-1$ 。

Page 81

定义：设二元 (n, k, d) 线性分组码 C 没有全1码字，在它的 G 矩阵上增加一组全1行，得到生成矩阵 G_1 ，则称由 G_1 定义的 $(n, k+1, d')$ 线性分组码 C_1 为原码 C 的增广码或增信删余码。

与增信删余码构造过程相反的码是增删删信码。

定义：设二元 (n, k, d) 线性分组码 C 的最小汉明距离 d 为奇数，则其全体偶数重量的码字组成一个新的 $(n, k-1, d')$ 线性分组码 C_1 ，称 C_1 为原码 C 的增删删信码，其最小距离 $d'=d+1$ 。

Page 82

缩短码：对于二元 (n, k, d) 线性分组码 C ，选出最高位码元 $C_{n-1}=0$ 的全体码字，并去掉这一位得到一个 $n-1$ 长的新码 C_1 ，称 C_1 为原码的缩短码。

反复使用缩短码的方法缩短 i 位后，就得到 $(n-i, k-i)$ 缩短的线性分组码。

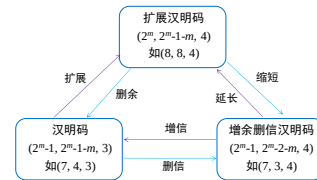
Page 83

与缩短码相反的就是延长码。

延长码：二元 (n, k, d) 线性分组码 C 先进行增广，再将得到的增广码进行扩展，则最后的 $(n+1, k+1)$ 扩展码，就称为原码的延长码。

Page 84

以 $(7, 3, 4)$ 汉明码为例



Page 85

定义：设 C_1 和 C_2 分别是二元 (n_1, k_1, d_1) 和 (n_2, k_2, d_2) 线性分组码，且 $C_1 \cap C_2 = \emptyset$ ，则定义 C_1 和 C_2 的直和码 C 是

$$C = C_1 + C_2 = \{c_1 \oplus c_2 | c_1 \in C_1, c_2 \in C_2\}$$

不难推导出直和码 C 的生成矩阵 $G = \begin{bmatrix} G_1 \\ G_2 \end{bmatrix}$

由该生成矩阵可生成 (n, k_1+k_2, d) 线性分组码，其最小汉明距离 $d \leq \min\{d_1, d_2\}$

Page 86

定义：设 C_1 和 C_2 分别是二元 (n_1, k_1, d_1) 和 (n_2, k_2, d_2) 线性分组码，则定义 C_1 和 C_2 的笛卡尔积码 C 是

$$C = C_1 \times C_2 = \{(c_1, c_2) | c_1 \in C_1, c_2 \in C_2\}$$

笛卡尔积码 C 的生成矩阵 $G = \begin{bmatrix} G_1 & \mathbf{0} \\ \mathbf{0} & G_2 \end{bmatrix}$

由该生成矩阵可生成 $(n_1+n_2, k_1+k_2, \min\{d_1, d_2\})$ 线性分组码

Page 87

定义：设 C_1 和 C_2 分别是二元 (n_1, k_1, d_1) 和 (n_2, k_2, d_2) 线性分组码，且 $k_2 \geq k_1$ ，则定义 C_1 和 C_2 的链接码 C 的生成矩阵为

$$G = \begin{bmatrix} G_1 & G_2 \\ 0 & \end{bmatrix}$$

笛卡尔积码 C 的生成矩阵

由该生成矩阵可生成 $(n_1+n_2, k_1+k_2, \min\{d_1, d_2\})$ 线性分组码

直积：设 $A=(a_{ij})$ 为 $m_1 \times q_1$ 维矩阵、 $B=(b_{ij})$ 为 $m_2 \times q_2$ 维矩阵，则矩

$$m_1 m_2 \times q_1 q_2 \text{ 维矩阵 } \begin{pmatrix} a_{11}B & \cdots & a_{1q_1}B \\ \vdots & & \vdots \\ a_{m_1 1}B & \cdots & a_{m_1 q_1}B \end{pmatrix}$$

称为两个矩阵 A 和 B 的直积或Kronecker积。

直积码：设 C_1 和 C_2 分别是二元 (n_1, k_1, d_1) 和 (n_2, k_2, d_2) 线性分组码，则定义 C_1 和 C_2 的直积码 C 是

$$C = C_1 \otimes C_2 = \{c_1 \otimes c_2 | c_1 \in C_1, c_2 \in C_2\}$$

直积码 C 是 $(n_1 n_2, k_1 k_2, d_1 d_2)$ 线性分组码，生成矩阵为

$$G = G_1 \otimes G_2$$

大多数线性分组码都是用来纠正随机错误。但是在有些信道中，如衰落信道，衰落严重时，就会发生一连串的错误，这种错误具有突发性。

交织码的思想：通过数据交织使得突发错误在一定程度上转变为随机错误，并且使原来的突发错误分布在多个码字之间，这样就可以用纠正随机错误的编码纠正突发错误。

交织码的编码和译码方法如下：

- 1) 将 N 个 (n, k, d) 分组码的码字，按每个码字为一行，排列成 N 行 n 列的码矩阵的形式；
- 2) 按列将码阵中的元素读出，并送入信道发送；
- 3) 接收端收到接收字后，按接收顺序以每 N 个元素为一列排列成 N 行 n 列的矩阵；
- 4) 按接收码阵的每一行，用相应的 (n, k, d) 分组码的译码方法进行译码。

Hadamard矩阵 M_n 是个 $n \times n$ 的矩阵，且任意一行与其他的行中有 $n/2$ 个位置不同。其中一行全为0，其余的行有 $n/2$ 个0和 $n/2$ 个1。

从 M_n 可以按下面的规则构造出 $M_{2n} = \begin{bmatrix} M_n & M_n \\ M_n & \bar{M}_n \end{bmatrix}$ $\bar{M}_n$ 通过将 M_n 中的0和1互反得到

$$n=2, \text{ Hadamard矩阵为 } M_2 = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$
$$\text{从 } M_2 \text{ 可以得到 } M_4 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}, \bar{M}_2 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$

这两个矩阵的行构成一个长 $n=4$ ，有 $2n=8$ 个码字的线性二元码



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第四章 循环码

李丽香, 彭海朋

北京邮电大学网络空间安全学院

网络与交换技术国家重点实验室





北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

循环码

1949年, 二元和三元Golay码
1957年, Prange提出循环码的概念
之后, Peterson和Kasami进一步开展了循环码的研究工作
1960s, Bose和Raychaudhuri进行了循环码最小距离方面的开创工作
1959年, Abramson进行了突发错误纠错方面的工作, Fire码

一个数学理论在被搞得如此清楚以至于你能给马路上第一个遇到的
人解释之前, 不能算是完备的。——David Hilbert (1862—1943)

Page 94



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

循环码的定义和优势

循环移位: 将 n 重 $v=(v_{n-1}, v_{n-2}, \dots, v_1, v_0)$ 的分量循环左移一位, 得到另一个 n 重 $v^{(1)}=(v_{n-2}, \dots, v_1, v_0, v_{n-1})$, 称为 v 的循环移位。若 v 的分量循环左移 i 位, 得到 $v^{(i)}=(v_{n-i-1}, \dots, v_1, v_0, v_{n-1}, \dots, v_{n-i})$ 。

循环码的定义: 一个 (n, k) 线性码 C , 若它的每个码字的任何循环移位都仍然是一个码字, 则称此码为循环码。

举例: 二元码 $C_1=\{0000, 0101, 1010, 1111\}$ 是一个循环码。

思考题: 循环码特殊的代数结构导致其具有什么优点?

循环码的优势: 具有许多固有的代数结构, 可以找到各种实用的译码方法, 使用具有反馈的线性移位寄存器可以容易的实现编码和伴随式计算。

Page 95



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

码字多项式

码字多项式: 为研究循环码的代数特性, 将码字 $v=(v_{n-1}, v_{n-2}, \dots, v_1, v_0)$ 的各个分量看成是多项式

$$v(x) = \sum_{i=0}^{n-1} v_i x^i = v_{n-1} x^{n-1} + \dots + v_1 x + v_0$$

的系数, 此多项式称为 v 的码字多项式。

对应于码矢 $v^{(i)}=(v_{n-i-1}, \dots, v_1, v_0, v_{n-1}, \dots, v_{n-i})$ 的码字多项式为 $v^{(i)}(x)=v_{n-i-1}x^{n-1}+v_{n-i-2}x^{n-2}+\dots+v_1x^{i+1}+v_0x^i+v_{n-1}x^{i-1}+\dots+v_{n-i+1}x+v_{n-i}$

码字多项式 $v^{(i)}(x)$ 就是 $x^i v^{(0)}(x) \pmod{x^n+1}$ 的余式。

思考题: 循环码有哪些代数性质?

Page 96



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

循环码的代数性质

定理1: 循环码 C 中次数最低的非零码字多项式是唯一的。

证明: 令 $g(x)=\sum_{i=0}^r g_i x^i, g_r=1$ 是 C 中次数最低的码字多项式, 若 $g(x)$ 不唯一, 则存在另一个码字多项式 $g^*(x)=\sum_{i=0}^r g_i^* x^i, g_r^*=1$, 由线性性质可知, $g(x)+g^*(x)$ 也是一个码字多项式, 但它的次数小于 r , 和前提矛盾, 因此, 循环码 C 中次数最低的非零码字多项式是唯一的。证毕。

Page 97



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

循环码的代数性质

定理2: 若 $g(x)=x^r+g_{r-1}x^{r-1}+\dots+g_1x+g_0$ 是 (n, k) 循环码 C 中次数最低的非零码字多项式, 则必有 $g_0=1$ 。

证明: 若 $g_0=0$, 则 $g(x)=x(x^{r-1}+g_{r-1}x^{r-2}+\dots+g_2x+g_1)$, 将 $g(x)$ 循环左移 $n-1$ 位之后, 可以得到一个次数更低的码字多项式, 其次数为 $r-1$, 和前提矛盾, 则必有 $g_0=1$ 。证毕。

结合定理1和定理2, 知在 (n, k) 循环码中, 次数最低的非零码字多项式形如 $g(x)=x^r+g_{r-1}x^{r-1}+\dots+g_1x+1$ 。

Page 98

考虑多项式 $g^{(1)}(x)=xg(x)$, $g^{(2)}(x)=x^2g(x), \dots, g^{(n-r-1)}(x)=x^{n-r-1}g(x)$ 都是 $g(x)$ 的循环移位, 它们都是码字多项式, 次数分别为 $r+1, r+2, \dots, n-1$ 。

根据码字多项式的线性性质可知, 它们的任意线性组合 $v(x)=u_{n-r-1}x^{n-r-1}g(x)+\dots+u_1xg(x)+u_0g(x)=(u_{n-r-1}x^{n-r-1}+\dots+u_1x+u_0)g(x)=$ 也是 C 的码字多项式。

(n, k) 循环码的码字多项式, 次数分别为 $r, r+1, r+2, \dots, n-1$, 次数小于或等于 $n-1$ 并且是 $g(x)$ 的倍式的二元多项式共有 2^{n-r} 个, 它们是 (n, k) 循环码 C 的所有码字, 因此, 必有 $k=n-r$ 或 $r=n-k$ 。

定理3: 若 $g(x)$ 是 (n, k) 循环码 C 中次数最低的非零码字多项式, 一个次数小于或等于 $n-1$ 的多元多项式 $f(x)$ 是码字多项式当且仅当 $f(x)$ 是 $g(x)$ 的倍式。

证明: 反证法。设 $f(x)$ 是码字多项式且其次数小于或等于 $n-1$,

假设 $f(x)=a(x)g(x)+b(x)$, $\deg(b(x))<r$,

由线性性质知 $b(x)=f(x)+a(x)g(x)$ 也是一个码字多项式, 而 $\deg(b(x))<\deg(g(x))$, 这和前提矛盾, 则只能当 $f(x)$ 是 $g(x)$ 的倍式的时候, $f(x)$ 是码字多项式。证毕。

定理1: 在 (n, k) 循环码 C 中存在且仅存在一个次数为 $n-k$ 的码字多项式 $g(x)=x^{n-k}+g_{n-k-1}x^{n-k-1}+\dots+g_2x^2+g_1x+1$, 称为生成多项式, 使得每个次数小于或等于 $n-1$ 的多元多项式是码字多项式当且仅当此多项式是 $g(x)$ 的倍式。

根据该定理, (n, k) 循环码的任意码字多项式可以表示为 $v(x)=u(x)g(x)=(u_{k-1}x^{k-1}+\dots+u_2x^2+u_1x+u_0)g(x)$ 。

若 $u(x)$ 的系数 $u_{k-1}, \dots, u_1, u_0$, 是待编码的 k 个信息数据, 则 $v(x)$ 就是相应的码字多项式, 即用 $g(x)$ 和 $u(x)$ 相乘就完成了编码。

因此, 一个 (n, k) 循环码就完全由次数最低的非零多项式 $g(x)$ 决定, $g(x)$ 的次数等于一致校验位数的数目。

定理2: (n, k) 循环码的生成多项式 $g(x)$ 是 x^n+1 的因式。

证明: 根据PPT第5页的结论: 码字多项式 $v^{(0)}(x)$ 就是

$x^i v^{(0)}(x) \bmod (x^n+1)$ 的余式。则有 $x^k g(x) = (x^n+1) + g^{(k)}(x)$, 而 $g^{(k)}(x)$ 是 $g(x)$ 的 k 次循环移位, 也是码字多项式, 因此, 它是 $g(x)$ 的倍式。

定理3: 若 $g(x)$ 是一个 $n-k$ 次多项式, 并且是 x^n+1 的因式, 则 $g(x)$ 生成一个 (n, k) 循环码。

消息	码矢	码多项式
0000	0000000	$0 = 0 \cdot g(x)$
1000	1101000	$1 + X + X^{n-1} = 1 \cdot g(x)$
0100	0110100	$Xg(x)$
1100	1011100	$(1+X)g(x)$
0010	0011010	$X^2g(x)$
1010	1110010	$(1+X^2)g(x)$
0110	0101110	$(X+X^2)g(x)$
1110	1000110	$(1+X+X^2)g(x)$
0001	0001101	$X^3g(x)$
1001	1100101	$(1+X^3)g(x)$
0101	0111001	$(X+X^3)g(x)$
1101	1010001	$(1+X+X^3)g(x)$
0011	0010111	$(X^2+X^3)g(x)$
1011	1111111	$(1+X^2+X^3)g(x)$
0111	0100011	$(X+X^2+X^3)g(x)$
1111	1001011	$(1+X+X^2+X^3)g(x)$

举例: 多项式 x^7+1 可以分解为 $(x^3+1)(x^4+x+1)$, 其中有两个次数为3的因式, 都可以生成(7,4)循环码, 左边表格的循环码是由 x^3+x+1 生成的, $d=3$, 能纠1个错。

考虑以 $g(x) = \sum_{i=0}^{n-k} g_i x^i$ 为生成矩阵的 (n, k) 循环码 C , 由于 $g(x), xg(x), \dots, x^{k-1}g(x)$ 生成 C , 将这 k 个多项式的 k 个 n 重做为 $k \times n$ 阶矩阵的行, 一般选择前 $k-1$ 位为0的向量作为 $g(x)$, 则得到 C 的生成矩阵为

$$G(x) = \begin{bmatrix} x^{n-k}g(x) \\ \vdots \\ xg(x) \\ g(x) \end{bmatrix} = \begin{bmatrix} g_{n-k} & \cdots & g_2 & g_1 & g_0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & g_{n-k} & \cdots & g_2 & g_1 & g_0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & g_{n-k} & \cdots & g_2 & g_1 & g_0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & g_{n-k} & \cdots & g_2 & g_1 & g_0 \end{bmatrix}$$

其中 $g_0 = g_{n-k} = 1$ 。

考虑(7, 3)汉明码, $n=7, k=3, t=2$, 码字为(1001110), (0100111), (1010011), (1101001), (1110100), (0111010), (0011101), 因为码字(0011101)前面有 $k-1=2$ 个零, 将它作为 $g(x)$, 即 $g(x)=x^4+x^3+x^2+1$, 得到

$$G(x) = \begin{bmatrix} x^2 g(x) \\ x g(x) \\ g(x) \end{bmatrix} = \begin{bmatrix} x^6+x^5+x^4+x^2 \\ x^5+x^4+x^3+x \\ x^4+x^3+x^2+1 \end{bmatrix} \Rightarrow G = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

其中 $g_0=g_{n-k}=1$ 。

Page 105

因生成多项式 $g(x)$ 是 x^n+1 的因子, 所以存在另一个多项式 $h(x)=\sum_{i=0}^{n-k-1} h_i x^i, h_k = h_0 = 1$, 使得 $g(x)h(x)=x^n+1$, 此多项式 $h(x)$ 称为码C的校验多项式 $h(x)$, 故可以得到码C的一致校验矩阵 H 。
设 $v=(v_{n-1}, \dots, v_1, v_0)$ 是C的一个码矢, 则 $v(x)=a(x)g(x)$, 有 $v(x)h(x)=a(x)g(x)h(x)=a(x)(x^n+1)=a(x)x^n+a(x)$, 由于 $\deg(a(x)) \leq k-1$, 故在 $a(x)x^n+a(x)$ 中不会出现 $x^{n-1}, \dots, x^{k+1}, x^k$ 等各次幂, 于是 $v(x)h(x)$ 的展开式中 $x^{n-1}, \dots, x^{k+1}, x^k$ 的系数等于零。
因此, 得到以下 $n-k$ 个方程 $h_0 v_{n-j} + h_1 v_{n-j-1} + \dots + h_k v_{n-j-k} = 0, j=1, \dots, n-k$ 。则码C的一致校验矩阵为

$$H = \begin{bmatrix} h_k & h_{k-1} & h_{k-2} & \dots & h_0 & 0 & 0 & \dots & 0 \\ 0 & h_k & h_{k-1} & \dots & h_1 & h_0 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & h_k & h_{k-1} & h_{k-2} & \dots & h_0 \end{bmatrix}$$

Page 106

用生成多项式 $g(x)$ 去除 $x^{n-k+i}, 0 \leq i \leq k-1$, 有 $x^{n-k+i} = a_i(x)g(x) + b_i(x)$, 其中 $b_i(x) = \sum_{j=0}^{n-k-1} b_{ij} x^j$ 。由于 $x^{n-k+i} + b_i(x)$ 是 $g(x)$ 的倍式, 它是一个码字多项式。将这 k 个码字多项式排成 $k \times n$ 阶矩阵的行, 得到循环码C系统形式的生成矩阵

$$G = \begin{bmatrix} b_{00} & b_{01} & \dots & b_{0,n-k-1} & 1 & 0 & \dots & 0 \\ b_{10} & b_{11} & \dots & b_{1,n-k-1} & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ b_{k-1,0} & b_{k-1,1} & \dots & b_{k-1,n-k-1} & 0 & 0 & \dots & 1 \end{bmatrix} \Rightarrow H = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 & b_{00} & b_{01} & \dots & b_{0,n-k} \\ 0 & 1 & 0 & \dots & 0 & b_{10} & b_{11} & \dots & b_{1,n-k} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 & b_{k-1,0} & b_{k-1,1} & \dots & b_{k-1,n-k} \end{bmatrix}$$

Page 107

给定循环码的生成多项式 $g(x)$, 可以使该码成为系统形式。

假定待编码的消息是 $u=(u_{k-1}, \dots, u_1, u_0)$, 则对应的消息多项式为 $u(x)=u_{k-1}x^{k-1}+\dots+u_1x+u_0$, 从而有 $x^{n-k}u(x)=u_{k-1}x^{n-1}+\dots+u_1x^{n-k+1}+u_0x^{n-k}$ 。用生成多项式 $g(x)$ 除 $x^{n-k}u(x)$, 得到 $x^{n-k}u(x)=a(x)g(x)+b(x), \deg(b(x)) < n-k$ 。

循环码的编码是由多项式的乘法、除法以及加法运算完成的, 这些运算都可以由移位寄存器、加法器以及门电路来实现。

系统循环码的编码步骤如下:

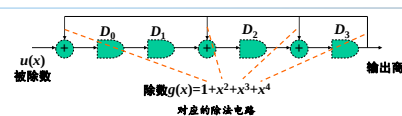
- 步骤1: 先用 x^{n-k} 乘以消息多项式 $u(x)=u_{k-1}x^{k-1}+\dots+u_1x+u_0$;
- 步骤2: 用生成多项式 $g(x)$ 除 $x^{n-k}u(x)$, 得到余式 $b(x)$;
- 步骤3: 联合 $b(x)$ 和 $x^{n-k}u(x)$, 得到码字多项式 $v(x)=x^{n-k}u(x)+b(x)$

Page 108

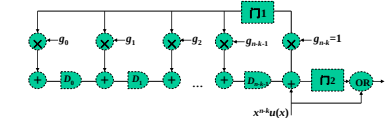
举例: $u(x)=x^6+x^4, g(x)=x^4+x^3+x^2+1$, 求 $u(x)/g(x)$ 。

111 商
除数 11101 | 111011010000 被除数
11101
10010
11101
11110
11101
11
余数

即 $u(x)=(x^4+x^3+x^2+1)(x^2+x+1)+(x+1)$

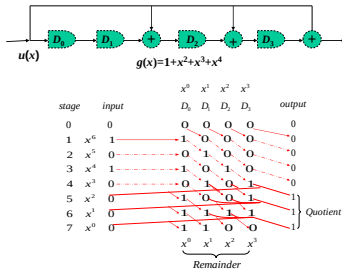


Page 109



编码电路的工作过程如下:
第1步: $n-k$ 级移位寄存器的初始状态都清零。门1通, 门2断, 进行移位, k 个信息数据(或信息组 $u(x)=u_{k-1}x^{k-1}+\dots+u_1x+u_0$ 的系数)被送入电路中, 高位次系数首先送入电路, 消息 $u(x)$ 从线路的前端移入, 等价于用 x^{n-k} 左乘 $u(x)$;
第2步: 经 k 次移位后, $u(x)$ 全部被送入电路, 完成了除法运算, 此时寄存器中的 $n-k$ 个数据就构成了余项, 它们就是一致校验数据;
第3步: 门1断, 门2通, 经过 $n-k$ 移位次后, 移出移位寄存器的 $n-k$ 个校验元, 这 $n-k$ 个校验元与原来的 k 个信息位组成 n 长的完整的码字;
第4步: 门1通, 门2断, 送入第2组信息数据, 并重复上述过程。

Page 110



Page 111

回忆：伴随式定义为 $S = rH^T$ 对系统循环码而言, $r(x) = \sum_{i=0}^{n-1} r_i x^i$ 是接收多项式, $g(x) = \sum_{i=0}^{n-k-1} g_i x^i$ 是生成多项式, 用 $g(x)$ 去除 $r(x)$ 所得的余式, 就是伴随式 $s(x)$, 即

$$r(x) = a(x)g(x) + s(x).$$

$$\because r(x) = v(x) + e(x), \quad v(x) = q(x)g(x), \quad r(x) = a(x)g(x) + s(x)$$

$$\because q(x)g(x) + e(x) = a(x)g(x) + s(x)$$

$$\therefore e(x) = [q(x) + a(x)]g(x) + s(x)$$

求伴随式的运算也可以用除法电路来实现

Page 112

性质1: 令 $s(x)$ 是接收多项式 $r(x)$ 的伴随式, 则用生成多项式 $g(x)$ 除 $xs(x)$ 所得的余式 $s^{(1)}(x)$ 就是 $r(x)$ 循环移位一次 $r^{(1)}(x)$ 的伴随式。

性质2: (n, k) 循环码能够检测长度小于等于 $n-k$ 的任何突发错误, 换句话说, 每个长度小于等于 $n-k$ 的突发错误的伴随式不等于 0。

性质3: 伴随式等于生成多项式除错误图样后所得的余式。

伴随式的这些性质对于译码非常有用。举例来说, 若 $e(x)$ 是可以纠正的错误图样, 则 $e(x)$ 的循环移位也是可以纠正的错误图样。

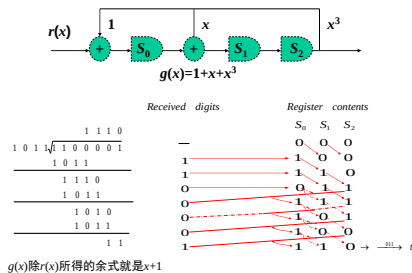
Page 113

$$g(x) = x^3 + x + 1, \quad r(x) = x^6 + x^5 + 1$$

$$G(x) = \begin{bmatrix} x^6 + x^4 + x^2 \\ x^5 + x^3 + x \\ x^4 + x^2 + x \\ x^3 + x + 1 \end{bmatrix} \rightarrow G = \begin{bmatrix} 1011000 \\ 0101100 \\ 0010110 \\ 0001011 \end{bmatrix} \rightarrow \begin{bmatrix} 1010011 \\ 0100111 \\ 0010110 \\ 0001011 \end{bmatrix} \rightarrow \begin{bmatrix} 1000101 \\ 0100111 \\ 0010110 \\ 0001011 \end{bmatrix} \rightarrow H^T = \begin{bmatrix} 101 \\ 111 \\ 110 \\ 100 \\ 010 \\ 001 \end{bmatrix}$$

$$S = rH^T = \begin{bmatrix} 1010001 \\ 011 \end{bmatrix} = (011) \quad s(x) = x + 1$$

Page 114



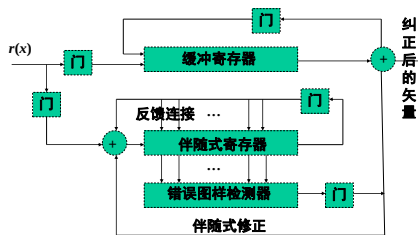
Page 115

循环码的译码分为3步：1) 计算伴随式; 2) 由伴随式得到错误图样; 3) 纠正错误。

具体过程描述如下：

- 1) 根据接收向量 $r(x) = \sum_{i=0}^{n-1} r_i x^i$ 计算出伴随式 $s(x)$;
- 2) 检查 $s(x)$ 是否与某个满足 $e_{n-1}=1$ 的错误图样 $e(x) = \sum_{i=0}^{n-1} e_i x^i$ 相对应;
- 3) 若 $s(x)$ 与此类 $e(x)$ 不相对应, 则接收多项式 (存在缓存器中) 和伴随式寄存器同时循环移一位, 从而得到 $r^{(1)}(x) = r_{n-2}x^{n-1} + \dots + r_0x^n + r_{n-1}$ 和与相对应的伴随式 $s^{(1)}(x)$, 再次检查 $s^{(1)}(x)$ 是否与某个满足 $e_{n-1}=1$ 的错误图样相对应;
- 4) 若 $s(x)$ 与某个满足 $e_{n-1}=1$ 的错误图样相对应, 则第一位接收数据 r_{n-1} 有误, 因而必须纠正, 这可以用 $r_{n-1} + e_{n-1}$ 实现;
- 5) 这一纠正得到了修正的接收多项式 $r_1(x) = (r_{n-1} + e_{n-1})x^{n-1} + r_{n-2}x^{n-2} + \dots + r_1x + r_0$;
- 6) 把 $r_1(x)$ 和其伴随式同时循环移一位, 便得到接收多项式 $r_1^{(1)}(x) = r_{n-2}x^{n-1} + \dots + r_0x^n + (r_{n-1} + e_{n-1})$, 再利用 $r_1^{(1)}(x)$ 的伴随式 $s_1^{(1)}(x)$ 纠正 r_{n-2} ;
- 7) 如此继续, 总共循环移 n 次后, 便完成译码。

Page 116



Page 117

通用译码器由三个部分组成：1)伴随式寄存器；2)错误图样检测器；3)存储接收矢量的缓冲寄存器。

译码算法描述如下：

- 步骤1：接收矢量全部移入伴随式寄存器，便得到了伴随式，与此同时，接收矢量也存入缓冲寄存器；
- 步骤2：把伴随式读入检测器，以探测相应的错误图样，检测器的输出是一个与缓冲器输出符号相应的错误估计值；
- 步骤3：由缓冲器输出第一个接收符号，与此同时，伴随式寄存器移位一次。若检测到第一个接收符号是错误的，则检测器的输出给予纠正。检测器的输出也反馈到伴随式寄存器以修正伴随式。这样便得到了一个新的伴随式，它相应于向右移位1次后所得的修正接收矢量的伴随式；
- 步骤4：由第3步所得的新伴随式，用来检测第2个接收符号是否有错。译码器重复第2-3步。第2个符号的纠正方法和第一个接收符号一样；
- 步骤5：如上所述，译码器逐个符号地译接收矢量，直到从缓冲器中读出全部接收矢量为止。

Page 118

码字多项式 $v^{(0)}(x)$ 就是 $x^d v^{(0)}(x)/(x^n+1)$ 的余式。

码字 $v^{(0)}=(v_{n-1}, v_{n-2}, \dots, v_1, v_0)$ 左移一位，得到 $v^{(1)}=(v_{n-2}, \dots, v_1, v_0, v_{n-1})$ 。

类似的，得到 $v^{(2)}=(v_{n-3}, v_{n-4}, \dots, v_{n-2})$

$$v^{(0)}(x) = v_{n-1}x^{n-1} + v_{n-2}x^{n-2} + \dots + v_1x + v_0$$

$$v^{(1)}(x) = v_{n-2}x^{n-1} + v_{n-3}x^{n-2} + \dots + v_0x + v_{n-1}$$

$$v^{(2)}(x) = v_{n-3}x^{n-1} + v_{n-4}x^{n-2} + \dots + v_{n-1}x + v_{n-2}$$

$$v^{(0)}(x) = v_{n-1}x^{n-1} + v_{n-2}x^{n-2} + \dots + v_1x^{n-1} + v_0x^{n-1} + \dots + v_{n-1}x + v_{n-2}$$

因有 $v^{(1)}(x) = x v^{(0)}(x) \bmod (x^n+1)$

$$v^{(2)}(x) = x^2 v^{(0)}(x) \bmod (x^n+1)$$

$$x^2 v^{(0)}(x) = v_{n-1}x^{n+1} + v_{n-2}x^{n+2} + \dots + v_{n-1}x^{n+1} + v_0x^{n+2} + \dots + v_{n-1}x^{n+1} + v_0x^{n+2}$$

$$= v_{n-1}x^{n+1} + v_{n-2}x^{n+2} + \dots + v_{n-1}x^{n+1} + v_0x^{n+2} + \dots + v_{n-1}x^{n+1} + v_0x^{n+2}$$

$$= q(x)(x^n+1) + v^{(2)}(x)$$

其中 $q(x) = v_{n-1} + v_{n-2}x + \dots + v_{n-1}x^{n-1}$

Page 119

定理： (n, k) 循环码 C 的生成多项式为 $g(x)$ ，则 C 的对偶码 $C_\perp$ 也是循环码，且由多项式 $x^k h(x)$ 生成，且 $h(x) = (1+x^n)/g(x)$ 。

证明提示：因为 $g(x)$ 是 C 的生成多项式，所以 $h(x)g(x) = x^n - 1$ 。故有

$h(x^1)g(x^1) = x^n - 1$ ，即 $x^k h(x^1)x^{n-k}g(x^1) = 1 - x^n$ 。因此，有 $x^k h(x^1)(x^n - 1)$ 。

Page 120



北京邮电大学

BEIHANG UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第五章 卷积码

李丽香, 彭海朋

北京邮电大学网络空间安全学院

网络与交换技术国家重点实验室





北京邮电大学

卷积码的发展历史

1955年, 埃利斯(P. Elias)首次提出卷积码的概念
1957年, J. M. Wozencraft提出卷积码的序列译码算法
1963年, J. L. Massey提出较易实现的门限译码算法
1967年, A. J. Viterbi提出卷积码的一种最大似然译码算法
卷积码在深空通信、卫星通信、移动通讯等领域得到了应用

现实中两个真理之间的最短路径要经过复杂的中间过程。——
Jacques Hadamard (1865—1963)

Page 122



北京邮电大学

卷积码与分组码的区别

- ❑ 卷积码有记忆性, 分组码无记忆性
- ❑ 卷积码充分利用了各组信息之间的相关性, 信息序列不被分段, 而是被连续处理
- ❑ 通常, 卷积码的n和k比分组码的n和k要小的多
- ❑ 在同样的编码效率下, 卷积码的性能优于分组码
- ❑ 在相似的纠错能力下, 卷积码的实现比分组码简单
- ❑ 卷积码必须用**序列逻辑电路**来实现, 更适用于前向纠错系统; 分组码是用**组合逻辑电路**来实现

现实中两个真理之间的最短路径要经过复杂的中间过程。——
Jacques Hadamard (1865—1963)

Page 123



北京邮电大学

离散序列的非循环卷积运算

定义: 设 $f(n)$ 和 $g(n)$ 是两个序列, 则序列 f 和 g 的离散卷积运算为

$$(f * g)(m) = \sum_n f(n)g(m-n)$$

举例: 若 $f=(10011), g=(111)$, 则 $f * g$ 的计算过程如下

$(f * g)(0) =$	$\begin{array}{r} 11001 \\ \times \quad 111 \\ \hline \end{array}$	$= 1$	$(f * g)(4) =$	$\begin{array}{r} 11001 \\ \times \quad 111 \\ \hline \end{array}$	$= 0$
$(f * g)(1) =$	$\begin{array}{r} 11001 \\ \times \quad 111 \\ \hline \end{array}$	$= 1$	$(f * g)(5) =$	$\begin{array}{r} 11001 \\ \times \quad 111 \\ \hline \end{array}$	$= 0$
$(f * g)(2) =$	$\begin{array}{r} 11001 \\ \times \quad 111 \\ \hline \end{array}$	$= 1$	$(f * g)(6) =$	$\begin{array}{r} 11001 \\ \times \quad 111 \\ \hline \end{array}$	$= 1$
$(f * g)(3) =$	$\begin{array}{r} 11001 \\ \times \quad 111 \\ \hline \end{array}$	$= 1$			

故 $f * g = (11111001)$

Page 124



北京邮电大学

二元(2,1,2)卷积码的编码器

编码器主要由 $m=2$ 级移位寄存器, $n=2$ 个模2加法器组成。所有的卷积码编码器都可以用这种类型的线性前馈移位寄存器来实现。

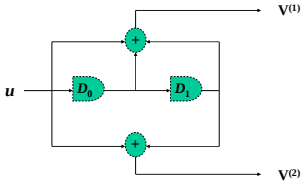


图6.1 二元(2,1,2)卷积码的编码器的电路图

Page 125



北京邮电大学

(2,1,2)卷积码的冲激响应

冲激响应: 编码器的冲激响应就是通过令 $u=(100\dots)$ 所得到的两个输出序列。

因为编码器有 m 个存储单元, 所以冲激响应(生成序列)至多持续 $m+1$ 个时间单元, 且可以写成

$$g^{(i)} = (g_0^{(i)}, g_1^{(i)}, \dots, g_m^{(i)}), g^{(2)} = (g_0^{(2)}, g_1^{(2)}, \dots, g_m^{(2)})$$

其中上标表示第几个输出端

举例: 图6.1的(2,1,2)卷积码的冲激响应为 $g^{(1)}=(111), g^{(2)}=(101)$

另外, 冲激响应向量还可以表示输入、移位寄存器和输出之间的连接关系, 1表示有连接, 0表示没有连接

Page 126

信息序列 $u=(u_1, u_2, u_3, \dots)$ 每次进入编码器1比特，编码器的两个输出序列

$$v^{(1)} = (v_0^{(1)}, v_1^{(1)}, v_2^{(1)}, \dots), v^{(2)} = (v_0^{(2)}, v_1^{(2)}, v_2^{(2)}, \dots)$$

可以通过输入序列 u 和编码器的两个冲激响应作卷积运算而得到。

举例：若 $u=(1011)$ ，图6.1的(2,1,2)卷积码的冲激响应为 $g^{(1)}=(111)$ ， $g^{(2)}=(101)$ ，则该卷积码的输出序列为 $V^{(1)}=u*g^{(1)}=(110001)$ ， $V^{(2)}=u*g^{(2)}=(100111)$

Page 127

输出序列分别为 $v^{(1)} = (v_0^{(1)}, v_1^{(1)}, v_2^{(1)}, \dots)$ ， $v^{(2)} = (v_0^{(2)}, v_1^{(2)}, v_2^{(2)}, \dots)$

的(2,1,m)的卷积码的码字，为 $V^{(1)}$ 和 $V^{(2)}$ 的交错，即

$$v = (v_0^{(1)}, v_0^{(2)}, v_1^{(1)}, v_1^{(2)}, v_2^{(1)}, v_2^{(2)}, \dots)$$

举例：图6.1的(2,1,2)卷积码，若 $u=(1011)$ ，冲激响应为 $g^{(1)}=(111)$ ， $g^{(2)}=(101)$ ，该卷积码的输出序列为 $V^{(1)}=u*g^{(1)}=(110001)$ ， $V^{(2)}=u*g^{(2)}=(100111)$ ，得到的码字为 $v=(11,10,00,01,01,11)$

Page 128

以 $u=(u_1, u_2, u_3, \dots)$ 为输入，以

$$g^{(1)} = (g_0^{(1)}, g_1^{(1)}, \dots, g_n^{(1)}), g^{(2)} = (g_0^{(2)}, g_1^{(2)}, \dots, g_n^{(2)})$$

为生成序列的(2,1,m)卷积码的输出序列分别由方程 $V^{(1)}=u*g^{(1)}$ 和 $V^{(2)}=u*g^{(2)}$ 得到。

举例：图6.1的(2,1,2)卷积码的编码方程为

$$\begin{cases} V_i^{(1)} = u_i + u_{i-1} + u_{i-2} \\ V_i^{(2)} = u_i + u_{i-2} \end{cases} \quad [V_i^{(1)} \ V_i^{(2)}] = [u_i \ u_{i-1} \ u_{i-2}] \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

另外，编码方程中每一项的系数，还可以表示输入、移位寄存器和输出之间的连接关系，1表示有连接，0表示没有连接。

Page 129

(2,1,m)卷积码的生成矩阵：将生成序列 $g^{(1)}$ 和 $g^{(2)}$ 交织后形成的半无限矩阵

$$G = \begin{bmatrix} g_0^{(1)} & g_0^{(2)} & g_1^{(1)} & g_1^{(2)} & g_2^{(1)} & g_2^{(2)} & \dots & g_n^{(1)} & g_n^{(2)} & 0 & 0 & 0 & 0 & \dots \\ 0 & 0 & g_0^{(1)} & g_0^{(2)} & g_1^{(1)} & g_1^{(2)} & \dots & g_{n-1}^{(1)} & g_{n-1}^{(2)} & g_n^{(1)} & g_n^{(2)} & 0 & 0 & \dots \\ 0 & 0 & 0 & 0 & g_0^{(1)} & g_0^{(2)} & \dots & g_{n-2}^{(1)} & g_{n-2}^{(2)} & g_{n-1}^{(1)} & g_{n-1}^{(2)} & g_n^{(1)} & g_n^{(2)} & \dots \end{bmatrix}$$

举例：图6.1的(2,1,2)卷积码冲激响应为 $g^{(1)}=(111)$ ， $g^{(2)}=(101)$ ，该码的生成矩阵为

$$G = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 1 & \dots \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & \dots \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & \dots \end{bmatrix}$$

Page 130

以 $u=(u_1, u_2, u_3, \dots)$ 为输入， G 为生成矩阵的(2,1,m)的卷积码的码字为 $V=uG$ 。

举例：图6.1的(2,1,2)卷积码，若 $u=(1011)$ ，则码字为 $v=(11,10,00,01,01,11)$

$$\begin{aligned} V &= uG \\ &= [1011] \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 1 \\ & 1 & 1 & 1 & 0 & 1 & 1 \\ & & 1 & 1 & 1 & 0 & 1 & 1 \\ & & & 1 & 1 & 1 & 0 & 1 & 1 \end{bmatrix} \\ &= (111010001010111) \end{aligned}$$

Page 131

编码器主要有2个输入，3个输出，主要由 $m=1$ 级移位寄存器， $n=3$ 个模2加法器组成。

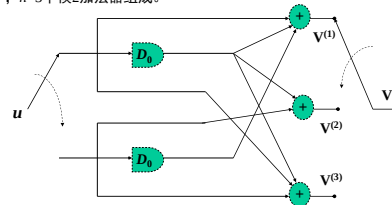


图6.2 二元(3,2,1)卷积码的编码器的电路图

Page 132



二元(3,2,m)卷积码的输入序列

对于一般的(3,2,m)卷积码, 由于k=2, 即每次有2比特进入编码器, 所以输入序列u可以写成

$$u = (u_0^{(1)}, u_0^{(2)}, u_1^{(1)}, u_1^{(2)}, u_2^{(1)}, u_2^{(2)}, \dots)$$

或分开写成2个序列

$$\begin{aligned} u^{(1)} &= (u_0^{(1)}, u_1^{(1)}, u_2^{(1)}, \dots) \\ u^{(2)} &= (u_0^{(2)}, u_1^{(2)}, u_2^{(2)}, \dots) \end{aligned}$$

Page 133



二元(3,2,m)卷积码的冲激响应

令 $u^{(1)}=(100\dots)$, 得到3个输出序列(长度均为 $m+1$)

$$\begin{aligned} g_1^{(1)} &= (g_{10}^{(1)}, g_{11}^{(1)}, g_{12}^{(1)}, \dots, g_{1m}^{(1)}) \\ g_2^{(1)} &= (g_{20}^{(1)}, g_{21}^{(1)}, g_{22}^{(1)}, \dots, g_{2m}^{(1)}) \\ g_3^{(1)} &= (g_{30}^{(1)}, g_{31}^{(1)}, g_{32}^{(1)}, \dots, g_{3m}^{(1)}) \end{aligned}$$

再令 $u^{(2)}=(100\dots)$, 又得到3个输出序列

$$\begin{aligned} g_1^{(2)} &= (g_{10}^{(2)}, g_{11}^{(2)}, g_{12}^{(2)}, \dots, g_{1m}^{(2)}) \\ g_2^{(2)} &= (g_{20}^{(2)}, g_{21}^{(2)}, g_{22}^{(2)}, \dots, g_{2m}^{(2)}) \\ g_3^{(2)} &= (g_{30}^{(2)}, g_{31}^{(2)}, g_{32}^{(2)}, \dots, g_{3m}^{(2)}) \end{aligned}$$

则称下面的序列为(3, 2, m)卷积码的生成序列

$$g_i^{(j)} = (g_{i0}^{(j)}, g_{i1}^{(j)}, g_{i2}^{(j)}, \dots, g_{im}^{(j)}), i=1,2, j=1,2,3$$

Page 134



二元(3, 2, 1)卷积码的冲激响应

举例: 图6.2的(3, 2, 1)卷积码的生成序列为

$$\begin{aligned} g_1^{(1)} &= (g_{10}^{(1)}, g_{11}^{(1)}, g_{12}^{(1)}, \dots, g_{1m}^{(1)}) = (11) \\ g_2^{(1)} &= (g_{20}^{(1)}, g_{21}^{(1)}, g_{22}^{(1)}, \dots, g_{2m}^{(1)}) = (01) \\ g_3^{(1)} &= (g_{30}^{(1)}, g_{31}^{(1)}, g_{32}^{(1)}, \dots, g_{3m}^{(1)}) = (11) \end{aligned}$$

$$\begin{aligned} g_1^{(2)} &= (g_{10}^{(2)}, g_{11}^{(2)}, g_{12}^{(2)}, \dots, g_{1m}^{(2)}) = (01) \\ g_2^{(2)} &= (g_{20}^{(2)}, g_{21}^{(2)}, g_{22}^{(2)}, \dots, g_{2m}^{(2)}) = (10) \\ g_3^{(2)} &= (g_{30}^{(2)}, g_{31}^{(2)}, g_{32}^{(2)}, \dots, g_{3m}^{(2)}) = (10) \end{aligned}$$

Page 135



(3, 2, 1)卷积码的码字

输出序列分别为

$$v^{(1)} = (v_0^{(1)}, v_1^{(1)}, v_2^{(1)}, \dots), v^{(2)} = (v_0^{(2)}, v_1^{(2)}, v_2^{(2)}, \dots), v^{(3)} = (v_0^{(3)}, v_1^{(3)}, v_2^{(3)}, \dots)$$

的(3,2,m)的卷积码的码字为

$$v = (v_0^{(1)}, v_0^{(2)}, v_0^{(3)}, v_1^{(1)}, v_1^{(2)}, v_1^{(3)}, v_2^{(1)}, v_2^{(2)}, v_2^{(3)}, \dots)$$

举例: 图6.2的(3, 2, 1)卷积码, 若 $u^{(1)}=(101)$, $u^{(2)}=(110)$

$$V^{(1)} = u^{(1)} * g_1^{(1)} + u^{(2)} * g_2^{(1)} = (101) * (11) + (110) * (01) = (1111) + (0110) = (1001)$$

$$V^{(2)} = u^{(1)} * g_1^{(2)} + u^{(2)} * g_2^{(2)} = (101) * (01) + (110) * (10) = (0101) + (1100) = (1001)$$

$$V^{(3)} = u^{(1)} * g_1^{(3)} + u^{(2)} * g_2^{(3)} = (101) * (11) + (110) * (10) = (1111) + (1100) = (0011)$$

得到的码字为 $v=(110, 000, 001, 111)$ 。

Page 136



二元(3, 2, 1)卷积码的编码方程

举例: 图6.2的(3,2,1)卷积码的编码方程为

$$\begin{cases} V_i^{(1)} = u_i^{(1)} + u_{i-1}^{(1)} + u_{i-1}^{(2)} \\ V_i^{(2)} = u_i^{(2)} + u_{i-1}^{(1)} \\ V_i^{(3)} = u_i^{(1)} + u_i^{(2)} + u_{i-1}^{(1)} \end{cases}$$

另外, 编码方程中每一项的系数, 还可以表示输入、移位寄存器和输出之间的连接关系, 1表示有连接, 0表示没有连接

Page 137



(3, 2, m)卷积码的生成矩阵

(3, 2, m)卷积码的生成序列为 $g_i^{(j)}, i=1,2, j=1,2,3$

(3, 2, m)卷积码的生成矩阵为

$$G = \begin{bmatrix} g_{10}^{(1)} & g_{10}^{(2)} & g_{10}^{(3)} & g_{11}^{(1)} & g_{11}^{(2)} & g_{11}^{(3)} & \dots & g_{1m}^{(1)} & g_{1m}^{(2)} & g_{1m}^{(3)} \\ g_{20}^{(1)} & g_{20}^{(2)} & g_{20}^{(3)} & g_{21}^{(1)} & g_{21}^{(2)} & g_{21}^{(3)} & \dots & g_{2m}^{(1)} & g_{2m}^{(2)} & g_{2m}^{(3)} \\ g_{30}^{(1)} & g_{30}^{(2)} & g_{30}^{(3)} & g_{31}^{(1)} & g_{31}^{(2)} & g_{31}^{(3)} & \dots & g_{3m}^{(1)} & g_{3m}^{(2)} & g_{3m}^{(3)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \end{bmatrix}$$

举例: 图6.2的(3,2,1)的生成矩阵为

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ & 1 & 0 & 1 & 1 & 1 & 1 \\ & & 0 & 1 & 1 & 0 & 0 \\ & & & 1 & 0 & 1 & 1 & 1 & 1 \\ & & & & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Page 138



计算题举例：(3,2,m)卷积码的码字

以 u 为输入、 G 为生成矩阵的(3,2,m)卷积码的码字为
 $V=uG$ 。

举例：图6.2的(3,2,1)卷积码，若 $u^{(1)}=(101)$ ， $u^{(2)}=(110)$ ，即
 $u=(11, 01, 10)$ ，则码字为 $V=uG=(110, 000, 001, 111)$

$$V = uG = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ & 1 & 0 & 1 & 1 & 1 & 1 \\ & 0 & 1 & 1 & 1 & 0 & 0 \\ & & 1 & 0 & 1 & 1 & 1 & 1 \\ & & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$= [110, 110]$$

$$= (110, 000, 001, 111)$$

Page 139



(n, k, m)卷积码的生成矩阵

$$G = \begin{bmatrix} G_0 & G_1 & G_2 & \dots & G_m \\ G_0 & G_1 & G_2 & \dots & G_m \\ G_0 & G_1 & G_2 & \dots & G_m \\ \vdots & \vdots & \vdots & \ddots & \vdots \end{bmatrix}$$

其中每个 G_L 都是一个 $k \times n$ 阶子阵

$$G_L = \begin{bmatrix} g_{L0}^{(1)} & g_{L0}^{(2)} & \dots & g_{L0}^{(n)} \\ g_{L1}^{(1)} & g_{L1}^{(2)} & \dots & g_{L1}^{(n)} \\ \vdots & \vdots & \ddots & \vdots \\ g_{Ln}^{(1)} & g_{Ln}^{(2)} & \dots & g_{Ln}^{(n)} \end{bmatrix}$$

Page 140



卷积码编码器的存储级数

对于 (n, k, m) 卷积码，图6.1-6.2这两个例子表明，当
 $k > 1$ 时，编码器及用来描述它的符号都比较复杂，并且该
 编码器所含的 k 个移位寄存器的长度未必相同。如在图6.2
 中， $m=1$ 。

存储级数：若 k_i 是第 i 个移位寄存器的长度，则称所有 k 个
 移位寄存器中的最大长度为存储级数，即编码器的存储级
 数 m 定义为 $m = \max_{1 \leq i \leq k} k_i$

Page 141



卷积码编码器的约束长度和码速率

对于 (n, k, m) 卷积码，编码器中每个信息位要保持 $m+1$
 时间个单位，每个时间单位都可以影响编码器输出中的任
 何一个，这由移位寄存器的连接决定。

约束长度：对于 (n, k, m) 卷积码，编码器的约束长度定义为
 $n_A = n(m+1)$

约束长度可以解释成1比特信息对编码器输出可以造
 成影响的最大数目。

举例：图6.1中(2,1,2)卷积码 $n_A=6$ ，图6.2中(3,2,1)卷积码
 $n_A=6$ 。

码速率：对于 (n, k, m) 卷积码，其码速率 $r=k/n$

Page 142



(2, 1, m)卷积码的多项式描述

信息序列： u 表示成 $u(x)$ ，**生成多项式：** $g^{(1)}$ 表示成 $g^{(1)}(x)$ ， $g^{(2)}$ 表示
 成 $g^{(2)}(x)$ ，**输出序列：** $V^{(1)}$ 表示成 $V^{(1)}(x)$ ， $V^{(2)}$ 表示成 $V^{(2)}(x)$ ，

码字： V 表示成 $V(x)$ 。于是 $V^{(1)}(x)=u(x)g^{(1)}(x)$ ， $V^{(2)}(x)=u(x)g^{(2)}(x)$ ，

码字： $V(x)=V^{(1)}(x^2)+xV^{(2)}(x^2)$

举例：对于图6.1的(2,1,2)卷积码，**信息序列：**若 $u=(1011)$ ，

则 $u(x)=1+x^2+x^3$ ，**生成多项式：**冲激响应为 $g^{(1)}=(111)$ ，

$g^{(2)}=(101)$ ，则 $g^{(1)}(x)=1+x+x^2$ ， $g^{(2)}(x)=1+x^2$ ，于是，**编码方程为**

$V^{(1)}(x)=u(x)g^{(1)}(x)=(1+x^2+x^3)(1+x+x^2)=1+x+x^5$ ，

$V^{(2)}(x)=u(x)g^{(2)}(x)=(1+x^2+x^3)(1+x^2)=1+x^4+x^5$

码字为 $V(x)=V^{(1)}(x^2)+xV^{(2)}(x^2)=1+x+x^2+x^7+x^9+x^{10}+x^{11}$

Page 143



(n, k, m)卷积码的转移函数矩阵

由于编码器是线性系统，同任何有 k 个输入 n 个输出的线性系
 统一样，总共有 $k \times n$ 个转移函数。 $k \times n$ 阶转移函数矩阵为

$$G(x) = \begin{bmatrix} g_1^{(1)}(x) & g_1^{(2)}(x) & \dots & g_1^{(n)}(x) \\ g_2^{(1)}(x) & g_2^{(2)}(x) & \dots & g_2^{(n)}(x) \\ \vdots & \vdots & \ddots & \vdots \\ g_k^{(1)}(x) & g_k^{(2)}(x) & \dots & g_k^{(n)}(x) \end{bmatrix}$$

于是，码字 $V(x)=u(x)G(x)$ ，其中， $u(x)=(u^{(1)}(x), u^{(2)}(x), \dots,$
 $u^{(k)}(x))$ ， $V(x)=(V^{(1)}(x), V^{(2)}(x), \dots, V^{(n)}(x))$ ，并路之后，码字变成
 $V(x)=V^{(1)}(x^m)+xV^{(2)}(x^m)+\dots+x^{m-1}V^{(n)}(x^m)$

Page 144

图6.2的(3,2,1)卷积码, 2×3 阶转移函数矩阵为

$$G(x) = \begin{bmatrix} 1+x & x & 1+x \\ x & 1 & 1 \end{bmatrix}$$

对于输入序列 $u^{(1)}(x)=1+x^2$, $u^{(2)}(x)=1+x$

于是, 编码方程

$$V(x) = (v^{(1)}(x), v^{(2)}(x), v^{(3)}(x)) = (1+x^2, 1+x) \begin{bmatrix} 1+x & x & 1+x \\ x & 1 & 1 \end{bmatrix} \\ = (1+x^3, 1+x^3, x^2+x^3)$$

并路之后, 码字变成 $V(x) = V^{(1)}(x^2) + xV^{(2)}(x^2) + x^2V^{(3)}(x^2) =$

$$1+x+x^8+x^9+x^{10}+x^{11}$$

Page 143

在图6.3中, 向上的实线代表输入比特为0, 向下的虚线代表输入比特为1, 对应的码字输出表示在树图的分支路径上。如输入为(1011)则输出的码字为(11, 10, 00, 01)。

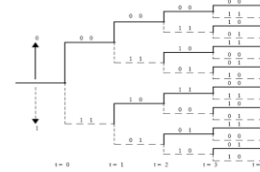


图6.3 图6.1卷积码的电路输入4个比特时对应的码树

Page 144

在图6.4中, 圆圈里是卷积码电路移位寄存器的状态, 每个输入比特都会导致卷积码电路图从一个状态到另外一个状态的转变; 每条路径边上标着 k/n , 表示输入信息比特 k 和输入码字 n 。默认从全0状态开始。如输入为(1011), 则状态转换序列为(10,01,10,11), 输出的码字为(11,10,00,01)。

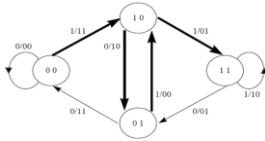


图6.4 图6.1卷积码的电路图对应的状态图

Page 147

Trellis图经常伴随一个图例(Legend), 给出状态的转换以及对应的输入输出 k/n 等。

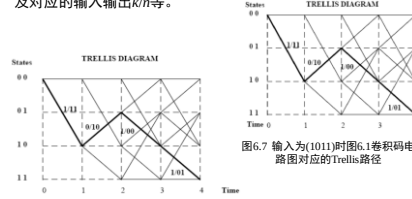


图6.6 图6.1卷积码的电路图对应的Trellis图和图例

Page 148

(n,k,m) 卷积码, 如输入序列为 x , 输出码字为 c , 经过有噪声的信号传输后, 得到接收序列 r , 然后经过Viterbi译码器, 得到序列 y 。

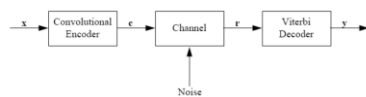


图6.8 卷积码编码及Viterbi译码系统

Page 149

Viterbi算法计算从接收序列到估值序列的最大似然估计使得 $p(r|y)$ 这个条件概率最大

对于码率为 r 的 (n,k,m) 卷积码, 输入的信息长度为 L , 则输入序列为

$$x = (x_0^{(1)}, x_0^{(2)}, \dots, x_0^{(k)}, x_1^{(1)}, x_1^{(2)}, \dots, x_1^{(k)}, \dots, x_{L+m-1}^{(1)}, x_{L+m-1}^{(2)}, \dots, x_{L+m-1}^{(k)})$$

对应输出的码字序列为

$$c = (c_0^{(1)}, c_0^{(2)}, \dots, c_0^{(n)}, c_1^{(1)}, c_1^{(2)}, \dots, c_1^{(n)}, \dots, c_{L+m-1}^{(1)}, c_{L+m-1}^{(2)}, \dots, c_{L+m-1}^{(n)})$$

其中下标表示时间。

消息序列的尾部需要 m 个0使得移位寄存器的状态归0。一般的, 默认移位寄存器从全零状态开始, 最终还是以全零状态结束。

Page 150

Viterbi译码算法是目前最大似然译码最知名一种方法，其原则为：

- (1) 错误不是经常出现的，发生错误的概率很小；
- (2) 在一行中发生2个错误的概率要比发生一个错误的概率小很多，即错误是随机分布的。

在工作过程中，Viterbi译码器计算给定长度的整个接收序列的度量。Viterbi译码器计算所有路径的度量并且基于这些度量进行决策。所有的路径都会进行度量累积，直到有2个路径收敛到同一个点上。这时，具有高度量的路径就是幸存者路径，否则就被丢弃。

最常用的度量就是汉明距离。

随着译码在Trellis图上按时间进行推进，路径上的度量累积起来，到最后，具有最大度量的那条路径就是幸存者路径。

例如：假设接收序列为(01, 11 01, 11, 01, 01, 11)，在Trellis图上用Viterbi译码算法进行译码。

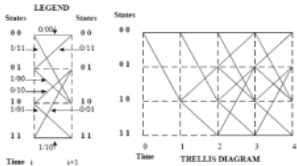


图6.6 图6.1卷积码的电路图对应的Trellis图和图例

步骤1: $t=0$ 时，接收向量是01，译码器最初状态是000。从这个点出发有2条备选路径，但是哪条路径上的向量跟接收向量都不匹配。译码器计算这2条路径度量，继续同时沿着两条路径往下进行，这2条路径度量都是1。

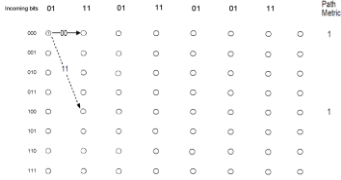


图6.9 Viterbi译码算法步骤($t=0$)

步骤2: $t=1$ 时，接收向量是11，译码器从这两个可能的状态出发，就有4条不同的路径，译码器计算这4条路径度量，并把这一些累加的尺度值标在图的右侧。

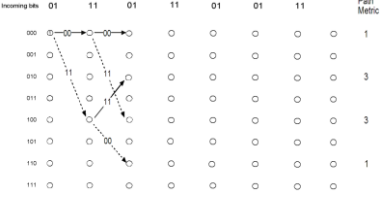


图6.10 Viterbi译码算法步骤($t=1$)

步骤3: $t=2$ 时，接收向量是01，译码器从这4个可能的状态出发，就有8条不同的路径，译码器计算这8条路径度量，把累加的尺度值标在图的右侧。

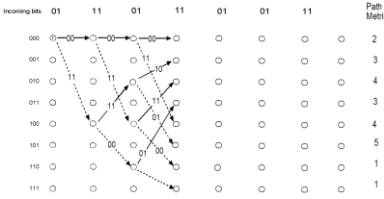


图6.11 Viterbi译码算法步骤($t=2$)

步骤4: $t=3$ 时，每个点都至少有一条到达它的路径。

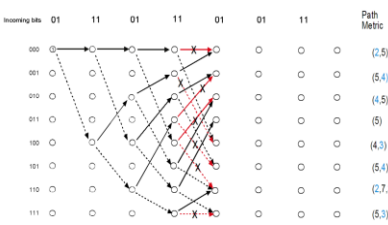


图6.12 Viterbi译码算法步骤($t=3$)

步骤5: $t=4$ 时, 根据最大似然准则, 将每行度量大的路径留下来。

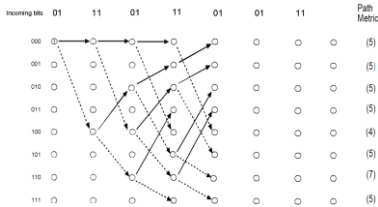


图6.13 Viterbi译码算法步骤($t=4$)

步骤6: $t=5$ 时, 继续把累加的尺度值标在右侧, 并且把每行度量大的路径留下来。

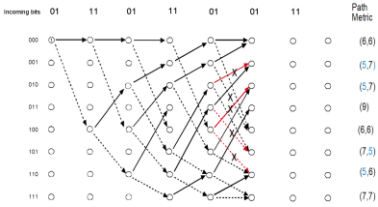


图6.14 Viterbi译码算法步骤($t=5$)

步骤7、步骤8都类似操作, 最终我们得到一条幸存者路径, 这条路径分别经过这些状态000, 100, 010, 101, 110, 011, 001, 000, 对应的译码后的码字也可以对应得到。

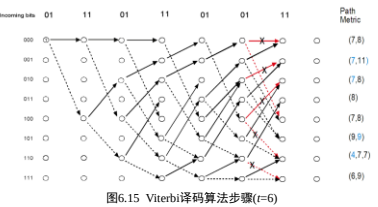


图6.15 Viterbi译码算法步骤($t=6$)

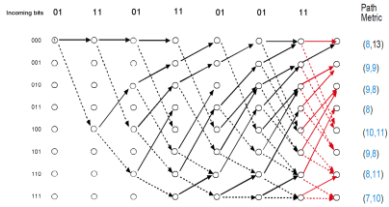


图6.16 Viterbi译码算法步骤($t=7$)

序列译码算法的描述最好是用类比。假设到一个地方去有多条路, 有的路上是有一些路标, 有些地方没有路标或者有路标但无法进行识别, 我们也许会走错路, 当我们意识到走错路的时候, 我们会回退到一个有路标的地点, 然后选择另外一条路径走下去, 如此往复, 直到我们到达目的地。在这个过程中, 我们有可能会回退多次。

在序列译码算法中, 类似的, 在Trellis图上, 每次我们只能沿着一条路径进行译码, 序列译码既允许往前走也允许往后退, 译码器记录我们的选择, 一旦走到两难的路上, 就会做标记, 当标记的数值不断增加, 超过某个预先设定的门限值的时候, 就会回退。

表6.1 (2,1,4)卷积码的真值表

Input Bits	Input State				Output Bits				Output State			
I_1	I_2	S_1	S_2	S_3	O_1	O_2	S_1	S_2	S_3	S_4	S_5	S_6
0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	1	1	1	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0	0	0	0	0	0
1	0	1	0	1	1	1	0	0	0	0	0	0
0	0	1	1	0	0	0	1	0	0	0	0	0
1	0	1	1	1	1	1	0	0	0	0	0	0
0	1	0	0	0	0	0	0	1	0	0	0	0
1	1	0	0	1	0	1	0	0	1	0	0	0
0	1	0	1	0	0	1	1	0	0	1	0	0
1	1	0	1	1	1	1	1	0	0	1	0	0
0	1	1	0	0	0	0	0	1	1	0	0	0
1	1	1	0	1	1	1	0	0	1	1	0	0
0	1	1	1	0	0	0	1	1	0	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	0

假设发送比特序列(1011000), 最后3比特是尾巴比特, 是为了使得移位寄存器可以归零。

如果没有任何错误发生, 将会接收到(11,11,01, 11,01,01,11), 但是现在实际接收到的是(01,11, 01,11,01,01,11), 发生了一个错误。

假设回退的门限为3, 下面我们序列译码算法进行译码如下。

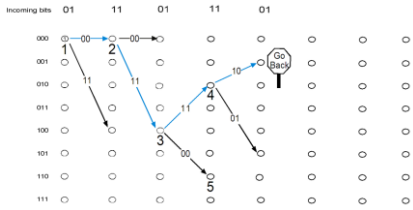


图6.17 序列译码算法

Page 143

- 点1: $t=0$ 时, 接收向量是01, 在点1, 译码器看两个路径, 哪个路径上的向量和接收向量01都不匹配, 并且走哪条路径, 都是发生了一个错误。假设译码器随机选择上面的分支路径00, 继续往下进行, 此时译码器把输入比特译成0, 把1放在误差计数器里。
- 点2: 接收向量是11, 所以直接选择下面的那条路径往下继续, 这步没有误差。
- 点3: 接收向量是01, 继续走哪条路径, 都是发生了一个错误, 假设译码器随机选择上面的分支路径11, 继续往下进行, 把1放在误差计数器里, 误差计数器里的数值增加1。
- 点4: 接收向量是11, 继续走哪条路径, 都是发生了一个错误, 假设译码器随机选择上面的分支路径10, 然后误差计数器里到达门限值3, 所以必须回退到点4, 选择下面的分支路径01, 然后误差计数器里到达门限值3, 所以就回退到点4的上一级点3。

Page 144

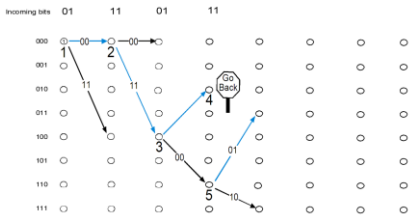


图6.18 序列译码算法

Page 145

- 点3: 选择下面那条路径00继续, 误差计数器里的数值增加1, 到达点5
 - 点5: 跟刚才到达点4的情况类似, 无论选择哪条分支路径, 误差计数器里到达门限值3, 所以就回退到点3的上一级点2继续
 - 点2: 选择上面的分支路径00, 此时误差计数器里到达门限值3, 所以回退到点1继续。
- 这样类似操作下去, 最终我们得到一条路径。

Page 146

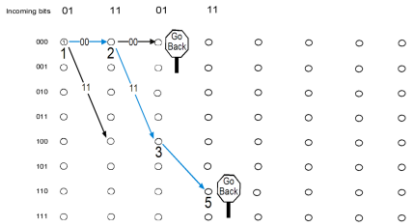


图6.19 序列译码算法

Page 147

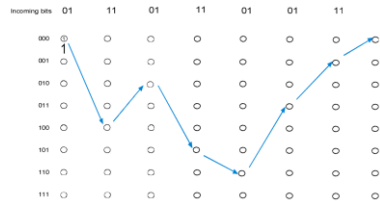


图6.20 序列译码算法

Page 148

第六章 Turbo码

李丽香, 彭海朋
北京邮电大学网络空间安全学院

Turbo码的研究背景

- 事实上，长期以来信道容量仅作为一个理论极限存在，实际的编码方案设计和评估都没有以香农限为依据。
- 随机性编译码被认为仅仅是为证明定理存在性而引入的一种数学方法和手段，在实际的编码构造中是不可能实现的。
- 1993年，两位任教于法国不列颠通信大学的教授C. Berrou、A. Glavieux和他们的缅甸籍博士生P. Thitimajshima在瑞士日内瓦召开的国际通信会议(ICC)上发表的论文《Near Shannon limit error-correcting coding and decoding: Turbo codes》中，提出了一种新的编码方法，即Turbo码。

课程思政

- Turbo码是一种级联码，利用迭代算法，以时间换取复杂度，第一次实现了切实可行的对香农极限的逼近。
- Turbo码的发明让人茅塞顿开，它不再是一种简单的编解码方案，而成为一种思想，人们基于这一思想开发了Turbo均衡、Turbo信道估计等应用，Turbo思想将信息理论推向了一个新的台阶。
- 而Turbo码在3G和4G中的应用，正是移动互联网大爆发的年代，它在无形之中服务信息社会，支撑其对数据传输的需求。
- Turbo码的发明的启示：就像学习中遇到的问题一样，解决方法并不是唯一的，大家要勤思考，找到解决问题的办法，提升自我解决问题的能力**

Turbo码的研究背景

- 香农有信道编码定理指出：在信道传输速率 R 不超过信道容量 C 的前提下，只有在码组长度无限的码集合中随机地选择编码码字并且在接收端采用最大似然译码算法时，才能逼近香农信道容量，误码率接近于零。
- 但是传统的编码都有规则的代数结构，远远谈不上“随机”；而且最大似然译码的复杂性随编码长度的增加而加大，当编码长度趋于无穷大时，最大似然译码是不可能实现的。出于译码复杂度的考虑，码长也不可能太长。因此，传统的信道编码性能与信道容量之间都有较大的差距。

Turbo码的研究背景

- C. Berrou等人提出的Turbo码就是并行级联卷积码，巧妙地将卷积码和伪随机交织器结合在一起，在实现随机编码思想的同时，通过交织器实现了由短码构造长码的方法，并在两个软入/软出(SISO)译码器之间进行多次迭代来逼近最大似然译码。
- Turbo码很好地应用了香农信道编码定理中的随机性编、译码条件，从而获得了几乎接近香农理论极限的译码性能。
- 从此以后，Turbo码得到了广泛的关注和发展，并对当今的编码理论和研究方法产生了深远的影响，信道编码理论的研究也随之进入了一个新的阶段。

Turbo码的优点

- 在信噪比较低的高噪声环境下性能优越（信道条件差的移动通信系统中有很大的应用潜力），而且具有很强的抗衰落、抗干扰能力。
- Turbo码具有超乎寻常的优异的译码性能，可以纠正高速率数据传输时发生的误码。在直扩（CDMA）系统中采用Turbo码技术可以进一步提高系统的容量。
- 在短帧情况下的仿真结果表明短交织Turbo码在AWGN信道和Rayleigh衰落信道下仍具有接近信道容量的纠错能力。

Turbo码的缺点

- 为获得较高的性能，Turbo码的编译码方式较为复杂。
- 具有较大的译码时延，这是由于block长度较大、译码需要多次迭代造成的。这样非常不利实时业务或高速数据的传输。
- BER(比特出错概率)在 10^{-5} 后会出现误码平台，这是由于Turbo码的重量分布造成的。对于某些对BER要求较高的应用就不适合，当然通过交织器的设计能够提供更大的码间最小距离，从而降低误码平台。

分量码

- 卷积码的类型多样，如非递归卷积码(NRC)、非系统卷积码(NSC)、递归系统卷积码(RSC)。
- 在实际的系统中，由于RSC有突出的优点而被广泛采用。RSC码在具有良好的译码性能，且存在较高的交织增益，译码性能也会随着交织长度增加而提高。
- 影响译码性能的因素很多，如卷积码的约束长度、生成多项式以及生成多项式的顺序等，因此，在实际中用不同种类的分量码构造的Turbo码，需要根据不同的场景和用户的需求，经过仿真和验证才能获得。

交织器

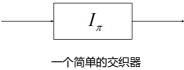
- 根据香农理论，码长越长，编码增益越大。因此，交织器的长度越长，Turbo码的码长越长，性能越好。
- Turbo码在交织器长度很长时可以获得优于卷积码的性能，但是在移动通信中短长度的帧是比较常见的，而在同等复杂度的情况下，Turbo码的性能是不如卷积码的。
- 另外，交织器的交织规则也是影响因素。最简单的交织器即行列交织器，数据按行写入，按列读出。
- 选用交织器时要根据交织器长度、交织规则、系统复杂度等综合考虑。
- 有分组交织器、伪随机交织器、循环移位交织器、分组螺旋交织器、二次算术交织器等。

Turbo码的应用

- 移动通信系统、卫星通信系统、数字音频广播、数字视频广播、深空通信、UMTS/3GPP、CDMA、信息隐藏领域、分布式信源编码、信源信道联合编码。
- 3G移动通信因为其业务类型对信道编码提出了更高的要求，而相应的WCDMA和CDMA2000方案都建议采用除与IS-95 CDMA系统类似的卷积编码技术和交织技术之外，采用Turbo编码技术。
- CDMA2000中，Turbo码被用在CDMA系统前向、反向链路信道中，RSC(递归系统卷积)编码器基于8状态的并行级联卷积码(8PCCC)。
- 在WCDMA中，Turbo码交织器是可载短型块交织器。

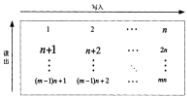
交织器

- 交织器的主要功能是使码重分布合理，降低数据序列的相关性，增大输出码字的最小汉明距，实现随机编码。
- 交织器将原始数据顺序按照给定的规则重新排序输出，能把通信产生的连续错误变成许多离散的单个错误，本质上改变序列的结构而不改变数据序列的内容。解交织即是交织的逆过程。



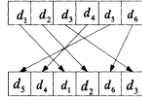
分组交织器

- **分组交织器**：将数据序列按行的顺序写入 $m \times n$ 的矩阵，然后按列的顺序读出，即完成交织过程。相应的解交织过程就是将交织后的数据序列按列写入 $m \times n$ 矩阵，再按照行的顺序读出。



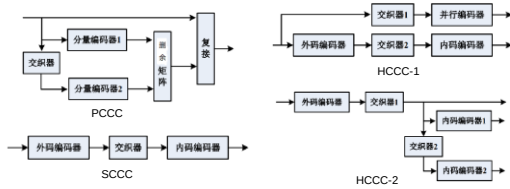
伪随机交织器

- **伪随机交织器**：指交织映射随机生成的交织器，每个长度为 N 的伪随机交织器共有 $N!$ 种可能的交织形式。这种交织器采用给定的随机地址交织映射，由这个已知的交织表对输入信息序列进行映射。



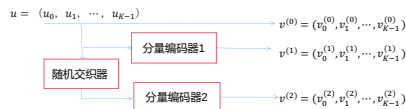
Turbo码编码器的结构

- 三种：并行级联卷积码(PCCC)、串行级联卷积码(SCCC)和混合级联卷积码(HCCC)



并行级联卷积码(PCCC)

- 第一个输出序列 = 信息序列，即 $u = v^{(0)} = (v_0^{(0)}, v_1^{(0)}, \dots, v_{K-1}^{(0)})$
- 第一个编码器输出的校验序列为 $v^{(1)} = (v_0^{(1)}, v_1^{(1)}, \dots, v_{K-1}^{(1)})$
- 交织器对 K 个比特进行排序处理，得到 u' ，第二个编码器输出的校验序列为 $v^{(2)} = (v_0^{(2)}, v_1^{(2)}, \dots, v_{K-1}^{(2)})$
- 最终的发送序列(码字)为 $v = (v_0^{(0)}, v_0^{(1)}, v_0^{(2)}, v_1^{(0)}, v_1^{(1)}, v_1^{(2)}, \dots, v_{K-1}^{(0)}, v_{K-1}^{(1)}, v_{K-1}^{(2)})$

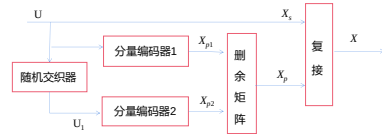


删余处理

- 一般的，Turbo码的码率只有1/3，所以频带利用率不高，为了解决这一问题，在编码器中加入了删余处理。
- 即在编码完成后，将分量码输出的两路校验信息输入删余器，按照一定的规则删除一部分校验信息，减小信息的冗余度，从而提高编码效率。
- 一般的删余确定冗余的规则是一位信息码至多需要一个校验位来验证错误，因此，对1/3码率的编码来说可以交替删余。
- 但Turbo码在经历交织后其第二个校验码已经不再与源码一一对应，此时交替删除还能起效的原因是因为turbo码的卷积特性使其每一位校验码对应着多位源码，因而在删余后其校验范围仍能包括整个源码。

并行级联卷积码(PCCC)

- 随机交织器：将信息序列 U 进行比特位置重排列得到 U_1 （内容不变）
- 分量编码器：生成校验序列 X_{p1} 和 X_{p2} （一般两个编码器结构相同）
- 删余矩阵：为了提高码率，周期地删除一些校验位，形成校验位序列 X_p
- 复接：未编码序列 X_u 与 X_p 经过复接后，生成Turbo码序列 X



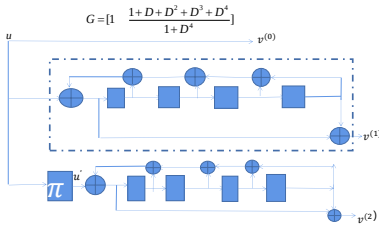
Turbo码的码率

- 若两个分量码的码率分别为 R_1 和 R_2 ，则Turbo码的码率为

$$R = \frac{R_1 R_2}{R_1 + R_2 - R_1 R_2}$$

Turbo码的编码过程举例

- 下图中，两个分量码都是(2,1,4)系统反馈编码器，具有相同的生成矩阵



注意事项

- 为了得到靠近香农限的系统性能，信息分组长度(交织器大小) K 一般比较大，通常至少几千个比特
- 对于分量码来说，一般选择相同结构，且约束长度较短，通常 $m \leq 4$
- 递归分量码会比非递归分量码有更好的性能
- 提高码率可通过删余矩阵实现
- 增加分量码和交织器会得到较低编码速率的Turbo码
- 最好的交织器能够比特以伪随机的方式进行排序，传统的块交织器(行列)在Turbo码中性能不好，除非块长度很短

Turbo码的译码

- Turbo码有一重要特点是其译码较为复杂，比常规的卷积码要复杂的多。
- 这种复杂不仅在于其译码要采用迭代的过程，而且采用的算法本身也比较复杂。
- 这些算法的关键是不但要能够对每比特进行译码，而且还要伴随着译码给出每比特译出的可靠性信息，有了这些信息，迭代才能进行下去。
- Turbo码译码算法：MAP(Maximum A Posteriori)、Max-Log-MAP、Log-MAP和SOVA(Soft Output Viterbi Algorithm)。

Turbo码的编码过程举例

- 删余矩阵的作用是提提高码率，元素取自集合{0,1}
- 矩阵的每一行对应两个分量编码器，“0”表示相应位置上的校验比特被删除，“1”表示保留相应位置的校验比特
- 对于1/3码率的Turbo编码器，输入序列 $U=(1011001)$ ，第一个分量编码器输出的校验序列 $X_{p1}=(1101000)$ ，第二个分量编码器输出的校验序列 $X_{p2}=(1001000)$ ，最终输出的序列为 $X=(111,010,100,111,000,000,100)$
- 添加删余矩阵 $P=\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ 提升到1/2码率，删除第一个分量编码器偶数位校验比特，删除第二个编码器奇数位校验比特，与系统输出复接后得到的序列为 $X=(11,00,10,11,00,00,10)$

注意事项

- 由于交织器只是对比位置进行重新排序，因此，交织后的序列 u' 与原始序列 u 具有相同的重量
- 对每个分量码来说，用BCJR(或MAP)算法作为SISO译码器能够获得最好的性能；因为MAP译码器使用了前向-后向算法，信息是以块的形式进行的，因此，对第一个分量译码器来说，附加 m 个0比特能够让它返回到全0态；但对于第二个译码器来说，由于交织器的作用，将不能返回到全0态

注：BCJR算法是一种定义在网格图上的用来最大化纠错编码的后验概率的算法，主要用于卷积编码。这种算法以它的发明者的名字命名，分别是Bahl、Cocke、Jelinek和Raviv。这个算法对于迭代的纠错编码来说是非常重要的，其中包括Turbo码和LDPC码。

译码算法

- **MAP算法**：MAP算法采用递归、迭代方法，将最大似然对数似函数作为软判决的输出。但由于该算法需要多次迭代，复杂度较高，运算量非常大，而且对数和指数的运算在数字电路中比较难以实现，导致其在工程实现上受到了很大程度的限制。
- **Max-log-MAP算法**：即最大值运算，在MAP算法基础上通过减少格图搜索状态来达到简化的目的。Max-log-MAP对MAP所做的修改是在对数域里对一些函数进行计算，这样省去了很多的指数的运算和对数域的运算，很大程度上降低了运算的复杂度，大大简化了运算量；但是由于在计算过程有一些近似处理，所以Max-log-MAP算法不是最优的。

译码算法

- **Log-MAP算法**：即对数域算法，Robertson等人在Max-log-MAP算法基础上进行了一些修改，使得Log-MAP算法相比于Max-log-MAP算法只多了一些查表及加法运算，尽管提高了一点复杂度但是性能却得到很大的提高。
- **SOVA算法**：即软输出Viterbi译码，SOVA算法在标准Viterbi译码算法上进行了一些修改：在进行最大似然路径选择时必须参考其先验信息；不但需要给出每个比特译码的准确结果，还需要计算出每个比特译码的可靠性；通过一些计算从中获取一些关于译码比特的先验信息，从而为下次迭代做好准备，大大减小了运算量，但性能受到了损失，大约损失了1dB。

MAP译码算法

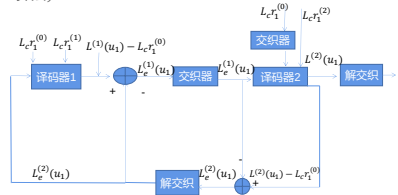
- MAP算法最初是用来估计无记忆噪声下的马尔可夫过程的，它是一种最优的算法。
- Bahl等人于1974年把它用于线性分组码和卷积码的译码中，在用于卷积码的译码时，对于给定接收序列Y，它不像Viterbi算法那样以栅格路径上的比特组错误最少为目的，而是以译码出来的符号 X_i 的错误最少为目的。
- 由于在卷积码的译码中，MAP算法要考虑栅格图中的所有可能路径，这样运算量就非常大，实际系统中很少用到。

MAP译码算法

- 虽然MAP算法早在1974年就被提出，但一直未被得到充分利用。
- 1993年Turbo码被提出来，MAP算法被用于Turbo码的译码之后，这种算法才得到广泛的应用。
- MAP算法不仅能译出序列的比特值，在译码的同时还能输出关于每比特译出的可靠性信息。这种特点正好符合了Turbo码的迭代译码特性，因此，被用于Turbo码的译码中。

迭代译码器

- 假设速率 $R = 1/3$ ，即没有删余处理的并行级联码，它使用了两个SISO译码器(MAP 算法)



MAP译码算法

- 每个时间单元，从信道接收到三个输出值，一个是对应着信息比特 u_1 ， $v_1^{(0)}$ ，用 $r_1^{(0)}$ 表示，其他两个对应着校验比特 $v_1^{(1)}$ 和 $v_1^{(2)}$ ，分别用 $r_1^{(1)}$ 和 $r_1^{(2)}$ 表示，这样接收到的向量(3K维)为 $r = (r_0^{(0)} r_0^{(1)} r_0^{(2)}, r_1^{(0)} r_1^{(1)} r_1^{(2)}, \dots, r_{K-1}^{(0)} r_{K-1}^{(1)} r_{K-1}^{(2)})$
- 现在假设每个发送比特的映射关系为 $0 \rightarrow -1$ 和 $1 \rightarrow +1$ ，软输出的AWGN 信道。

MAP译码算法

- 给定接收值 $r_1^{(0)}$ ，发送信息比特 u_1 的对数似然值(L值)定义为

$$L(v_1^{(0)} | r_1^{(0)}) = L(u_1 | r_1^{(0)}) \text{ (译码前):}$$

$$\begin{aligned} L(u_1 | r_1^{(0)}) &= \ln \frac{p(u_1 = +1 | r_1^{(0)})}{p(u_1 = -1 | r_1^{(0)})} = \ln \frac{p(r_1^{(0)} | u_1 = +1) p(u_1 = +1)}{p(r_1^{(0)} | u_1 = -1) p(u_1 = -1)} = \ln \frac{p(r_1^{(0)} | u_1 = +1)}{p(r_1^{(0)} | u_1 = -1)} + \ln \frac{p(u_1 = +1)}{p(u_1 = -1)} \\ &= \ln \frac{e^{-(E_s/N_0)(r_1^{(0)} - u_1)^2}}{e^{-(E_s/N_0)(r_1^{(0)} + u_1)^2}} + \ln \frac{p(u_1 = +1)}{p(u_1 = -1)} = 4 \frac{E_s}{N_0} r_1^{(0)} u_1 + \ln \frac{p(u_1 = +1)}{p(u_1 = -1)} = L_c r_1^{(0)} + L_a(u_1) \end{aligned}$$

其中 $\frac{E_s}{N_0}$ 是信道SNR， u_1 和 $r_1^{(0)}$ 都是用 $\sqrt{E_s}$ 归一化的， $L_c = 4E_s/N_0$ 是信道可靠度因子， $L_a(u_1)$ 是比特 u_1 的先验L值。

MAP译码算法

- 在发送校验比特是 $v_1^{(j)}$ 的情况下，给定接收到的值为 $r_1^{(j)}$, $j=1,2$, L 值(译码前)为：

$$L(v_1^{(j)}|r_1^{(j)})=L_c r_1^{(j)} + L_a(v_1^{(j)}) = L_c r_1^{(j)}, j = 1, 2$$

- 因为在一个线性码中信息比特等概率，校验比特为+1和-1的概率也是相同的，所以校验比特的先验 L 值为0，即

$$L_a(v_1^{(j)})=\ln \frac{p(v_1^{(j)}=+1)}{p(v_1^{(j)}=-1)}=0, \quad j=1, 2$$

- 对于译码器1的第一次迭代， $L_a(u_1)$ 也等于0，之后，信息比特的先验 L 值就被另一个译码器输出的外部 L 值所取代。

MAP译码算法

- 接收到的信道 L 值(软信息) $L_c r_1^{(0)}$ (for u_1)和 $L_c r_1^{(1)}$ (for $v_1^{(1)}$)进入译码器1，交织后的信道 L 值(软信息) $L_c r_1^{(0)}$ (for u_1) 和 $L_c r_1^{(2)}$ (for $v_1^{(2)}$) 进入译码器2，译码器1的输出包括两项：

- 给定接收到的向量(部分) $r_1 \triangleq [r_0^{(0)}, r_0^{(1)}, r_1^{(0)}, r_1^{(1)}, \dots, r_{K-1}^{(0)}, r_{K-1}^{(1)}]$ 以及先验输入向量 $L_a^{(1)} \triangleq [L_a^{(1)}(u_0), L_a^{(1)}(u_1), \dots, L_a^{(1)}(u_{K-1})]$ ，每个信息比特的后验 L 值(译码后)为

$$L^{(1)}(u_1) = \ln \left[\frac{p(u_1 = +1|r_1, L_a^{(1)})}{p(u_1 = -1|r_1, L_a^{(1)})} \right]$$

MAP译码算法

- 与每个信息比特相关的外部后验概率值(译码后)

$$(u_1) = L^{(1)}(u_1) - [L_c r_1^{(0)} + L_e^{(2)}(u_1)]$$

该信息经交织后送给译码器2，作为先验值 $L_a^{(2)}(u_1)$

- 同样地，译码器2的输出也包括两项：

- $L^{(2)}(u_1) = \ln \left[\frac{p(u_1 = +1|r_2, L_a^{(2)})}{p(u_1 = -1|r_2, L_a^{(2)})} \right]$ ，其中 r_2 是(部分)接收向量， $L_a^{(2)}$ 是先验输入向量。

MAP译码算法

- $L_e^{(2)}(u_1) = L^{(2)}(u_1) - [L_c r_1^{(0)} + L_e^{(1)}(u_1)]$ 是译码器2产生的外部后验 L 值，经过解交织后，作为先验值 $L_a^{(1)}(u_1)$ 反馈作为译码器1的输入。这样，每个译码器的输入包含三项：软信道 L 值 $L_c r_1^{(0)}$ 、 $L_c r_1^{(1)}$ (或 $L_c r_1^{(2)}$)，以及从另一个译码器传送的外部后验 L 值 $L_e^{(2)}(u_1)=L^{(1)}(u_1)$ (或 $L_e^{(1)}(u_1)$)
 $L_a^{(2)}(u_1)$ 。

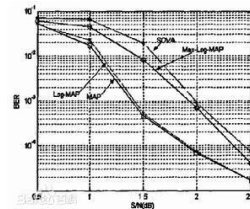
注意：在译码器1的初始迭代时，外部后验 L 值 $L_e^{(2)}(u_1)$

$L_a^{(1)}(u_1)$ 就是初始先验 L 值 $L_a(u_1)$ (对于等概信息比特，该值为0)，在译码器1的随后迭代中，先验 L 值 $L_a^{(1)}(u_1)$ 就用接收到的外部后验 L 值 $L_e^{(2)}(u_1)$ (进行解交织处理后)代替。

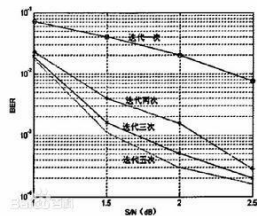
MAP译码算法

- 对于译码器2，其第一次迭代和随后的迭代都是相同的，外部后验 L 值一直都是 $L_e^{(1)}(u_1)$ 。译码迭代处理，每个译码器将其外部 L 值传送给另一个译码器，形成一个Turbo的效果，使得译码结果越来越可靠。
- 经过一定次数的迭代后，译码后的信息比特就从译码器2输出的后验 L 值 $L^{(2)}(u_1)$ 中判决得到，如果该 L 值为正，就判为+1，否则就判为-1。
- 值得一提的是：Turbo的很多特性与LDPC很类似，如编码方案都是产生随机重量分布的码、译码方法都是在迭代过程中利用了后验概率似然值、都利用了外部信息的概念等。实际上，正是由于Turbo码的出现，才使得人们对LDPC码的优点有了重新的认识。

不同译码算法对Turbo码性能的影响



不同迭代次数对Turbo码性能的影响



未来展望

- **亟待解决的问题**：译码算法的改进、复杂性的降低、译码时延的减小
- **研究方向**：最优分量码与交织器的联合设计、低复杂性译码算法、译码迭代过程的优化、收敛性以及迭代停止准则的设计、联合信道估计/多用户检测/均衡和译码算法、Turbo码与高阶调制技术的结合、Turbo编译码器的硬件实现、Turbo码在无线通信，移动通信以及多媒体通信中的应用



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第七章 二维码

李丽香, 彭海朋

北京邮电大学网络空间安全学院

网络与交换技术国家重点实验室





北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

条形码简史

20世纪20年代, 约翰·科芒德实现邮政单据自动分拣方法。

1959年, 吉尔·德塞伊塞尔申请了用条形码表示数字的专利。

70年代, UPC码 (Universal Product Code) 在北美超市得到广泛应用。

1997、2000年确定了国家标准。

1920195019591960197019882000

1949年, 美国乔·伍德兰德、伯尼·西尔沃发明了第一个条形码专利。

60年代, 西尔沃尼亚发明一套条形码系统并在北美铁路系统成功应用。

1988年, 中国成立了中国物品编码中心, 80年代末国外研制出二维码。

Page 208



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

二维码概念

二维码 (2-dimensional bar code), 也叫二维条码或二维形码, 它是用某种特定的几何图形按一定规律在平面 (二维方向上) 分布的黑白相间的记录数据符号信息的图形。它在代码编制上巧妙地利用构成计算机内部逻辑基础的0、1比特流的概念, 使用若干与二进制相对应的几何形体来表示文字数值信息, 通过图像输入设备或光电扫描设备自动识读以实现信息自动处理。

它的起源是日本, 原本是DensoWave公司为了追踪汽车零部件而设计的一种条码。



Page 209



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

二维码的分类与原理

二维码可以分为行排式二维码和矩阵式二维码。

二维码

- 行排式二维码
 - Code 16K
 - PDF417
 -
- 矩阵式二维码
 - Code One
 - QR Code
 -



PDF417



Code One

Page 210



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

行排式二维码

行排式二维码形态上是由多行短截的一维条码堆叠而成。

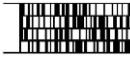
行排式二维码, 又称堆积式二维码或层排式二维码, 其编码原理是建立在一维条码基础之上, 按需要堆积成二行或多行。

它在编码设计、校验原理、识读方式等方面继承了一维条码的一些特点, 识读设备和条码印刷与一维条码技术兼容。但由于行数的增加, 需要对行进行判定, 其译码算法与软件也不完全相同于一维码。

有代表性的行排式二维码有Code 49、Code 16K、PDF417等。



Code 49



Code 16K



PDF417

Page 211



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

矩阵式二维码

矩阵式二维码以矩阵的形式组成。

矩阵式二维码, 又称棋盘式二维码, 它是在一个矩形空间通过黑、白像素在矩阵中的不同分布进行编码。

在矩阵相应元素位置上, 用点 (方点、圆点或其他形状) 的出现表示二进制“1”, 点的不出现则表示二进制“0”, 点的排列组合确定了矩阵式二维码所代表的意义。矩阵式二维码是建立在计算机图像处理技术、组合编码原理等基础上的一种新型图形符号自动识读处理码制。

有代表性的矩阵式二维码有Code One、QR Code、Data Matrix等。



Code One



1234567890123456789012
Data Matrix



QR Code

Page 212

1

- **可靠性强**
条形码的读取准确率远远超过人工记录，平均每15000个字符才会出现一个错误。
- **效率高**
条形码的读取速度很快，相当于每秒40个字符。
- **易于制作**
条形码的编写很简单，制作也仅仅需要印刷，被称为“可印刷的计算机语言”。
- **高密度**
二维码通过利用垂直方向的堆积来提高条码的信息密度，而且采用高密度图形表示。



300个字符或数字能编进这样大小的QR码里面



同样的数据QR码只有条码空间的1/11的大小

Page 213

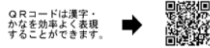
- **构造简单**
条形码识别设备的构造简单，使用方便。
- **灵活实用**
条形码符号可以手工键盘输入，也可以和有关设备组成识别系统实现自动化识别，还可以和其他控制设备联系起来实现整个系统的自动化管理。
- **成本低**
与其它自动化识别技术相比较，条形码技术仅仅需要一小张贴纸和相对构造简单的光学扫描仪，成本相当低廉。
- **纠错功能**
二维码不仅能防止错误，而且能纠正错误，即使条形码部分损坏，也能将正确的信息还原出来。



QR码损坏一部分依然能还原信息

Page 214

- **多语言形式，可表示图像**
二维码具有字节表示模式，即提供了一种表示字节流的机制。不论何种语言文字它们在计算机中存储时以机内码的形式表现，而内部码都是字节码，可识别多种语言文字的条码。
 - **具有加密体制**
可以先用一定的加密算法将信息加密，再用二维码表示在识别二维码时，再加以一定的解密算法，便可以恢复所表示的信息。
- 除此之外，二维码还具有**可表示信息量密度高、尺寸大小比例可变**、**可以使用激光或CCD阅读器方便识读**等众多优点。



QR码能够有效地表示中国汉字、日本汉字

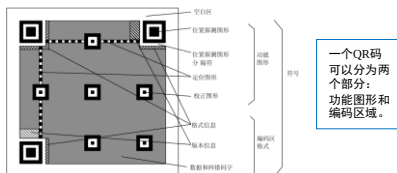
Page 215

- 目前二维码方面流行三大国际标准：
- PDF417：不支持中文；
 - DM（Data Matrix）：专利未公开，需要支付专利费用；
 - QR Code：专利公开，支持中文。

QR Code（全称Quick Response Code）是由日本Densho公司于1994年9月研制的一种矩阵式二维条码，它具有以下特点：

1. 高密度编码，信息容量大
2. 编码范围广
3. 容错能力强，具有纠错功能
4. 译码可靠性高
5. 可引入加密措施
6. 成本低，易制作，持久耐用
7. 条码符号形状、尺寸大小比例可变
8. 二维条码可以使用激光或CCD阅读器识读，QR Code具有全方位（360°）识读特点
9. 能够有效地表示中国汉字

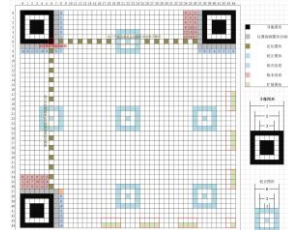
Page 216



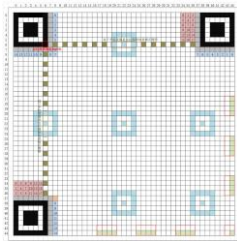
QR码共有40个版本（版本1~版本40），其中版本1的模块数为21*21，而每增加一个版本，其每边的模块数各增加4。因此版本40的模块数为177*177。

Page 217

版本7的模块数为45*45。
以版本7为例，介绍QR码各个部分的结构。



Page 218



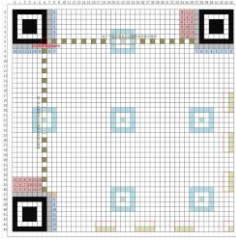
寻像图形、位置探测图形：
协助扫描软件定位QR码并转换坐标系。

寻像图形包括三个相同的位置探测图形，分别位于图形中的左上角、右上角和左下角。每个位置探测图形由7*7个模块组成。如图，寻像图形为黑色区域。

位置探测图形分隔符：
区分功能图形和编码区域。

每个位置探测图形和编码区域之间宽度为1个模块的分隔符。位置探测图形分隔符为灰色区域，此区域应为空白。

Page 219



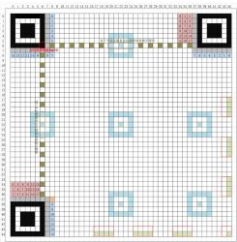
定位图形：
确定符号的密度和版本，提供决定模块坐标的基准位置。

定位图形为图中的茶色区域。水平和垂直的定位图形分别始于第6行和第6列。

此外，坐标 (6,6) 同时是位置探测图形和定位图形，且为定位图形的开始坐标。第6行和第6列中的6个水绿色部分也是定位图形。

定位图形是由深色与浅色模块交替组成，开始与结尾都是深色模块。

Page 220



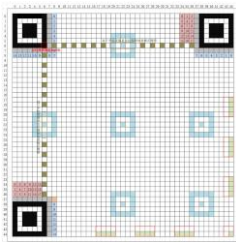
校正图形：
在图像有一定程度损坏的情况下，译码软件可以通过它同步图像模块的坐标映像。

校正图形的数量视符号和版本号而定，版本1没有校正图形，版本2及以上均含有校正图形。

如图，版本7中，校正图形为水绿色区域。

校正图形为固定的参照图形，每个校正图形由5*5的模块组成。可以通过中心模块行列坐标值推出校正图形的具体位置。

Page 221



格式信息：
存放纠错等级和掩模信息。

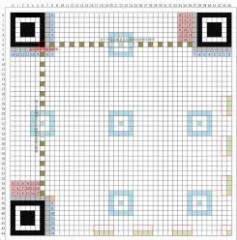
格式信息是一个15位数据，由2位纠错指示符+3位掩模图形参考+10位纠错码组成，为图中深蓝色区域。

同时，左下角格式信息编号8上方的橙色区域，此区域永远为深色模块，不用于存放任何信息。

版本信息：
用于存放QR码的版本号。

为图中红色区域，两个3*6模块。

Page 222



扩展图形：
最初的目的用于将来对QR码功能的扩展，并不用于对数据的编码。

扩展图形由位于符号右下角的一个4个模块组成的方块以及位于符号右边和下边的一些8个模块的块组成。8个模块的块的数目取决于符号的版本，计算公式为：

$8 \text{ 模块的数目} = 2 * (N \text{ DIV } 2)$
其中N为版本号，DIV表示除法运算。

扩展图形为图中红框区域，即版本7中有6个8模块组成的块和1个4模块组成的块。

Page 223

QR码的纠错能力：

级别	纠错能力
L	约7%
M	约15%
Q	约25%
H	约30%

纠错能力相当于全部码字（组成数据的单位，在QR码中，1码字相当于8比特的）恢复率。

Page 224

QR码的编码流程如下：

1. 数据分析
确定要进行编码的字符类型，选择所需的版本信息和纠错版本。
2. 数据编码
采用既定规则，将数据字符转换为位流，并加必要符号，后将位流转换为码字（每8位一个码字）。
3. 纠错编码
生成纠错码字放到数据码字后。
4. 构造最终信息
按规定将每一块中置入数据和纠错码字，必要时加剩余位。
5. 在矩阵中布置模块
将寻像图形、分隔符、校正图形与码字按规则排列，放入二维码矩阵。
6. 掩模
用8种掩模图形依次对符号的编码区域的位置图进行掩模处理，评价所得的8种结果，选择最优的一种。
7. 格式和版本信息
生成版本信息和格式信息，构成符号。

Page 225

表2：QR码可表示的数据类型及其相应的模式指示符。

表3：各版本及模式所对应的字符计数指示符的长度。

表2 模式指示符	
模式	指示符
ECI	0111
数字	0001
字母数字	0010
8位字节	0100
日本汉字	1000
中国汉字	1101
结构延伸	0011
FNC1	0101 (第一位置) 1001 (第二位置)
终止符 (信息结束)	0000

表3 字符计数指示符的长度					
版本	数字 模式	字母数字 模式	8位字节 模式	日本汉字 模式	中国汉字 模式
1~9	10	9	8	8	-
10~26	12	11	16	16	-
27~40	14	13	16	12	-

Page 226

举例说明QR码的编码过程。

➤ 数据分析：确定编码的字符类型、版本信息和纠错等级。

假设：
字符类型：数字模式 “01234567”
版本信息：版本1
纠错等级：H

Page 227

➤ 数据编码

- 数据字符转换为位流
首先将数字每三位分为一组，然后将每组数据转换为10位长度的二进制。如果最后一组只有两位，则使用7位长度的二进制；如果只剩一位，则使用4位长度。最后将每组的二进制连接起来形成数据位流，并在前面加上模式指示符和字符计数指示符。

- 如：01234567
1. 分组：012 / 345 / 67
 2. 转换为二进制：
012：000001100（加粗的0是为了凑够10位长度）
345：0101011001
67：1000011
 3. 数据位流：000001100 0101011001 1000011

Page 228

➤ 数据编码

- 数据字符转换为位流
- 4. 确定模式指示符和字符计数指示符
根据表2，数字模式下的模式指示符为0001；
根据表3，数字模式下版本1的字符计数指示符的位数为10。

“01234567”共有8位数字，则将位数8转换为10位长度的二进制。
8：000001000

表2 模式指示符	
模式	指示符
ECI	0111
数字	0001
字母数字	0010
8位字节	0100
日本汉字	1000
中国汉字	1101
结构延伸	0011
FNC1	0101 (第一位置) 1001 (第二位置)
终止符 (信息结束)	0000

版本	数字 模式	字母数字 模式	8位字节 模式	日本汉字 模式	中国汉字 模式
1~9	10	9	8	8	-
10~26	12	11	16	16	-
27~40	14	13	16	12	-

Page 229

➤ 数据编码

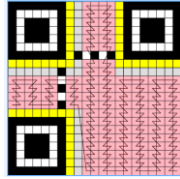
- 数据字符转换为位流
- 5. 最终位流：模式指示符+字符计数指示符+数字位流
0001 + 000001000 + 000001100 0101011001 1000011

Page 230

在矩阵中布置模块

- 右图展示了数据码字填写时的走位
- 符号字符的布置
在QR码符号的编码区域中，
符号字符以2个模块宽的纵列从符
号右下角开始布置，并自右向左，
且交替地向下向上或从上向下安排。

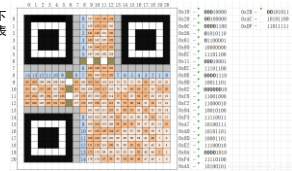
具体走位路线如右图。



Page 237

在矩阵中布置模块

在上一个步骤中，我们已经生成了最后的数据码字：
0x10 0x20 0x00 0x56 0x61 0x80 0xE0 0x11 0xE0 0x9D 0x02
0xC8 0x62 0x94 0xF3 0xA7 0xA0 0x8D 0xE2 0x0A 0xF4 0xA5 0x2B
0xAC 0xDF
将这26个16进制数转换为8为的二进制数，并按照上一页的路线依次
填入到对应位置。
最终结果如下
图。其中浅色代表
0，深色代表1。



Page 238

掩模

QR码中如果出现大面积的空白或者黑块，会导致扫描器识别困难。为了让QR图形看起来尽可能混乱，且尽可能避免位置探测图形中的位图1011101的出现，需对QR图形进行掩模操作，步骤如下：

- 掩模不用于功能图形及格式图形，如寻像图形、定位图形、校正图形、位置探测图形分隔符、格式信息和版本信息；
- 数据码字与掩模图形进行异或操作后再进行绘制；
- 对每个结果图形的不合要求的部分计分，以评估这个结果；
- 选择得分最低的结果。

Page 239

掩模

表23给出了掩模图形的参考（放置于格式信息中的二进制参考）和掩模图形生成的条件。

掩模图形是通过编码区域（不包括格式信息和版本信息）内那些条件为真的模块定义为深色而产生的。
i代表模块的行的位置，j代表模块的列的位置，其中i和j均从0开始。

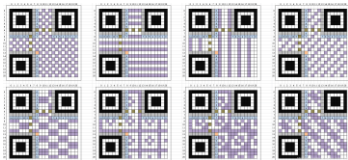
表23掩模图形的生成条件。

掩模图形参考	条件
000	$(i+j) \bmod 2 = 0$
001	$\text{row}(i) = 0$
010	$\text{row}(j) = 0$
011	$(i+j) \bmod 3 = 0$
100	$((i \bmod 2) + (j \bmod 3)) \bmod 2 = 0$
101	$(i \bmod 2) = (j \bmod 3) = 0$
110	$((i \bmod 2) + (j \bmod 3)) \bmod 2 = 0$
111	$((i \bmod 2) + (j \bmod 3)) \bmod 2 = 0$

Page 240

掩模

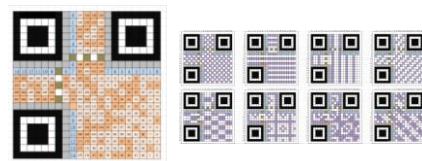
下图为版本1符号的所有掩模图形。
分别模拟了用掩模图形参考000、001、010、011、100、101、110、111的掩模结果。



Page 241

掩模

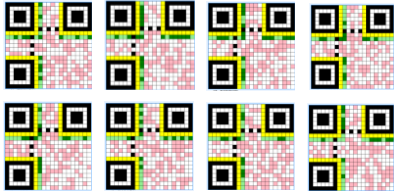
掩模数据是将左图的橙色区域的数据信息分别与右图的8种情况下的紫色部分进行异或操作得到的，掩模的最终结果在下页显示。



Page 242

掩模

下图的粉色区域分别为掩模为000、001、010、011、100、101、110、111得到的最终结果。



Page 243

格式和版本信息

计算出纠错码和格式信息

根据表25，可以得到纠错等级H的二进制指示符为10。
假设使用的掩模图形参考为011。

将2位纠错指示符和3位掩模图形参考连接在一起，得到5位的**数据码**，并使用BCH(15,5)编码计算得到**纠错码**。其中，BCH(15,5)表示BCH码的总长度为15，数据码为5位，纠错码为10位。

表25纠错等级指示符

纠错等级	二进制指示符
L	01
M	00
H	11
E	10

Page 244

格式和版本信息

计算出纠错码与格式信息

- 连接2位**纠错指示符** + 3位**掩模图形参考**：10 + 011
- 由于BCH码总长度为15，因此需要将10011凑够15位，在数据后方补0：100110000000000
- 计算**纠错码**，纠错码为数据码除以生成码所得的余数，因此：假设BCH码使用的生成码为：10100110111
纠错码=100110000000000 % 10100110111，即0111000010
- 则**最终格式信息码**：2位纠错指示符 + 3位掩模图形参考 + 10位纠错码，即
10 + 011 + 0111000010

格式信息的掩模

作用：确保纠错等级和掩模图形参考合在一起的结果不全为0。
需要将15位格式信息码与掩模图形101010000010010进行异或运算。

Page 245

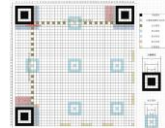
格式和版本信息

格式信息的掩模

格式信息码	100110111000010
掩模图形	101010000010010
异或结果	001100111010000

将掩模后的格式信息按照序号填入对应位置，即下图中的蓝色区域中。

对应位	14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
格式信息	0 0 1 1 0 0 1 1 1 0 1 0 0 0 0



Page 246

格式和版本信息

计算版本信息

使用BCH(18, 6)编码计算得到纠错码。原理同上，BCH(18, 6)表示BCH码总长度为18位，其中数据码为6位，纠错码12位。
版本信息的计算和格式信息类似，也是使用**长除法**。只是这一次使用的生成码为：1111100100101，使用数据码除以生成码，所得余数就是纠错码。

由于版本1无需绘制版本信息，因此暂时使用版本7来进行版本信息生成的讲解。

- 将版本号7转换为6位长度的二进制数据码
7：000111
- 由于这里的BCH码总长度为18，需将000111凑够18位，于是在数据后方补0得到数据码：
000111000000000000
- 纠错码=0011100000000000 % 1111100100101
即为110010010100

Page 247

格式和版本信息

计算版本信息

- 6位数据码 + 12位纠错码即为最终版本信息码
000111 + 110010010100(0x7c94)
与格式信息不同，版本信息码生成后不再需要单独进行掩模运算。
- 将结果分别填入左下角和右上角的版本信息中，注意顺序。数字是倒着填入的，即末尾数字对应编号0，开头数字对应编号17，并且编号是从0开始的。

对应位	17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
版本信息	0 0 0 1 1 1 1 1 0 0 1 0 0 1 0 1 0 0

Page 248

最终结果：



识别步骤

1. 条码定位
2. 条码分割
3. 解码



条码定位

条码的定位就是找到条码符号的图像区域，对有明显条码特征的区域进行定位。然后根据不同条码的定位图形结构特征对不同的条码符号进行下一步的处理。



条码定位

- 定位采用以下步骤：
1. 利用点运算的阈值理论将采集到的图像变为二值图像，即对图像进行二值化处理；
 2. 得到二值化图像后，对其进行膨胀运算；
 3. 对膨胀后的图像进行边缘检测得到条码区域的轮廓。



条码定位

- 定位采用以下步骤：
4. 确定寻像图形
 5. 探测图形中心坐标
 6. 确定两个举例
 7. 确定版本号
 8. 构造位图
 9. 得到纠错等级和掩模图形



QR Code的定位图形

条码分割

- 分割采用以下步骤：
1. 将原图像按比例缩小进行分割，计算其特征值；
 2. 分块继承父块纹理类别，结合其周围纹理类型进行修正；
 3. 重复步骤二直至图像被划分为2*2大小，分割结束；
 4. 分割结束后，图中可能出现的孤立的小区域可作为噪声删除。

> 解码

得到一幅标准的条码图像后，对该符号进行网络取样，对网格每一个文点上的图像像素取样，并根据阈值确定是深色块还是浅色块。构造一个位图，用二进制的“1”表示深色像素，“0”表示浅色像素，从而得到条码的原始二进制序列列值，然后对这些数据进行纠错和译码：

1. 异或处理 (XOR)
2. 确定符号码字
3. 重新排列码字序列
4. 执行错误检测和纠错译码程序

最后根据条码的逻辑编码规则把这些原始的数据位流转换成数据码字。

Page 255

> 食品追溯

原材料供应商在向食品厂家提供原材料时候进行批次管理，将原材料的原始生产数据制造日期、食用期限、原产地、生产者、遗传基因组合等信息录入到二维码中，并打印带有二维码的标签，粘贴在包装箱上后交给食品厂家。

此外，消费者购买产品时，需要手机扫码或编辑码号发短信，即可随时查询产品信息与质量认证等信息，并可及时举报虚假、错误信息。



Page 256

> 餐厅应用

手机电子菜单：

通过将餐饮店中的菜品介绍、餐饮文化等信息录入电子菜单中，用户通过扫码获得该手机网站的跳转链接获取商家相关信息。



Page 257

> 景点门票应用

景点门票、火车票等告别传统的文字纸张模式，采用二维码进行售票、检票等，可以大幅度提高通行效率，并且防止仿票。



Page 258

> 车辆管理应用

行驶证、驾驶证、车辆的年审文件等都印制有二维码，可以将有关车辆上的基本信息都保存在二维码中，信息的数字化便于管理部门的管理网络实施实时监控。



Page 259

> 课堂思政

上述举的这些例子，都是二维码真正应用于生活中的，并且随着技术的不断发展、移动设备的快速普及，二维码在生活中得到了越来越广泛的应用。你能想到的还有哪些非常普及的二维码应用方式呢？

古往今来，有许多的科研成果都投入到实际之中，并且获得广泛、实用的良好效果。当代大学生在平时也要坚持解决思想问题和解决实际问题相结合的思想，既要重视道理又要办实事，从而完成自身的全方位发展，并且能够为建设中国现代化社会主义强国作出贡献。

Page 260

二维码的制作

二维码是由很多0、1组成的数字矩阵。二维码是用某种特定的几何图形按一定规律在平面（二维方向上）分布的黑白相间的图形记录数据符号信息的；在代码编制上巧妙地利用构成计算机内部逻辑结构基础的“0”、“1”比特流的概念，通过图像输入设备或光电扫描设备自动识读以实现信息自动处理。同时它具有条码技术的一些共性。二维码生成器的制作需要一个二维码生成算法，或者一个二维码插件，然后用JAVA，C#，VB等编程语言编写一个调用软件就可以做成二维码生成器了。

二维码的代码实现-java

```
import java.awt.Color;
import java.awt.Graphics2D;
import java.awt.Image;
import java.awt.image.BufferedImage;
import java.io.File;
import javax.imageio.ImageIO;
import com.swetake.util.Qrcode;

public static void getQrcodeImage(String content, String imgPath)
{
    int width = 235;
    int height = 235;
    // 实例化一个对象
    Qrcode qrcode = new Qrcode();
    // 编码方式
    qrcode.setQrcodeMode('W');
```

二维码的代码实现-java

```
// 设置分辨率
qrcode.setQrcodeErrorCorrect('M');
// 二维码的版本
qrcode.setQrcodeVersion(15);
// 绘制二维码
// 画板
BufferedImage image = new BufferedImage(width, height,
    BufferedImage.TYPE_INT_RGB);
// 画板
Graphics2D gs = image.createGraphics();
// 设置背景颜色 白色
gs.setBackground(Color.white);
// 设置二维码的颜色
gs.setColor(Color.black);
```

二维码的代码实现-java

```
// 创建一个二维码的绘制区域
gs.clearRect(0, 0, width, height);
byte[] codeOut;
try {
    codeOut = content.getBytes("utf-8");
    boolean[] code =
        qrcode.calQrcode(codeOut);
    for (int i = 0; i < code.length; i++) {
        for (int j = 0; j < code.length; j++) {
            if (code[i][j]) {
                gs.fillRect(i * 3 + 2, j * 3 + 2, 3, 3);
            }
        }
    }
}
```

二维码的代码实现-java

```
// 加载logo D:/111.png 这个你需要改成你自己的
File file = new File("D:/111.png");
Image srcImage = ImageIO.read(file);
int imgWidth = srcImage.getWidth(null);
int imgHeigh = srcImage.getHeight(null);
gs.drawImage(srcImage, 83, 83, imgWidth, imgHeigh, null);
// 释放资源
gs.dispose();
image.flush();
```

二维码的代码实现-java

```
// 写入指定路径
ImageIO.write(image, "png", new File(imgPath));
System.out.println("二维码生成成功了！");
} catch (Exception e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
}

public static void main(String[] args) {
    getQrcodeImage("hello!", "d/xxx.png");
}
```



北京邮电大学

BEDING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第八章 极化码

李丽香, 彭海朋

北京邮电大学网络空间安全学院

网络与交换技术国家重点实验室





北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

极化（Polar）码起源

极化码（Polar Code）是由土耳其 Bilkent 大学的 Erdal Arkan 教授于 2008 年发明的一种信道编码方法。Polar 码是目前唯一的一种能够达到香农极限（Shannon Limit）的编码方法，而比较其他编码方法只是接近香农极限。也是目前国际移动通信标准化组织认定的 5G 场景控制信道的编码方案。



Page 265



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

极化码优缺点

极化码优点：

1、是目前唯一的香农信道容量可达的编码方式

2、有坚实的理论基础

3、编解码复杂度低

4、精细的码率调整机制（信息块长度可以一比特一比特的递减）

5、极化码的递归特性易于通过硬件实现（母码长度为 N 的 Polar 码可以用两个母码长度为 N/2 的 Polar 码实现）

极化码缺点：

1、汉明距离较小，在短码情况下影响解码性能，一定程度上可以通过选择合适的冻结比特位置进行规避。适合长码编码。

2、SC 译码延迟较大。

Page 266



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

国外研究现状

2009

日本京都大学的教授使用密度递进的方法研究了 Polar 码的性能。

2013

英国德州大学的 L. S. 研究了在衰落信道下的分段 Polar 码，其编码方案达到了衰落 DM-CC 下的性能。

2014

加拿大和 P. Guo 研究了在普通 CPU 上实现的 Polar 码软件解码算法，其吞吐量超过了 200Mbps。

2017

在美国 Spokane 的 W. P. RAN WG10B 会议上，李丽香和特翻特模块的 Polar 码得以确定在 5G 中使用

2018

美国 RAN 的 W. P. TSG RAN WG10B 会议上，Polar 码被确定为第五代移动通信 5G 的增强移动宽带 eMBB 信道控制信道的编码方案。

2019

在 5G 的 W. P. TSG RAN 会议上，下一代新的无线接入网（NR，5G，5G-NR）的研究项目的立项通过，Polar 码在短码长，低码率上的性能优势。

Page 267



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

国内研究现状

- 在国内，北京邮电大学是最早研究 Polar 码的机构之一。牛凯教授的团队研究了 Polar 码的编码和解码算法等。
- 西安电子科技大学李颖教授的团队研究了 Polar 码的高斯近似的码构造方法和解码算法。
- 哈尔滨工业大学王学东教授的团队研究了 Polar 码的编码与译码算法。

Page 268



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

极化（Polar）码起源

截止频率越高，系统响应越快
因此 Polar 码的最初设计的出发点是为了提升截止频率

在随机编码和最大似然条件下，信道截止速率 R_0 决定了相应的码块的错误概率：

$$P_e = 2^{-N R_0}$$

其中，N 为码块长度。实际信道的传输速率 R 小于 R_0 ，使用随机编码和 ML 编码时，信道平均错误概率：

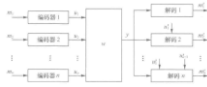
$$\bar{P}_e = 2^{-N (R_0 - R)}$$

Page 269

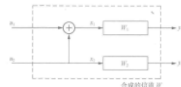
1

因此，在2004-2006年，Arikan 教授尝试使用信道合并与分离的方法来提升信道截止速率。

对于一个离散无记忆的信道 W ，分离出两个相关的子信道 W_1 和 W_2 ，有 $C(W_1)+C(W_2)\leq C(W)$ ，同时有 $R_0(W_1)+R_0(W_2)>R_0(W)$ 。其中 C 是信道容量， R_0 是信道截止速率，即两个子信道信道容量不超过原本的总信道容量，但分离后总信道截止速率提高了。



信道分离示意图



信道合并示意图

Page 270

对于Polar码基本原理可以归结为三点：信道合并、信道分离、信道极化。其中信道合并和信道极化是在编码时完成的；信道分离是在解码时完成。

信道选择：

对于编码方案通常根据某种信道设计，Polar码主要考虑以下信道进行分析：

二进制离散无记忆信道 (B-DMC) $\left\{ \begin{array}{l} \text{二进制删除信道 (BEC)} \\ \text{二进制对称信道 (BSC)} \end{array} \right.$

Page 271

信道 $W: X \rightarrow Y$ 。输入变量 X 只能取 $\{0,1\}$ 中的值，输出变量 Y 取任意值，信道迁移概率为 $P=W(y|x)$ ， $x \in X$ ， $y \in Y$ 。



信道示意

对于对称信道 (BEC/BSC)，当输入在 $\{0,1\}$ 概率时，其信道容量定义为输入输出的相互信息。

$$C(W)=I(X;Y)=\sum_{y \in Y} \sum_{x \in X} \frac{1}{2} W(y|x) \log \left(\frac{W(y|x)}{\frac{1}{2} W(y|0) + \frac{1}{2} W(y|1)} \right)$$

Page 272

信道合并（编码时）：

将两个BEC信道 W_1 和 W_2 合并成一个信道 W ，其中，“+”为模2加。由于 u_1 和 u_2 都取自 $\{0,1\}$ 且独立同分布，那么信道容量为：

$$C(W)=I(U;Y)$$

$$\text{其中，} U=[u_1, u_2], Y=[y_1, y_2]$$

$$C(W)=I(X;Y)$$

$$\text{其中，} X=[x_1, x_2]$$

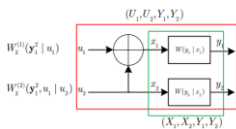
$$C(W)=I(x_1, y_1) + I(x_2, y_2) = C(W_1) + C(W_2) \\ \approx 2 \times C(W_1) = 2 \times C(W_2)$$

也就是说，信道合并并保持总信道容量守恒。若 N 个信道合并，则

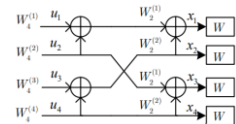
$$C(W) \approx N \cdot C(W_1)$$

即，合成信道的总容量是各个子信道容量之和，容量守恒。

Page 273

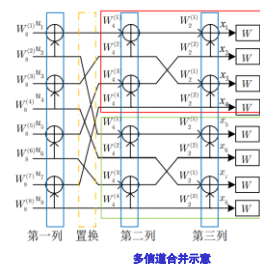


双信道合并示意



4个BEC信道合并示意

Page 274



多信道合并示意

第二列和第三列的上半部分是长度为4的极化码

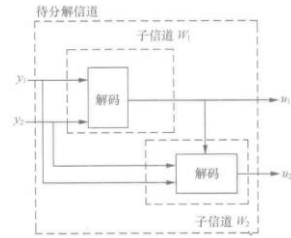
第二列和第三列的下半部分是长度为4的极化码

Page 275

信道分离（解码时）：

信道分离是将混有多个输入比特信息的接收信息 $y_1^N = (y_1, y_2, y_3, \dots, y_N)$ 分解成独立的多个输入比特信息 $u_1^N = (u_1, u_2, u_3, \dots, u_N)$ 。考虑针对G2 Polar 码编码而成的信道解码过程。首先解调器从 $y^N = y^2 = (y_1, y_2)$ 分解出第一个子信道 u_1 。之后再从 (u_1, y_1, y_2) 中分解出第二个子信道 u_2 ，从而完成了 $y^N \rightarrow u^N$ ， $N=2$ 的信道分解过程。信道分离过程也是容量守恒的。

Page 276



信道分离示意图

Page 277

信道极化（编码时）：

信道极化是指随着Polar码母码长度 $N=2^n$ 的增大，子信道（Polar码输入比特序号）的信道容量会越来越明显的趋向两个极端：无噪信道（信道容量为1）和纯噪信道（信道容量为0）。

信道极化（编码时）：

假设使用BEC信道，且删除概率为 $P = 1/2$ ，那么随着Polar码母码长增大，子信道趋近于两个极端，当母码长度 $N \geq 512$ 时，极化现象体现较为充分（大于80%的子信道得到极化），利用这个极化现象，可以选择高容量子信道去传输数据，而把其他子信道设置为已知比特（冻结比特，如0）。

Page 278

若使用以2为底的对数，则 $0 \leq C(W) \leq 1$ 。当 $C(W)=1$ 时，称 W 为无噪信道（极好）；当 $C(W)=0$ 时，称 W 为纯噪信道（无用）。信道极化的目的就是大多数常规信道转化为这两种极端情况的信道，上述信道容量计算复杂，可采用巴氏参数的最大后验概率（MAP）判决错误概率的上界，进而进行信道容量的范围考察。巴氏参数定义为 $Z(W)$ ：

$$Z(W) = \sum_{y \in Y} \sqrt{W(y|0)W(y|1)}$$

主要通过信道容量分析极化码。

$$C(W) \geq \log_2 \frac{2}{1+Z(W)}$$

$$C(W) \leq \sqrt{1-Z(W)} \cdot Z(W)$$

Page 279

因此，针对二进制删除信道（BEC），信道容量 $I(w_N^{(i)})$ 数据可以通过递归关系计算出来。

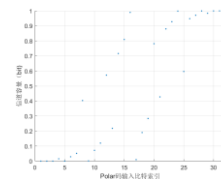
$$I(w_N^{(2^{i-1})}) = I(w_{N/2}^{(i)})^2$$

$$I(w_N^{(2^i)}) = 2I(w_{N/2}^{(2^i)}) - I(w_{N/2}^{(i)})^2$$

其中 $I(w_1^{(1)}) = 1 - \epsilon$ ，删除概率 $\epsilon = 0.5$ 。

Page 280

信道极化的统计图显示了 $I(w^{(i)})$ 对于小的指数 i 趋向接近于“0”，对于大的指数 i 趋向接近于“1”。然而，对于中间范围的指数 i ， $I(w_N^{(i)})$ 显示了不规则的特性。图中的所有点是呈现中心对称的，这也是极化码的信道极化现象所具有的特性。



Polar码信道容量 (N=32)

码长 $N=2^5$ 的信道极化现象

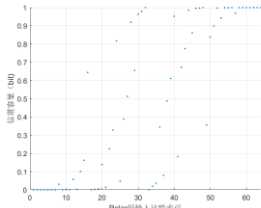
Page 281



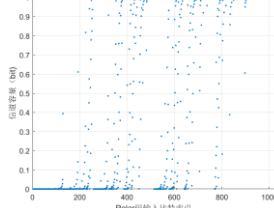
极化码基本原理

信道极化的统计图

随着子信道越来越多，信道容量就会越来越趋近于两端。



Polar码信道容量 (N=64)
码长N=2^n的信道极化现象



码长N=2^10的信道极化现象

Page 282



Polar码的构造

Polar码构造是指为Polar码能够良好工作设计的框架，其中包括编码矩阵生成、序列、速率匹配、交织



Polar码构造

Page 283



Polar码的构造—编码矩阵生成

Polar码的编码矩阵是指在进行极化编码时用来计算编码字的矩阵（ $C=U \cdot G$ 中的G）编码矩阵通常是事先存储好而不需要动态改变的。一个母码长度为 $N = 2^n$ 的极化码，其编码矩阵可由：

$$\text{Arikan核} F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

计算出来 $G_N = F^{\otimes n}$ ，其中 $\otimes$ 表示张量积。

$$\text{例如：} G_2 = F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, G_4 = F^{\otimes 2} = F \otimes F = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Page 284



Polar码的构造—码序列

Polar码序列也称为“好信道选择”。用于指示Polar码编码前的比特选择顺序。Polar码可以按照信道的可靠度从低到高排列（默认用3GPP方式），也可以按照可靠度从高到低排列。

通常使用行权重（RW）序列，对于生成矩阵G的每一行，计算出所有元素的总和得到各行的行权重，然后对行权重由低到高排序（越大可靠度越高），则各行权重对应序号为所述RW序列。

Page 285



Polar码的构造—码序列

例如，对于 G_8 Polar码，其生成矩阵为：

$$G_8 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

各行的权重分别为1、2、2、4、2、4、4、8那么 G_8 的Polar码的（RW）序列为{0, 1, 2, 4, 3, 5, 6, 7}。如果有两行权重相同，则行号大的排在后面（也可排在前面）

Page 286



Polar码的构造—码序列特性

1、在线计算

不用存储数据或只需存储少量数据即可，但是需要消耗计算量，若计算量较大会产生时延。

2、嵌套性

在母码长度为N的序列中，删除大于或等于N/2的元素而保持剩下的元素顺序，得到母码长度为N/2的序列，则称母码长度为N/2的序列是嵌套在母码长度为N的序列中的。

嵌套的好处在于不需要重新计算，不需要重新排序，只需一个查找表。

3、对称性

在母码长度为N的序列中，前后N/2是元素关于母码长度对称，对称性可以减少存储量。

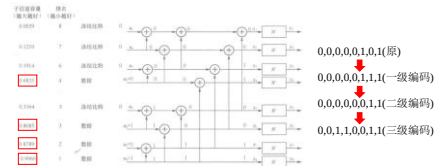
Page 287

Polar码编码需选择合适的承载数据信道和放置其他信道（冻结比特）的子信道，然后进行逻辑运算的过程。通常选用承载数据的信道应选用信道容量高或可靠度高的信道。

——根据行权重进行选取（**码序列选择**）

Page 288

例1：选用N=8、R=1/2的编码，则应选用 u_4, u_6, u_7, u_8 （0,1,0,1）来进行编码。由于要发射的比特数不一定刚好是编码之后的比特数（如要发射12个比特），因此，在编码结束后，还要进行比特选择过程（后续讲解）。



Page 289

例2：（计算）设码长N=8，消息比特数k=4。（部分教材极化码用c表示，也可用x表示）

需要的公式：

$$\begin{aligned} c_1^N &= u_1^N G_N \\ G_N &= B_N F^{\otimes n} \\ F^{\otimes n} &= F \otimes F^{\otimes (n-1)} \\ F &= \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \\ B_N &= R_N (I_2 \otimes B_{N/2}) \\ B_2 &= I_2 \end{aligned}$$

Page 290

1、假设得到的码序列选择出的四个子信道序号为“4,6,7,8”记作

$$A = \{4, 6, 7, 8\}$$

固定比特序号集合为

$$A^c = \{1, 2, 3, 5\}$$

$$\text{得到 } u_A^8 = [0, 0, 0, 1, 0, 1, 1, 1]$$

2、构造矩阵，根据公式 $B_N = R_N (I_2 \otimes B_{N/2})$ 计算 B_N

首先 $B_2 = I_2$

$$B_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$I_2 \otimes B_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Page 291

R_4 由 I_4 变换得来，先排 I_4 的奇数列，再排 I_4 的偶数列：

$$\begin{aligned} R_4 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \xrightarrow{I_4} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ B_4 &= R_4 (I_2 \otimes B_2) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ I_2 \otimes B_4 &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

Page 292

R_8 由 I_8 变换得来，先排 I_8 的奇数列，再排 I_8 的偶数列：

$$\begin{aligned} R_8 &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \xrightarrow{I_8} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \\ B_8 &= R_8 (I_2 \otimes B_4) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

Page 293

3、求F的n次克罗内积 $F^{\otimes n} = F \otimes F \otimes \dots \otimes F$ (n-1)

$$F^{\otimes 1} = F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

$$F^{\otimes 2} = F \otimes F^{\otimes 1} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} F^{\otimes 1} & 0 \\ 0 & F^{\otimes 1} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$F^{\otimes 3} = F \otimes F^{\otimes 2} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \otimes \begin{bmatrix} F^{\otimes 2} & 0 \\ 0 & F^{\otimes 2} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Page 294

4、求生成矩阵 $G_N = B_N F^{\otimes n}$

$$G_8 = B_8 F^{\otimes 3} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Page 295

5、生成极化码 $c_1^N = u_1^N G_N$

$$c_1^8 = u_1^8 G_8 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Page 296

速率匹配是把Polar码编码后的比特数量 $N = 2^n$ 通过调整（增加或减少比特数量），适配到对应的物理资源（所承载的比特数量）上。

速率匹配包括两个操作，一是**交织**，二是按照一定方式**选取比特**。

速率匹配需要几个参数：

K: （分段后）编码前的信息比特长度。

M: 对应资源所承载的比特数。

$R_{\min}$: 1/8（最小码率）

$N_{\max}$: 最大码母长度。下行方向为512，上行方向是1024。

$N_{\min}$: 最小码母长度。固定为32。

N_{dm} : 大于或等于M并且是2的幂的数。

N_m : 另一个候选的码母长度。若 $M \leq (9/8) \times N_{dm}/2$ 且 $K/M < (9/16)$

则 $N_m = N_{dm}/2$; 否则, $N_m = N_{dm}$ 。

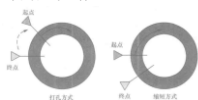
Page 297

速率匹配（选取比特）过程：

第一步：将编码之后的比特写入循环缓冲区。 **第四步**：若 $M < N$ ，并且码率 $K/M > (7/16)$ ，使用缩短

第二步：若 $M = N$ ，则没有操作。 方式。即从第一个比特开始选取，直到第M个比特。

第三步：若 $M < N$ ，并且码率 $K/M \leq (7/16)$ ，使 **第五步**：若 $M > N$ ，则使用重复方式。除去要取出N比特之外，还需要取第一个到第 $\text{mod}[(M-N), N]$ 个比特（对M-N取N的模）。

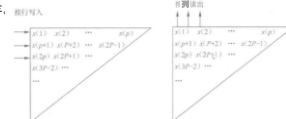


Page 298

交织是指将信息块中的比特顺序打乱，使前后比特之间尽可能减少关联性。有助于对抗突发干扰。本部分讲述后交织（交织规模较大），对时延影响较大。

一、等腰直角三角形交织

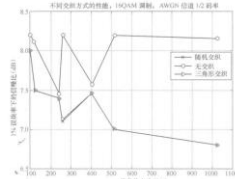
设直角边长P（3GPP规定交织器最大8192bit，即 $P \times P/2 \leq 8192$, $P \leq 128$ ），M为速率匹配后的比特数，边长P满足 $P \times (P+1)/2 \geq M$ 。需按行写入，按列读出。如果 $P \times (P+1)/2 > M$ ，需要在最后添加哑元（NULL），读出时碰到NULL则丢弃。



Page 299

等腰三角形交织器重要性质：

- 交织之前连续排列的数据，在交织之后它们的距离变成P-1,P-2,P-3...它们之间的距离不再相等。
- 每一列的长度不同，从而使得每列的转置模式都不同。

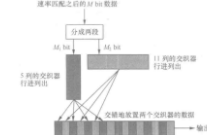


等腰三角形交织器性能

Page 300

二、双矩形交织

如图所示双矩形交织（并行矩形交织）中，首先将待交织的M bit数据分成两个部分： $M_1 = 1 + \text{ceil}(M/2)$ bit和 $M_2 = M - M_1$ bit。其次，把 M_1 bit按行写入深度为5的矩形交织器；把 M_2 bit按行写入深度为11的矩形交织器；必要时，填入NULL。然后，从这两个矩形交织器中按列读出数据。如果有NULL，则丢弃它。最后将这些数据交叠地写入一个M bit的交织器。



双矩形交织器示意

Page 301

Polar码解码方式与编码过程一样，即对接收到的信息y估计成c，然后对c进行Polar码编码即可得到原始的原始的发送信息u。

由于编码过程： $x = u \cdot G$
 等式两边同时右乘 G^{-1} 得到： $u = x \cdot G^{-1}$ （ G^{-1} 是G的逆矩阵）
 由于 $G^{-1} = (F^{\otimes n})^{-1} = (F^{-1})^{\otimes n} = G$ ，
 所以， $u = x \cdot G^{-1} = x \cdot G$ 。
 其中F为 $F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$ 。

Page 302

Polar码解码经常使用串行消去（SC）解码方法。

串行消去：

- ① 将比特索引号大的一半（N/2个数据）当做随机噪声，根据接收到的数据去解码比索引号小的一半（N/2个数据）；
- ② 把解出来的比特索引号小的一半（N/2个数据）当做已知比特，根据接收到的数据去解码比索引号大的一半（N/2个数据）。

Page 303

举例：假设从比特到符号的BPSK映射规则为 $y=2x-1$ （比特“0”映射为“-1”，比特为“1”映射为“+1”），还假设接收到的数据为 y_1, y_2, y_3, y_4 。那么，对这些接收到的数据转换成对数似然比（LLR）

$$x = LLR(y) = \ln \frac{W(y|0)}{W(y|1)}$$

其中， $W(y|0)$ 表示发射端发射的是比特“0”，而接收端收到的是y的概率， $W(y|1)$ 表示发射端发射的是比特“1”， $\ln(*)$ 表示自然对数的底e为底的对数。

Page 304

举例：在现实计算时， $W(y|1)$ 可用下式计算：

$$W(y|1) = \frac{1}{1 + e^{-2y/\delta^2}}$$

其中， δ^2 为噪声方差（可通过对接收到的数据y统计得到，一半设置 $\delta^2=1$ ）

对于 $W(y|0)$ ，可用 $W(y|0)=1-W(y|1)$ 计算。

Page 305



解码步骤

N=4解码步骤示意:

第一步：解码器把来自下半部分 (x_3, x_4) 的信息当做噪声，根据上半部分 (x_1, x_2) 计算出临时变量 t_1, t_2 。在这里，“当做噪声”的含义是从编码（或数据发射）角度看，有

$$x_1 = t_1 + n_1$$

$$x_2 = t_2 + n_2$$

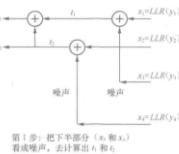
其中 t_1, t_2 为有用数据， n_1, n_2 为噪声，因此在解码时为消除噪声影响需要

$$t_1 = x_1 - n_1$$

$$t_2 = x_2 - n_2$$

这里的“-”不是简单的减法，二是/运算（盒子加，Box-plus），如下。

$$t_1 = x_1 - n_1 = f(x_1, n_1) = x_1 \boxplus n_1 = \ln \frac{1 + e^{x_1} \cdot e^{n_1}}{e^{x_1} + e^{n_1}}$$



第1步：把下半部分 (x_3 和 x_4) 看成噪声，去计算出 t_1 和 t_2

Page 306



解码步骤

第二步：解码器把下半部分（临时变量 t_2 ）看成噪声，求解出 u_1 。



第2步：把下半部分 (t_2) 看成噪声，去解出 u_1

第三步：把上半部分 (u_1) 当做已知比特，去解出 u_2 。此时应根据 u_1 的取值 (0,1) 来对临时变量进行 t_1, t_2 的操作，然后再译出 u_2 。



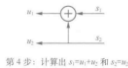
第3步：把上半部分 (u_1) 当作已知比特，去解出 u_2

Page 307



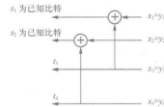
解码步骤

第四步：计算出另一组临时变量 $s_1 = u_1 + u_2$ （这里指模2加“ $\boxplus$ ”）和 $s_1 = u_2$



第4步：计算出 $s_1 = u_1 \boxplus u_2$ 和 $s_2 = u_2$

第五步：把上半部分（临时变量 s_1 和 s_2 ）当做已知比特，去计算又一组临时变量 t_3, t_4



第5步：把上半部分 (s_1 和 s_2) 当作已知比特，去计算 t_3 和 t_4

Page 308



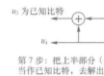
解码步骤

第六步：与第二步类似，把下半部分（临时变量 t_2 ）看成噪声，求解出 u_3 。



第6步：把下半部分 (t_2) 看成噪声，去解出 u_3

第七步：与第三步类似，把上半部分 (u_3) 当做已知比特，去解出 u_4 。



第7步：把上半部分 (u_3) 当作已知比特，去解出 u_4

Page 309



解码示例

例3：假设接收到数据为 $y_1 = -1.61$ 、 $y_2 = -0.06$ 、 $y_3 = -1.89$ 、 $y_4 = 1.69$ ，因为 $x = \text{LLR}(y)$ ，所以 $x_1 = 3.22$ 、 $x_2 = 0.12$ 、 $x_3 = 3.78$ 、 $x_4 = -3.38$ 。计算解码结果。

第一步：将 x_3, x_4 当做噪声，根据上半部分 x_1, x_2 计算临时变量 t_1, t_2 。

$$t_1 = f(x_1, x_3) = x_1 \boxplus x_3 = 2.77$$

$$t_2 = f(x_2, x_4) = x_2 \boxplus x_4 = -0.11$$

Page 310



解码示例

第二步：解码器把下半部分（临时变量 t_2 ）看成噪声，求解出 u_1 。那么，

$$z_1 = f(t_1, t_2) = t_1 \boxplus t_2 = -0.09$$

对 z_1 进行判决（若是冻结比特，则无需判决，直接输出冻结比特值），得到 $u_1 = 1$ （这一比特解码错误；从仿真程序看，SNR=0dB，下同；考虑码长 N=4 不够大，极化现象不明显，且码率 R=1，有错误正常）

$$\text{判别条件: } u_i = \begin{cases} 0, & \text{如果 } \text{LLR}_i \geq 0 \\ 1, & \text{其他} \end{cases}$$

Page 311

解码示例

第三步: 把上半部分 (u_1) 当做已知比特, 去解出 u_2 。此时应根据 u_1 的取值 (0,1) 来对临时变量进行 t_1, t_2 的操作, 然后再译出 u_2 。

根据编码规则, 如果 $u_1=0$, 则编码之后有 $t_1 = t_2$, 那么对 u_2 进行解码时, $z_2 = t_1 + t_2$, 对 z_2 进行判决得到 u_2 ;

若 $u_1=1$ ，则在编码之后有 $t_1 = 1 - t_2$ （经BPSK后 $t_1+t_2=0$ ），那么对 u_2 解码时，有 $z_2 = t_2 - t_1$ ，对 z_2 进行判决，得到 $u_2=1$ （解码正确）

Page 312



解码示例

第四步：计算出另一组临时变量 $s_1 = u_1 \oplus u_2$ 和 $s_2 = u_2$ 。根据上面第2~3步结果得到 $s_1=0, s_2=1$ 。

第五步:把上半部分(临时变量 s_1 和 s_2)当做已知比特,去计算又一组临时变量 t_3, t_4 。类似于第三步,有

$$t_3 = x_3 + (1-2 \times s_1) \cdot x_1 = 3.78 + (1-2 \times 0) \times 3.22 = 7.00$$

$$t_4 = x_4 + (1-2 \times s_2) \cdot x_2 = -3.38 + (1-2 \times 1) \times 0.12 = -3.50$$

Page 313



解码示例

第六步：与第二步类似，把下半部分（临时变量 t_2 ）看成噪声，求解出 u_3 。

$$z_3 = f(t_3, t_4) = t_3 \boxplus t_4 = -3.47$$

对 z_3 进行判决, 得到 $u_3=1$ (解码错误)

第七步：与第三步类似，把上半部分 (u_3) 当做已知比特，去解出 u_4 。

$$z_4 = t_4 + (1-2 \times u_3) \times t_3 = -3.50 + (1-2 \times 1) \times 7.00 = -10.50$$

对 z_4 进行判决, 得到 $u_4=1$ (解码正确)

Page 314



解码示例

小结:

上述解码方法是串行进行的(先解 u_1 , 然后解 u_2 , 再解 u_3 , 最后解 u_4)并且和解码后面的比特(如 u_2)时, 充分考虑了前面比特(u_1)的影响, 从而逐步消除前面比特的影响, 故称之为串行消除算法。

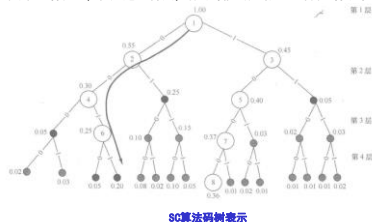
当母码长度更大的时候，需要执行多次上面这些步骤。

Page 315



解码算法

普通SC算法解码方式主要通过后验概率进行判决，总是通过最大后验概率的路径进行分析，但通过SC算法，有时可能会走错。（码树进行表示）



Page 314



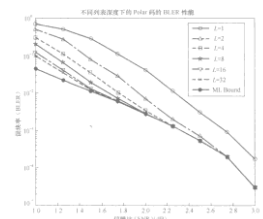
解码算法

第二种方式:SC-L算法

SC-L算法是在SC算法的基础上,通过保存L条具有最大后验概率的路径来进行下一层的解码,算法在最后选取一条具有最大后验概率的路径。即相对SC算法,SC-L算法增加了路径保存和回退功能。

当列表深度 $L=1$ 时, SC-L算法等同于SC算法;

当列表 $L = \min(2^N, 2^{\text{Length}(A)})$ 时, SC-L 算法就变成了 ML 算法。Length(A) 为信息集的长度。当列表长度 $L=32$ 时, SC-L 算法已经与 ML 非常接近了。



Page 317

北京邮电大学
BEIDING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第九章 纠删码

李丽香, 彭海朋
北京邮电大学网络空间安全学院
网络与交换技术国家重点实验室

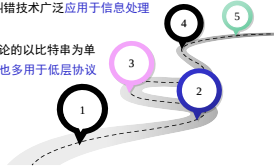


Wikipedia: In coding theory, an erasure code is **forward error correction** (FEC) code under the assumption of bit erasures (rather than bit errors), which transforms a message of k symbols into a longer message (code word) with n symbols such that the original message can be recovered from a subset of the n symbols.



在编码理论中, 有一种前向纠错技术(FEC)也称为纠删码。
这种技术可以将原始数据中丢失的 k 字节数据从 n 个含编码字节的
信息中恢复。

Page 305

- 01 在香农定理的指导下, 信道编码理论和技术逐步发展成熟。
 - 02 纠删码(Erasure Code)原来是无线通信中有噪信道编码的一种。
 - 03 Kamin提出的密钥共享(Secret Sharing)以及M.O.Rabin提出的IDA (Information Dispersal Algorithm)算法, 将其引入到计算机应用中。
 - 04 近年来, 前向纠错技术广泛应用于信息处理的各个领域。
 - 05 各种纠错码所讨论的以比特串为单位的错误检测和纠错也多用于低层协议(如链路层)。
- 

Page 306

- 1 数据的存储呈现出集中化(以分布式存储系统为基础的云存储系统)和移动化(互联网移动终端)的趋势, 数据可靠性愈发引起重视。
- 2 集群所承载的数据量大大上升, 但存储介质本身的可靠性进步却很小, 这要求必须以更加经济有效的方式来保障数据安全。
- 3 副本技术通过增加冗余数据的方式来保证数据在发生部分丢失时, 原始数据不发生丢失。但相较于副本, 纠删码能以低得多的存储空间代价获得相似的可靠性。

Page 307

要想知道 x_1, x_2, x_3 的值, 只需要任意3个方程式即可解出。

- (1) $x_1 = 1$
- (2) $x_2 = 2$
- (3) $x_3 = 3$
- (4) $x_1 + x_2 + x_3 = 6$
- (5) $x_1 + 2x_2 + 4x_3 = 17$
- (6) $x_1 + 3x_2 + 9x_3 = 34$

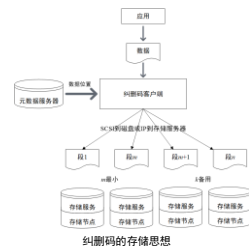
副本策略: 若方程(1)(2)(3)丢失任何一个方程数据都能恢复, 则需要把(1)(2)(3)三个方程都存储两份, 也就是存储6个方程。

纠删码策略: 把方程(1)(2)(3)看做是分布式系统的数据, (4)(5)(6)看做是code, 只要一个code, 即使丢了(1)(2)(3)中的任何一个方程, 数据都可以恢复, 故只需要存储4个方程。

策略存储效率对比: 纠删码策略存储4个方程, 副本策略存储6个方程, 存储效率是4/6, 可以节省1/3的空间。实际根据code的多少, 存储效率会不一样。

Page 308

纠删码是一组数据冗余和恢复算法的统称, 是一种数据保护方法。



纠删码技术将数据分割成片段, 把冗余数据块扩展、编码, 并将其存储在不同的位置, 比如磁盘、存储节点或者其它地理位置。

纠删码技术具有良好的容错性和安全性, 主要应用于网络传输中丢包错误的解决。

Page 309

网络传输方面：纠删码的基本思想是“当发送方将n个源数据包编码，得到m个校验数据包，将这n+m个数据包在网络上进行传输。接受方只要接收到不少于n个编码包，就可以将源数据恢复。一般来说，这个传输过程允许最多m个数据包的丢失。”



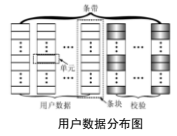
数据存储方面：纠删码的基本思想是“当冗余级别为n+m时，将这些数据块分别存放在n+m个硬盘上，这样就能容忍m个(假设初始数据有n个)硬盘发生故障。当不超过m个硬盘发生故障时，只需任意选取n个正常的数据块就能计算得到所有的原始数据。”



单元(Element)是用户数据或校验的一个基本单位，可以是一个比特(Bit)、一个字节(Byte)、一个扇区(Sector)或一个更大的磁盘数据块。它是纠删码的构造块(Building Block)。在编码理论中，它对应于码符号(Code Symbol)中的一个比特。

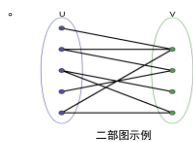
条带(SStripe)是一个由用户数据单元和校验单元组成的具有独立编码关系的集合。在编码理论中，它对应于一个码字(Code Word)。这里，条带长度(Stripe Length)定义为一个条带所跨越的磁盘个数，记为n。

条块(strip)是一个条带中存储在同一个磁盘上的所有单元组成的集合。在编码理论中，它对应于码字中的一个码符号。这里，条块宽度(strip Width)定义为一个条块中包含的单元个数，记为w。



Page 331

二部图又称为二分图，是图论中的一种特殊模型。设 $G=(V,E)$ 是一个无向图，如果顶点V可以分割为两个互不相交的子集(U,V)，并且图中每条边(i,j)所关联的两个顶点i和j分别属于这两个不同的顶点集(i in U, j in V)，则称图G是一个二部图。



二部图示例

范德蒙矩阵是法国数学家范德蒙提出的一种各列为几何级数的矩阵。

该矩阵第i行、第j列可以表示为 g_{ij}^{j-1} 。

$$G = \begin{bmatrix} g_0^0 & g_1^0 & g_2^0 & \dots & g_{n-1}^0 \\ g_0^1 & g_1^1 & g_2^1 & \dots & g_{n-1}^1 \\ g_0^2 & g_1^2 & g_2^2 & \dots & g_{n-1}^2 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ g_0^{n-1} & g_1^{n-1} & g_2^{n-1} & \dots & g_{n-1}^{n-1} \end{bmatrix} = \begin{bmatrix} 1 & g_1 & g_1^2 & \dots & g_1^{n-1} \\ 1 & g_2 & g_2^2 & \dots & g_2^{n-1} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & g_{n-1} & g_{n-1}^2 & \dots & g_{n-1}^{n-1} \end{bmatrix}$$

柯西矩阵中 $\{x_1, x_2, \dots, x_m\}$ 和 $\{y_1, y_2, \dots, y_m\}$ 为同一个有限域的两个不同子集。它们必须满足：

- (1) 不同子集中的任意两个元素相加不能为0。
 - (2) 这两个子集中的每个元素都不相同。
- 这两个条件都满足才能构成柯西矩阵。

Page 332

迦罗华域，用 $GF(2^n)$ 来表示，代表含有 2^n 个元素的有限域。迦罗华域中的四则运算(即基于多项式的运算)，多项式一般为 $f(x) = x^6 + x^4 + x^2 + x + 1$ 。

(1) 多项式加法

将两个多项式中相同指数的项系数相加或相减。例如 $(x^2 + x) + (x + 1) = x^2 + 2x + 1$

(2) 多项式乘法

将其中一个多项式的各项分别与另一个多项式的各项相乘，然后把相同指数的项的系数相加。例如 $(x^2 + x)(x + 1) = x^3 + x^2 + x^2 + x = x^3 + 2x^2 + x$

(3) 多项式除法

使用长除法。例如计算 $x^3 - 12x^2 - 42$ ，除以 $x^2 - 9x + 27$ ，商 $x - 27$ ，余数 -123 。

(4) $GF(2^n)$ 上的多项式运算

对于 $GF(2^n)$ 上的多项式计算，多项式系数只能取0或1。(如果是 $GF(3^n)$ ，那么系数可以取0、1、2)

$GF(2^n)$ 的多项式加法中，合并阶数相同的同类项时，由于 $0+0=0$ ， $1+1=0$ ， $0+1=1+0=1$ ，因此系数不是进行加法操作，而是进行异或操作。

$GF(2^n)$ 的多项式减法等于加法，例如 $x^4 - x^4$ 就等于 $x^4 + x^4$ 。

Page 333

编码方式：

将原数据重复k次后，就得到k-重复码。例如，将原数据x重复3次，拼接得到数据 $\{x, x, x\}$ ，即为原数据的3-重复码。发送方传输的就是k-重复码。

解码方式：

将接收到的数据分为k段，k为原数据的重复次数。每一段即是一个原数据，这样就完成了数据恢复。将这k段数据比较，可保证传输数据的可靠性。

Page 334

编码方式：

奇偶校验码是一种通过增加冗余位，使得码字中“1”的个数恒为奇数或者偶数的编码方法。

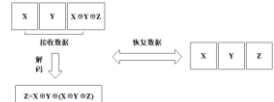
将原数据分为k个等长的部分，将这k个部分进行异或操作，与数据段尾部的冗余比较，以保证数据传输的可靠性。



奇偶校验码编码示例

解码方式：

冗余检测方式，接收方根据接收到的数据和冗余恢复出丢失的数据。也可以对接收到的数据进行异或操作，与数据段尾部的冗余比较，以保证数据传输的可靠性。



奇偶校验码解码示例

Page 335

重复码可容忍擦除率很高，但编码率低，效率低下，造成浪费。

奇偶校验码的编码率高，但可容忍擦除数少，容错性差。

二者的具体区别如表所示。其中n为原数据个数，m为冗余数。

	重复码	奇偶校验码
可容忍擦除数	最多 $(k-1)$ 倍于原数据包	只能容忍1个数据包
编码率	低，只有 $1/n$	高达 $n'/(m+n)$

Page 336

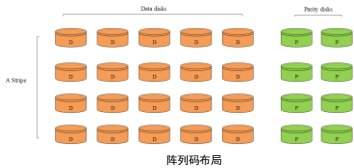
目前，纠错码技术在分布式存储系统中的应用主要有四类。



Page 337

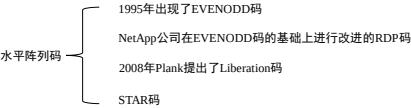
阵列码(Array Code)，即奇偶校验阵列码(Parity Array Code)的简称。它将一个条带中的用户数据单元和校验单元排列成一个二维或多维阵列，并基于XOR运算对条带内的单元进行编码。阵列码的提出主要是为了满足高容错磁盘阵列设计的需要。

阵列码分为水平阵列码、垂直阵列码和水平垂直阵列码。

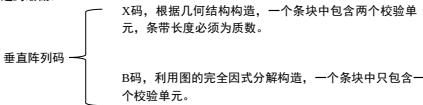


Page 338

水平阵列码的一个条带中的用户数据单元和校验单元分别存放在不同的条带中。



垂直阵列码的一个条带中的每个条块既包括用户数据单元，又包括校验单元。垂直阵列码的出现是为了使更新复杂度达到最低。



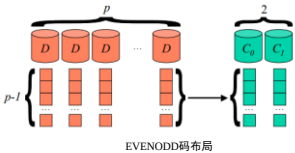
Page 339

水平垂直阵列码的一个条带中一些条块只包括校验单元，另一些条块同时包括用户数据单元和校验单元。

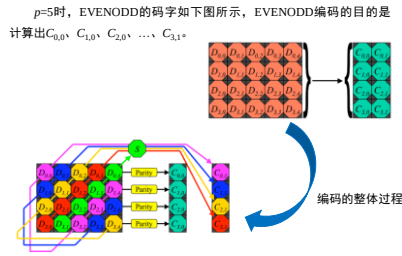


Page 340

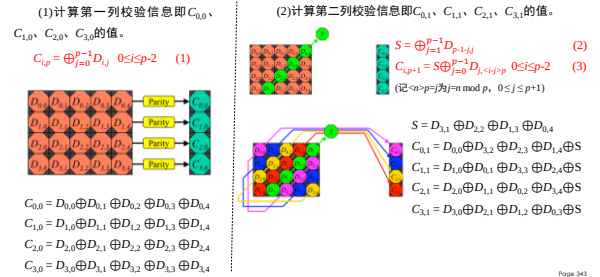
在阵列码中，EVENODD码是首个可以容忍两列丢失的编码方法，被公认为阵列码的始祖。在EVENODD码中，一共有p列数据列和2列冗余校验列，每一列有p-1个信息位，因此其码字为 $(p-1)*(p+2)$ 的二维阵列，其中p为大于2的素数。



Page 341



Page 342



Page 343

在这里介绍的是EVENODD双列删错的译码算法, 在译码过程中只需要简单的异或操作。
假定数据块 i 和 j 损坏, $0 \leq i < j \leq p-1$, 假设数据块的排列从左到右是 $0 \sim p-1$, 其中 $0 \sim p-1$ 列为用户数据块, $p, p+1$ 列为校验块。

(1) $i=p, j=p+1$, 两个校验数据块损坏, 数据恢复方式是重新编码。

(2) $i < p, j = p+1$, 一个用户数据块和一个校验块损坏, 恢复用户数据块 i 是通过参数 S 进行恢复, 即

$$S = D_{i,i+1 \sim p,p+1} \bigoplus (\bigoplus_{j=0}^{p-1} D_{i,j+1 \sim p,j}) \quad (4)$$

假定 $D_{p-1,j}=0, 0 \leq j \leq p$, 然后利用下式计算 i 数据块的信息:

$$D_{i,i} = S \bigoplus D_{i,i+1 \sim p,p+1} \bigoplus (\bigoplus_{j=0, j \neq i}^{p-1} D_{i,j+1 \sim p,j}), 0 \leq i \leq p-2 \quad (5)$$

Page 344

(3) $i < p, j = p$, 一个用户数据块和一个校验块损坏, 用户数据块 i 可以根据平行校验数据块进行异或得到恢复, 即

$$D_{i,i} = \bigoplus_{j=0, j \neq i}^p D_{i,j}, 0 \leq i \leq p-2 \quad (6)$$

(4) $i < p, j < p$, 两个用户数据块损坏, 数据恢复方法如下:

① 首先将最后两列校验数据块的信息异或, 恢复参数 S

$$S = (\bigoplus_{j=0}^{p-2} D_{i,j}) \bigoplus (\bigoplus_{j=0}^{p-2} D_{i,j+1}) \quad (7)$$

② 然后计算 $S^{(0)} = S_0^{(0)}, S_1^{(0)}, \dots, S_{p-1}^{(0)}, S^{(1)} = S_0^{(1)}, S_1^{(1)}, \dots, S_{p-1}^{(1)}$, 其中

$$S_0^{(0)} = \bigoplus_{j=0, j \neq i, j}^{p-1} D_{0,j} \quad (8)$$

$$S_0^{(1)} = S \bigoplus D_{0,p-1} \bigoplus (\bigoplus_{j=0, j \neq i, j}^{p-1} D_{i,j+1 \sim p,j}) \quad (9)$$

③ 然后按照下面的顺序恢复 i, j 数据块的各个信息:

Page 345

- 1 Set $s \leftarrow \langle (j-i)-1 \rangle p$ and $a_{m-1,j} \leftarrow 0$ for $0 \leq l \leq p-1$
- 2 Let $a_{i,j} \leftarrow S_{\langle j+1 \rangle p}^{(1)} \bigoplus a_{\langle s+(j-i) \rangle p, j}$ and $a_{i,i} \leftarrow S_i^{(0)} \bigoplus a_{i,j}$
- 3 Set $s \leftarrow \langle s-(j-i)-1 \rangle p$ if $s=p-1$ then stop, else goto step 2

利用译码算法恢复第0和第2个错误数据块的信息:



初始数据

?	0	?	1	0	1	1
?	1	?	0	0	0	1
?	1	?	0	0	1	1
?	1	?	1	1	0	0

Page 346

初始数据

?	0	?	1	0	1	1
?	1	?	0	0	0	1
?	1	?	0	0	1	1
?	1	?	1	1	0	0

第一步: 找到公共因子 S , 将这个数组的最后两列相异或, 得到 $S=1$ 。

这意味着, 对角校验是奇校验。根据公式(6)得到校验数组:

$$S^{(0)} = 01010, S^{(1)} = 01010$$

Page 347

第二步：利用递归恢复丢失的数据 $a_{i,j}$, $a_{i,2}$, $0 \leq i \leq 3$ 。

由于 $i = 0, j = 2$, 故 $s = \langle -(j-i)-1 \rangle p = \langle -3 \rangle 5 = 2$, 后续计算过程如表所示。

1	2	3	4
$S \leftarrow 2$	$a_{2,2} \leftarrow SD_2 \oplus 0$	$a_{2,2} \leftarrow SH_2 \oplus a_{2,2} = 0$	$S \leftarrow 0$
$S \leftarrow 0$	$a_{0,2} \leftarrow SD_2 \oplus a_{2,0} = 0$	$a_{0,2} \leftarrow SH_2 \oplus a_{0,2} = 0$	$S \leftarrow 3$
$S \leftarrow 3$	$a_{1,2} \leftarrow SD_2 \oplus a_{0,0} = 0$	$a_{1,0} \leftarrow SH_2 \oplus a_{1,2} = 1$	$S \leftarrow 1$
$S \leftarrow 1$	$a_{1,2} \leftarrow SD_2 \oplus a_{1,0} = 0$	$a_{1,0} \leftarrow SH_2 \oplus a_{1,2} = 1$	$S \leftarrow 4$

由于 $S=4=p-1$,故算法停止。

Page 348

定理一：EVENODD译码算法可以恢复任意2个数据块出错的数据。

证明：如果只有一列出错，我们可以利用剩余的正确校验位数据相异或进行恢复。根据异或的性质可以很容易知道这是完全可行的。

如果有两列出错，假设第 i 和 j 列出错， $0 \leq i < j \leq p-1$ 。我们假设译码算法的四种情况：

- (1) $i=p$, $j=p+1$, 这种情形和编码过程完全一样，显然此算法可行；
- (2) $i < p$, $j=p+1$, 注意公共因子 S 的校验值是由公式(4)得到的，它由对角线位 $\langle -i-1 \rangle p, 0, \langle -i-2 \rangle p, 1, \dots, \langle i, p-1 \rangle$ 上的数值相异或得到。这是一条唯一不与第 j 列相交的对角线(此时第 j 列无效)。因此所有的对角线都有相同的公共因子 S 。

Page 349

经过分析公式(3)得到 $a_{i,j}$ 的过程，我们可以推出能够找到恢复第 i 列数值的公式(5)，一旦第 i 列的值被恢复，这个算法就可以利用公式(1)得到第 p 列的值。

(3) $i < p$, $j=p$, 这种情形和编码的过程相似，在这里不再赘述。

(4) $i < j < p$, 第一步通过公式(7)计算出共同因子 S ，也就是公式(3)和公式(7)是等价的，但是公式(3)中存在未知的变量 i, j 的值，因此不能使用公式(3)直接得到共同因子 S ，因此根据公式(1)和(2)可以得到下面的式子(也就是公式(7))：

$$\begin{aligned}
 & (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-2} a_{l,p+1})) = (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (\oplus_{t=0}^{p-2} (S \oplus \oplus_{l=0}^{p-1} a_{-i-l,p,j})) \\
 & = (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (p-1)S \oplus (\oplus_{t=0}^{p-2} (S \oplus \oplus_{l=0}^{p-1} a_{-i-l,p,j})) \\
 & = (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{-i-l,p,j})) \quad (10)
 \end{aligned}$$

Page 350

因为 $p-1$ 为偶数，所以 $(p-1)S=0 \bmod 2$ ，又因为最后一个虚拟的 $p-1$ 行所有的值都为0，即 $a_{p-1,j}=0$ ，因此可以得到 $(\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j}))$ 所有水平校验位的值异或，也就是所有的0到 $p-1$ 列的数据(每列是 $p-1$ 行)，所有数据位异或。 $(\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{-i-l,p,j}))$ 表示所有的垂直校验位的值异或(除掉共同因子 S)，从0到 $p-1$ (每列 p 行，但最后一行的值为0)，因此也就是所有的0到 $p-1$ 列所有数据位异或。

因此：

$$\begin{aligned}
 0 & = (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (\oplus_{t=0}^{p-1} (\oplus_{l=0}^{p-1} a_{-i-l,p,j})) \\
 & = (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{-i-l,p,j})) \oplus (\oplus_{t=1}^{p-1} a_{p-1,j})
 \end{aligned}$$

所以得到如下公式：

$$(\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{-i-l,p,j})) = \oplus_{t=1}^{p-1} a_{p-1,j} \quad (11)$$

公式(11)证明了公式(2)和公式(7)都能给出公共因子 S 的值。

Page 351

下面给出证明“这个算法只能唯一恢复第 i 列和第 j 列的数据”。

假设校验组 $S^{(0)}$ 和对角校验组 $S^{(1)}$ 已经分别由公式(8)和公式(9)获得。

第一步给 S 赋初值 $S = \langle -(j-i)-1 \rangle p$ 得到 S 。

第二步计算：

$$a_{\langle -(j-i)-1 \rangle p, j} \leftarrow S_{\langle -(j-i)-1 \rangle p}^{(1)} \quad (12)$$

该公式是正确的，通过公式(3)和公式(9)可以得出。

如果首先计算出 $a_{\langle -(j-i)-1 \rangle p, j}$ 的值，可以根据下面的公式(13)计算得到 $a_{\langle -(j-i)-1 \rangle p, j}$ ：

$$a_{\langle -(j-i)-1 \rangle p, j} \leftarrow S_{\langle -(j-i)-1 \rangle p}^{(0)} \oplus a_{\langle -(j-i)-1 \rangle p, j} \quad (13)$$

公式(13)的正确性，根据公式(1)和公式(8)可以得到。

第三步，参数 S 的值设置为 $\langle -2(j-i)-1 \rangle p$ ，返回第二步，根据算法可以得到：

$$a_{\langle -2(j-i)-1 \rangle p, j} \leftarrow S_{\langle -2(j-i)-1 \rangle p}^{(1)} \oplus a_{\langle -(j-i)-1 \rangle p, j} \quad (14)$$

$$a_{\langle -2(j-i)-1 \rangle p, j} \leftarrow S_{\langle -2(j-i)-1 \rangle p}^{(0)} \oplus a_{\langle -2(j-i)-1 \rangle p, j} \quad (15)$$

Page 352

奇偶校验码，是一种完全基于XOR运算的纠错码，他是一种条块宽度为 $w-1$ 的特殊水平码。最简单的奇偶校验码是单奇偶校验码：在一个条带中，只有唯一的一个校验条块，这个校验条块由所有用户数据条块的XOR运算之和得到。于是单奇偶校验的容错能力为1，其最典型的应用是RAID 5。

基于单奇偶校验码可以构造出容错能力更高的奇偶校验码，大致可分为两类：

- (1)根据几何结构构造的奇偶校验码，例如水平垂直奇偶校验码
- (2)根据图结构构造的奇偶校验码，例如基于Tanner图的LDPC码

Page 353

根据几何结构构造的奇偶校验码。这类奇偶校验码将一个条带中的条块在逻辑上组织成多维几何结构，然后在每个方向上采用单奇偶校验码进行编码，因此具有十分规整的结构，十分易于实现。但是，他们的存储效率通常较低。

eg: 水平垂直奇偶校验码

编码方式：他将一个条带中的条块在逻辑上组织成二维阵列结构，然后在水平和垂直方向上分别采取单奇偶校验码进行编码。

$D_{0,0}$	$D_{0,1}$	...	$D_{0,(K-1)}$	$\rightarrow$	$P_{0,K}$
$D_{1,0}$	$D_{1,1}$	...	$D_{1,(K-1)}$	$\rightarrow$	$P_{1,K}$
$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\rightarrow$	$\vdots$
$D_{(K-1),0}$	$D_{(K-1),1}$	...	$D_{(K-1),(K-1)}$	$\rightarrow$	$P_{(K-1),K}$
$\downarrow$	$\downarrow$	...	$\downarrow$	$\searrow$	$\downarrow$
$P_{0,K}$	$P_{1,K}$	...	$P_{(K-1),K}$		$P_{K,K}$

D_{ij} 表示用户数据条块

$P_{i,K}$ 表示校验条块

$$P_{i,K} = \sum_{j=0}^{K-1} D_{i,j}$$

$$P_{K,K} = \sum_{i=0}^{K-1} P_{i,K}$$

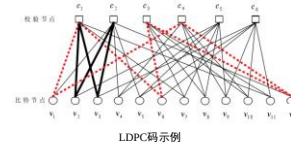
$$P_{K,K} = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} D_{i,j}$$

$$(0 \leq i \leq K-1, 0 \leq j \leq K-1)$$

Page 354

低密度奇偶校验码(Low-Density Parity-Check Code, LDPC)最初是为通信链路的容错而设计的，近年来开始基于广域网的存储应用中被研究。LDPC码的主要优点在于，构造出的高容错LDPC码可以具有接近最高的存储效率并且解码复杂度很低。

LDPC码是一种由级联的随机稀疏二部图和一个传统的纠错码构成的特殊的纠错码，级联二部图的意思就是多级二部图。



LDPC码示例

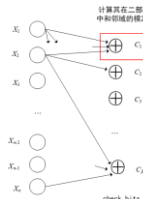
Page 355

低密度奇偶校验码的一个典型的例子是龙卷风码(Tornado Code)

编码方式：

龙卷风码通过层层递进的异或操作得到校验码。

如图所示，把原数据分成n个等长的数据块($x_1, x_2, x_3, \dots, x_n$)，这些数据被称为用户数据块(message bits)。根据多级二部图 $B_0, B_1, \dots, B_{k-1}$ 的连接关系，将相连的数据节点进行异或得到下一层校验节点(check bits)的值($C_1, C_2, \dots, C_p$)。



龙卷风码编码示例

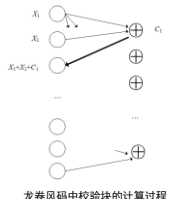
Page 356

举例来说，在右图中，某校验层各节点的值分别为 $x_1, x_2, x_3, x_4, x_5, x_6, \dots$ ，下一级第一个校验块由 $x_1 \oplus x_2$ 得到。

选择哪些变量来进行异或操作由二部图确定，一般来说，级联低密度纠错码二部图的生成具有随机性，通过使用每一级二部图的关系，就可以依次生成级联二部图的所有右节点。

在此过程中，进行异或操作的次数(或者说是运算层数)由每层中的数据块数目决定。

不妨引入一个参数 $e, e = m/(m+n)$ ，其中m是最终的校验码数目，也就是说，e是擦除概率，它与码率的和为1。显然，e满足 $e < 1$ 。那么，在校验块计算时，第一层得到的数据块数目就是 $e \times n$ ，第二层的数目是 $e^2 \times n$ ，第三层的数目是 $e^3 \times n$ ，依次递推，第k层的数据块数目就是 $e^k \times n$ 。

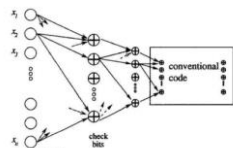


龙卷风码中校验块的计算过程

Page 357

下图说明，最后一级(也就是二部图中的 B_{k-1})的计算使用传统的编码方法(如RS码的编码方法)。最后一级的运算中， B_{k-1} 和 B_k 具有相同的变量节点，根据连接关系生成新节点。

至此，整个编码工作完成。发送方传输的编码就是原数据包加上校验块的总数数据包。



龙卷风码解码过程

Page 358

解码方式：

接收方收到 $n+m$ 个数据包，如果包含源数据的n个数据包有丢失，就可以通过m个冗余数据包进行重构。

解码过程实际上是一个在二部图上去边的过程，从最高级二部图开始：

- (1)首先，利用 B_k 和 B_{k-1} 得到它们的信息块，即 B_{k-1} 的校验块。
- (2)然后，依次得到 $B_{k-2}, \dots, B_1, B_0$ 的校验块。
- (3)最后，得到 B_0 的m个信息块。

当然，如果在解码开始时就已经得到了某一层 $B_i (0 \leq i < k)$ 的所有校验块，那么就可以直接从 B_i 开始解码。

Page 359

容错性:

如果与某一校验节点相邻的信息节点中存在一个丢失数据的信息节点,那么可以通过该校验节点和其它节点进行异或运算,得到丢失的信息数据。

数据恢复:

这里的“信息节点”与“校验节点”是相对的,在每一层异或运算中,参数节点称为“信息节点”,生成节点称为“校验节点”。具体步骤如下:

- (1)所有校验节点赋初值0,所有信息节点根据接收的信息赋值为0,1,或D (删除)。
- (2)根据边的连接关系,把所有未被删除的信息节点的值和与其相连的校验节点的值(模二)相加,并将得到的值作为该校验节点的新值,然后从二部图中去掉这些正确接收的变量节点以及与之相连的边。

(3)在得到的二部图中寻找度数为1的校验节点。如果某一校验节点度数为1,则该校验节点的值就等于唯一与它相连的信息节点的值,此信息节点所对应的信息比特得到恢复。

(4)再根据边的连接关系,把恢复出来的信息节点的值与和它相连的校验节点的值(模二)相加,作为这些校验节点的新值,并从二部图中去掉恢复出来的信息节点和与之相连的边。

(5)在去边之后的二部图中重复步骤(3)(4),直至二部图中不存在度数为1的校验节点,或所有的原数据(message bits)值均被恢复。

这样,所有能恢复的丢失数据都能被恢复。此解码过程中只有异或计算,故解码速度较其他纠错码快速。

龙卷风码的复杂度分析	Tornado code
编码时间复杂度	$O(n \ln(1/e))$
解码时间复杂度	$O(n \ln(1/e))$
编码率	$1 - p(e) \sim \exp(-1+e)$

优点:

- (1)由于只涉及异或操作,编码率高,其中 $p(e)$ (e 是擦除概率)是一个极小量,与二部图的选择有关。
- (2)编码过程提到,进行异或操作的次数,也就是二部图的边的数目,由每层中的数据块数目决定。因此该算法的复杂度为线性,编解码效率高。
- (3)由于所有的丢失数据基本能恢复,龙卷风码能提供接近最优的损失保护。

缺点:

- (1)需要擦除概率 e 作为输入,而擦除概率难以精确获得、随时可能变化、多个接受者的擦除概率不同的特点也带来了这种算法的缺点。
- (2)和RS Code一样,在数据规模较大,且在数据规模较大的情况下,编解码优势才比较明显。数据规模较小时并不能体现很大优势。

数字喷泉码是一类码率不受限制的纠错码(Rateless Erasure Code),也称为无速率编码,是一类基于图的线性纠错码。喷泉码是指可以由 k 个原始数据分组生成任意数量的编码分组,而接收方只要收到其中任意 m 个编码分组,即可通过译码以高概率成功恢复全部原始数据分组。

可以看到,上述编码过程就如同源源不断产生水滴(编码分组)的喷泉(编码器),而我们只要用杯子(译码器)接收足够数量的水滴,即可达到饮用(成功译码)的目的。正因如此,该种编码被称为喷泉码。



数字喷泉码的形象说明



John Byers 及Michael Luby 等人于1998 年首次提出了数字喷泉(Digital Fountain)的概念,它是针对大规模数据分发和可靠广播的应用特点而提出的一种理想的解决方案。但当时并未给出实用数字喷泉码设计方案。



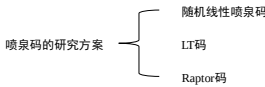
2002年, M.Luby提出了第一种现实可行的喷泉码——LT码。在学术理论日渐完善的同时,喷泉码也日益受到产业界的关注,获得了越来越多的实际应用。



目前,一种由Digital Fountain公司设计的系统Raptor码也已经 被DVB-H标准和3GPP组织的MBMS标准采用,并且正在参与其他 多项国际标准的制定。

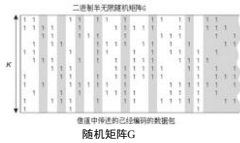
喷泉码的研究现状:

- (1)应该尽量减小译码开销 c ,使其趋于零。
- (2)应该尽量减小编译码复杂度,理想情况下,应该使生成每个编码分组需要的运算量是一个与 k 无关的常数,而成功译码 m 个编码分组获得 k 个原始数据分组需要的运算量是一个关于 k 的线性函数。

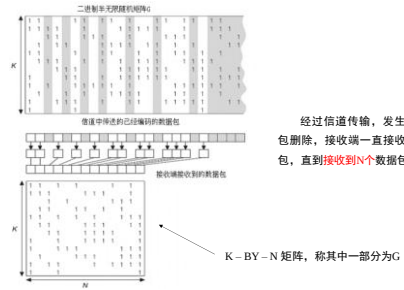


假设编码器需要编码的文件为 $S_1, S_2, \dots, S_k$ ，每个 S_i 都是一个信息符号，它只能被完整的传输或者被删除。假定每个 S_i 大小都是相同的比特数。在每一个时钟周期，编码器产生一个 K 阶随机二进制矩阵 (G_{kn}) ，编码器的输出由 S_i 异或而成，**参加运算的 S_i 对应的 $G_{kn}=1$** ，即 $t_n = \sum_{k=1}^K S_k G_{kn}$

这个和可连续的由不同的 S_i 异或而来，可以对**每 K 个随机 t_n 在一个半无限的二进制矩阵中定义一个新的列**，如图所示。



Page 364



Page 367

(1) 如果 $N < K$ ，接收端就没有接收到足够的码元，不能恢复源文件。

- 如果 $N = K$ ，则接收端有可能恢复源文件。如果矩阵 G 可逆（二进制），就可以计算出 G 的逆矩阵 G^{-1} ，就能恢复 S_i ，即 $S_k = \sum_{n=1}^K t_n G_{nk}^{-1}$ 。此时数据恢复的概率为 $(1-2^{-8})(1-2^{-(K-1)}) \dots (1-1/8)(1-1/4)(1-1/2)$ ，当 $K > 10$ 时，概率约为0.289。
- 如果 $N > K$ ，令 $N = K + E$ ， E 是多接收到的很小量的码元。令此时数据恢复的概率为 $1-e$ ， e 为接收端接收到额外的码元时无法恢复的概率。对于任何 K ，无法恢复源文件的概率 $\alpha(E) \leq 2^{-E}$ 。总的来说，恢复源文件的概率为 $1-e$ 时，需要接收的数据包数目大概是 $K + \log_2(1/e)$ ，对一个数据包编码的代价是需要 $K/2$ 次数据包运算，解码的代价是对随机矩阵 G 求逆的代价(大概 K^2 次二进制运算)和对接收的数据包对应的随机矩阵求逆的代价(大概是 $K^2/2$ 次数据包运算)之和。

当一个随机码对删除信道而言并不是技术上的最佳码，在接收端接收到 K 个数据包时，只有0.289的概率恢复源文件，这几乎可以认为是最好的了。但当接收到额外的 E 个数据包时，无差错的恢复源文件的概率就至少为 $1-e$ ，其中 $e=1-2^{-E}$ 。因此，随着源文件大小的增加，随机线性喷泉码能够无限地接近香农理论的极限。唯一的缺点是随机线性喷泉码的编码和解码的代价都是二次方，编码时还会出现三次方。当 K 比较小时，这并不重要，但若 K 很大时，编码和解码的代价就会很大。因此，我们希望更小的编码和解码代价来解决问题。

Page 368

将长为 n 的文件分割成 $k=n/w$ 个等长的输入符号 s_k (每个输入符号的长为 w)，每个数据包称为输入符号，每个编码包为输出符号。若一个输出符号结点的度为 d ，定义LT码的度分布 $p(d)$ ($d \geq 1$)为 d 的概率分布。编码步骤如下：

- 编码器首先在 $1-k$ 范围内按某一分布 p (称为编码度分布)**随机选取一个整数 d** ，其中 k 称为该码的码长， d 称为编码分组的度；
- 在所有输入符号中随机均匀地选取 d 个符号**作为该编码符号的邻节点符号；
- 将 d 个数据进行异或并输出**，即对这 d 个输入符号求异或得到一个编码符号 t_n 。

不断重复上面的步骤，就可以得到无限多个编码分组。

在这些步骤中一些细节需要注意：**LT码编码之前要先确定度分布函数**，度值的确定由度分布函数来计算。不同的度分布， k 的概率值是不同的，但**度分布概率函数只是确定了取度值为 d 的概率是多少，而不能确定这 d 个原始编码分组是哪些**，所以**对于这 d 个原始编码分组的选择来说是任意的**，这里对这 d 个原始编码分组的选择采用均匀分布来进行选取，即从区间 $[1, k]$ 中按均匀分布随机生成 d 个整数。区间 $[1, k]$ 中所有整数被选取的概率是相等的。把这 d 个整数进行异或，就得到了一个编码分组。

Page 369

随机度 d 的确定(弧波分布)：

$$p(d) = \begin{cases} \frac{1}{k} & d = 1 \\ \frac{1}{d(d-1)} & d = 2, 3, \dots, k \end{cases} \quad (1)$$

根据度分布函数确定编码时的度(参与编码的源文件的个数)。

假设 $k=100$ ，则由弧波分布可知：

度为1的概率为1/100，度为2的概率为1/2，...， d 为100的概率为1/9900。

在 $[1, 10000]$ 区间上划分100个区间：

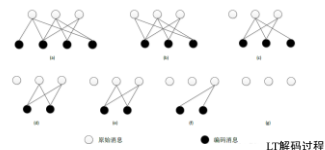
度	取该度的概率	取该度的空间	S_k	取该度的概率	取该度的空间
1	1/100	(0,100]	S_1	1/100	(0,100]
2	1/2	(100,5100]	S_2	1/100	(100,200]
3	1/6	(5100,6766]	S_3	1/100	(200,300]
...	...	...	...	...	...
100	1/9900	(9999,10000]	S_{100}	1/100	(9900,10000]

在0到10000之间随机产生 d 个数，看这 d 个数落在哪个区间，选取这些区间所对应的 s_k 进行编码。

Page 370

LT码的解码采取**迭代法(BP算法)**。在解码的每一步，解码器都在**编码符号集合中寻找度为1的符号，从而直接恢复出它们所连接的输入符号**，然后将这些输入符号与跟它们有连接的编码符号进行异或并将结果取代该编码符号原来的值，完成之后删去它们之间的连接关系，并相应地修改编码符号的度。重复上述过程直至不存在度为1的符号为止。

如果所有输入符号都被恢复则解码成功，否则解码失败。



Page 371



LT 码的解码

具体步骤是:

(1)如图(a)所示,接收一定数量的编码分组,根据编码分组间的对应关系(可以通过分组头部等方式显式传递,也可以通过事先约定伪随机序列等方式隐式传递)建立双向图。顶层节点代表原始分组,底层节点代表编码分组,连接它们的边代表该原始分组是该编码分组的模二和分量。

(2)任意选取一个度数为1的编码分组。如果不存在,则译码停止;如果存在,则通过简单的复制运算,即可恢复与之相连的唯一原始分组,如图(b)所示。

(3)将已经恢复的原始分组模二和到与其相连的所有其他编码分组中,消除其在这些编码分组中的模二和分量。相应地,将双向图中对应的边删除,使得这些编码分组的度数减1。如图(c)所示。如果某个编码分组的度数减小为1,则称该编码分组被“释放”,如图(c)最右侧的编码分组。

(4)重复步骤(2)和(3),直至译码停止。如果所有原始分组都已经恢复,则译码成功;否则,译码失败,必须接收更多的编码分组才能继续译码。

一般来说,为了保证所有数据均能恢复出来,度的选择期望值必须高于 $O(\log n)$,舍弃编码过程中关于计算量的考虑。

Page 372



LT 码的特点

LT 码	
编码时间复杂度	$O(n \log n)$
解码时间复杂度	$O(n \log n)$
编码率	低

LT码的编解码时间复杂度均为 $O(n \log n)$ 。另外,LT码因为随机性,编码率并不确定,不过确定的是其编码效率较低下。

Page 373



Raptor 码

产生原因:

LT码生成的编码包中有少量连接度很高的包,这些高连接度包的存在消耗了很多编译码异或操作,同时也降低了低连接度包的比例,从而减小了译码过程中可译集的大小,降低了译码成功率。

基本思想:

Raptor码提出采用两步编码的方式。首先对原始信息用一个分组码进行预编码,然后采用一个弱化的LT码对数据进行编码并发送。

课堂思政

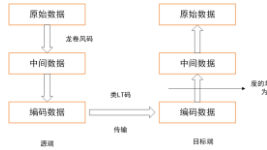
Raptor码是在LT码的基础上发展出来的改进版,Raptor码的出现告诉我们有问题不可怕,改正就是了。正如党的十八大以来习总书记反复强调的“直面问题、解决问题”的思想,并且敢于直面问题、勇于修正错误使我们党的显著特点和优势。

Page 374



Raptor 码的编解码

如下图所示,通过预编码过程,首先采用某种传统的编码方法(图中采用龙卷风码编码方法,如同Raptor码的命名Rapid-Tornado所暗示,这里一般采用的就是龙卷风码),将原始数据编码成为中间编码校验单元。然后将这些中间编码校验单元作为LT码的输入符号,进行编码。



与编码过程相对应地,解码时只需要利用LT码解码技术恢复所有的中间编码校验单元,再根据中间编码的获得方式,使用传统编码的解码方式得到源数据包。

Page 375



Raptor 码的特点



Page 376



EC 码的应用

替代RAID

RAID的目的是在硬盘驱动器(HDD)失效时保护数据,它里面的数据可通过奇偶校验或镜像副本重新创建,但电子机械式硬盘发生故障的几率很高。Erasure code 存储消除了所有RAID问题。2009年Google在第二代GFS中使用了RS(6,3)编码,Facebook早期也在HDFS中使用了RS(10,4)编码。

各大互联网公司的存储服务

例如Google在GFS中使用了RS(6,3)编码,Facebook在HDFS中使用了RS(10,4)编码,Microsoft在Azure中使用了LRC(12,2,2)编码,阿里云在盘古中使用了RS(8,3)编码。

应用在AWCloud中

AWCloud中利用FPGA技术提高了纠删码的性能。

Page 377

- 从数据类型的角度而言，**归档数据适合使用纠删码**。首先，归档数据需要保存相当长的一段时间，一份、两份或多份数据出现问题的可能性会急剧上升；其次，归档数据的写入操作很少，使得纠删码可以很轻易的恢复数据，并且纠删码的额外开销很少。如果没有故障发生，纠删码对读取操作通常不会有任何影响。
- 从应用的角度来看，对于读写都很繁忙的应用，使用纠删码并不是一个很好的选择。但对于**只有大量读取的应用**，纠删码可以提供强大的可靠性，并且在恢复数据的时候只产生很小的额外开销，因为纠删码的额外开销主要来自写操作。

——来自TechTarget中国原创内容中Miller的回答

Page 378

- 评判纠删码的**第一条标准是性能**：即部署了一种纠删码后写入速度有多快，进行数据恢复时的读取速度有多快。
- 另一个标准是**纠删码如何将数据单元分割**：可以是整盘的分割，可以是卷级的分割，也可以是部分盘或部分卷的分割等等。**分割时还会遇到的一个问题是纠删码单元的数量有多少**，有10个数据单元和5个纠删码单元组成的纠删码系统，也有12个数据单元和4个纠删码单元组成的纠删码系统。但大部分纠删码产品都能配置数据单元和纠删码单元的个数。
- 最后需要考虑的是**某些纠删码类型会带来一些问题**：比如在部署之前需要了解每个厂商的纠删码是如何工作的，它们能在什么样的场景恢复数据故障。

——来自TechTarget中国原创内容中Miller的回答

Page 379

- 在某些领域，纠删码会在未来掀起一波浪潮，特别是在**存储归档领域纠删码是必备的**。因为当把数据保存10年甚至20年时会经历各种故障并将它们恢复，并且在数据归档中，需要经常读取数据，但不会产生额外的性能开销。
- 此外，未来纠删码是否会成为潮流**取决于设备容量大小和读写速率的相对速度提升差异**。当存储容量的增速大于读写速率的增速时，纠删码的作用就更能体现，这就是为什么从90年代中期的RAID5走到了今天的RAID6。

——来自TechTarget中国原创内容中Miller的回答

Page 380

灵活的设定副本数量，从而在数据持久性和成本间做出权衡，满足不同的应用场景。比如储存冷数据时，可将副本数缩小至1.2，即冗余数据是0.2。

Erasure code在**面对自然灾害或技术故障时具有很好的恢复能力**。另外，Erasure code允许同时发生多种故障。

Erasure code有**内置的数据安全性**。Erasure code使用的**数据块可以分布在城市、地区、一个数据中心或全球任何地方的不同存储位置**。

Page 381

纠删码技术提高存储资源利用率的表现□：提供近似三副本的可靠性

纠删码技术提高存储资源利用率的表现□：减小冗余存储□

纠删码技术的代价1：□□□计算量□

纠删码技术的代价□：网络负载□

EC码的优化——LRC

Page 382